# 핸즈온 사전 준비 가이드

## Build Your Own AI Office with Gemma 4

| | |
|---|---|
| 세션명 | Build Your Own AI Office with Gemma 4 |
| 진행자 | 박제창 |
| 행사 | Build with AI Seoul 2026 |
| 트랙 | Hands-On A |
| 진행 시간 | 13:30 ~ 14:30 (60분) |
| 수록 범위 | docs/01-19 전체 문서 원문 |
| 기준 확인일 | 2026-05-02 |

이 PDF는 저장소의 01-19번 사전 준비 문서를 순서대로 하나의 배포본으로 묶은 버전입니다.

표지 다음 페이지부터 각 문서의 본문을 전체 수록합니다.

# 한장 요약: 행사 전 준비

행사 당일에는 설치와 모델 다운로드를 시작하지 않는 것을 기준으로 준비합니다. 아래 항목 중 본인 환경에 맞는 경로를 선택하고, 모델 다운로드와 1회 실행 테스트까지 끝내 주세요.

### 가장 중요한 기준

하나만 설치한다면 LM Studio를 우선 권장합니다. ChromeOS 또는 Intel Mac은 Ollama를 우선 고려하세요.

8GB 장비는 E2B only, best-effort입니다. 가능하면 16GB 이상 장비로 참석해 주세요.

## 설치해야 하는 소프트웨어와 LLM

| 내 상황 | 설치 소프트웨어 | 미리 받을 LLM | 완료 기준 |
|---|---|---|---|
| 처음 준비하는 경우 | LM Studio | Gemma 4 E4B 권장, 8GB는 E2B | 앱 실행, 모델 다운로드, Chat 1회 성공 |
| CLI/API도 필요한 경우 | LM Studio + Ollama | `gemma4:e2b` 또는 `gemma4:e4b` | `ollama run` 1회 성공 |
| ChromeOS / Intel Mac | Ollama | `gemma4:e2b` 우선 | Linux 환경 또는 터미널에서 1회 실행 |
| 고급 에이전트 실습 | LM Studio 또는 llama.cpp + OpenCode/Hermes | 32GB+는 26B A4B, 36GB+는 31B 가능 | local endpoint와 에이전트 연결 확인 |
| Gemini CLI Gemma 실험 기능 확인 | Gemini CLI | Preview는 Gemma 4 모델 선택, setup은 Gemma 3 라우터 | `gemini --model gemma-4-26b-a4b-it`, `gemini gemma setup` |

## 메모리별 LLM 선택

| 노트북 메모리 | 권장 LLM | 권장 도구 | 준비 메모 |
|---|---|---|---|
| 8GB | Gemma 4 E2B만 | LM Studio 또는 Ollama | 속도 저하 가능, 다른 앱 종료 |
| 16GB | Gemma 4 E4B | LM Studio 우선 | 가장 무난한 참가 사양 |
| 32GB | E4B 또는 26B A4B | LM Studio, 필요 시 llama.cpp | 에이전트 실습은 26B A4B 권장 |
| 36GB+ | 26B A4B 또는 31B | LM Studio 또는 llama.cpp | 품질 우선은 31B, 반응성 우선은 26B A4B |

## 공통 최종 체크리스트

- 설치 권한 또는 관리자 권한 확인
- 디스크 여유 공간 최소 20GB, 가능하면 40GB 이상 확보
- 선택한 소프트웨어 설치 완료
- 사용할 Gemma 4 모델 다운로드 완료
- 행사 전 최소 1회 응답 생성 테스트 완료
- 실습 당일 전원 어댑터 지참

# 수록 문서

아래 문서를 순서대로 모두 포함했습니다. 각 문서는 다음 페이지부터 원문 내용을 전체 수록합니다.


- 01. 하드웨어 / 운영체제 / 모델 선택 가이드
- 02. Windows 빠른 준비 가이드
- 03. 메모리 기준으로 Gemma 4 모델 고르는 방법
- 04. GGUF, MLX, llama.cpp 개념 설명
- 05. LM Studio 설치 가이드
- 06. Ollama 설치 가이드
- 07. llama.cpp 설치 가이드
- 08. Apple Silicon + MLX 안내
- 09. Gemma 4 벤치마크와 코딩 에이전트 기대치
- 10. OpenCode 설치 및 LM Studio 연동 가이드
- 11. Pi 경량 코딩 에이전트 대안과 이번 핸즈온 선택 이유
- 12. Hermes Agent란 무엇인가
- 13. Hermes Agent 설치 가이드
- 14. Gemma 4 아키텍처 상세 정리
- 15. Gemini CLI Gemma 4 Preview 및 Gemma 라우팅 사전 준비 가이드
- 16. 트러블슈팅 / 최종 체크 / 참고 링크
- 17. 자주 묻는 질문
- 18. uv 설치 가이드
- 19. 코드 편집기 설치 가이드: Visual Studio Code / Google Antigravity


### 편집 방침

세션명과 진행자명은 표지에 반영했습니다.

기존 요약 위주의 본문은 제거하고, 각 마크다운 문서의 전체 내용을 넣도록 생성 방식을 바꿨습니다.

문서 내부의 상대 링크는 PDF 안에서는 링크 라벨 텍스트로 정리해 표시합니다.

문서 01

docs/01-hardware-and-model-selection.md

# 하드웨어 / 운영체제 / 모델 선택 가이드

## 이 문서에서 다루는 내용

- 어떤 도구를 기본으로 설치할지
- 운영체제별 권장 경로
- 8GB, 16GB, 32GB, 36GB 메모리별 권장 모델

- 행사 전에 어떤 모델을 미리 받아야 하는지

추가 참고:

- 코딩 에이전트 기대치와 모델별 벤치마크는 Gemma 4 벤치마크와 코딩 에이전트 기대치 문서를 확인해 주세요.
- 메모리로 어떤 모델을 골라야 하는지는 메모리 기준으로 Gemma 4 모델 고르는 방법 문서를 확인해 주세요.
- OpenCode를 개발자 에이전트처럼 연결해 보고 싶다면 OpenCode 설치 및 LM Studio 연동 가이드 문서를 확인해 주세요.
- Hermes Agent가 정확히 어떤 도구인지 먼저 이해하고 싶다면 Hermes Agent란 무엇인가 문서를 확인해 주세요.
- Gemini CLI에서 Gemma 4 preview 모델 선택이나 Gemma 라우팅 실험 기능을 미리 확인해 보고 싶다면 Gemini CLI Gemma 4 Preview 및 Gemma 라우팅 사전 준비 가이드 문서를 확인해 주세요.

## Ollama와 LM Studio 비교

| 항목 | Ollama | LM Studio |
|---|---|---|
| 사용하기 좋은 경우 | 개발자, CLI/API 선호 | 개발자, 비개발자 모두 / GUI 선호 |
| 설치 방식 | 앱 + CLI 중심 | 데스크톱 앱 중심 |
| 모델 받기 | `ollama pull` 또는 `ollama run` | Discover 탭에서 검색/다운로드 |
| 사용 방식 | 터미널, 로컬 API, 자동화에 강함 | 채팅 UI, 모델 관리 UI, 로컬 서버에 강함 |
| 오프라인 사용 | 모델 다운로드 후 로컬 실행 | 모델 다운로드 후 완전 오프라인 사용 가능 |
| macOS 지원 | Apple Silicon 지원, Intel Mac은 CPU 전용 | Apple Silicon만 지원, Intel Mac 미지원 |
| Windows 지원 | 네이티브 앱, NVIDIA/AMD GPU 지원 | x64/ARM 지원, 16GB+ RAM 권장 |
| ChromeOS | Linux 개발 환경에서 가능하지만 느릴 수 있음 | 공식 권장 경로 아님 |
| 행사 권장도 | 보조 또는 고급/CLI 경로 | 기본 권장 |

## 운영체제별 빠른 추천

| 운영체제 | 가장 권장 | 대안 | 비고 |
|---|---|---|---|
| Windows | LM Studio | Ollama | 하나만 설치한다면 LM Studio를 권장합니다. |
| macOS Apple Silicon | LM Studio | Ollama | LM Studio는 Apple Silicon 전용이며, GGUF를 기본으로 쓰고 필요하면 MLX도 활용할 수 있습니다. |
| macOS Intel | Ollama | 없음에 가까움 | LM Studio 데스크톱 앱은 Intel Mac 미지원입니다. |
| Linux (Ubuntu) | LM Studio | Ollama | GUI 설치가 가능하면 LM Studio를 우선 권장합니다. |
| ChromeOS | Ollama | 가능하면 다른 노트북 준비 | Linux 개발 환경에서는 로컬 LLM용 전용 GPU/VRAM 가속을 기대하기 어렵습니다. |

## 메모리별 권장 모델

아래 권장은 행사 당일 안정적인 진행을 기준으로 잡았습니다. 모델 페이지의 표시 크기나 최소 메모리는 더 낮게 보일 수 있지만, 실제로는 운영체제와 브라우저도 메모리를 함께 사용하므로 여유가 필요합니다.

| 노트북 메모리 | 행사 권장 모델 | 권장 도구 | 안내 |
|---|---|---|---|
| 8GB | `gemma4:e2b` / `google/gemma-4-e2b` | E2B만 시도 | 8GB는 공식 권장 사양보다 낮아 LM Studio/Ollama 어느 쪽도 무난한 기본 조합으로 보기 어렵습니다. E2B만 시도하고, 가능하면 16GB 이상 노트북을 권장합니다. 실제로는 속도가 많이 느리거나 시스템이 일시적으로 멈춘 것처럼 보일 수 있습니다. |
| 16GB | `gemma4:e4b` / `google/gemma-4-e4b` | LM Studio 우선 | 가장 무난한 참가 사양입니다. 실패 시 E2B로 낮추면 됩니다. |
| 32GB | `gemma4:e4b`, `gemma4:26b` / `google/gemma-4-26b-a4b` | LM Studio 우선 | 속도와 안정성을 우선하면 E4B, 더 큰 모델을 원하면 26B A4B를 선택하세요. |
| 36GB | `gemma4:26b`, `gemma4:31b` / `google/gemma-4-31b` | LM Studio 우선 | 31B도 가능하지만, 반응 속도는 26B A4B가 더 나을 수 있습니다. |

## 모델 이름의 B는 무엇인가요?

B는 `Billion`, 즉 10억 개를 뜻합니다.

- E2B: 대략 20억 개 규모로 이해하면 됩니다.
- E4B: 대략 40억 개 규모로 이해하면 됩니다.
- 26B: 대략 260억 개 규모입니다.
- 31B: 대략 310억 개 규모입니다.

여기서 세는 것은 파라미터(parameter)입니다. 파라미터는 모델이 학습하면서 조정한 숫자 값입니다. 모델은 이 숫자들을 사용해서 다음 token을 예측하고, 질문에 답하고, 코드를 작성합니다.

쉽게 말하면:

- B는 모델의 내부 숫자 규모입니다.
- GB는 파일 크기나 메모리 용량 단위입니다.
- 26B라고 해서 파일 크기가 26GB라는 뜻은 아닙니다.
- 숫자가 클수록 대체로 더 똑똑해질 가능성이 있지만, 더 많은 메모리와 계산이 필요합니다.

실제 다운로드 크기와 실행 중 메모리는 양자화 방식, 실행 도구, 컨텍스트 길이, 이미지 입력 여부에 따라 달라집니다. 그래서 모델 이름의 B와 다운로드 크기는 1:1로 맞지 않습니다.

보통 사용자가 LM Studio나 Ollama에서 받는 것은 원본 BF16 가중치 전체가 아니라 로컬 실행에 맞게 양자화된 배포본입니다. 예를 들어 4-bit 양자화는 파라미터 1개를 대략 0.5 byte 수준으로 저장하므로, BF16 기준보다 훨씬 작은 파일로 받을 수 있습니다.

## A4B는 무슨 뜻인가요?

26B A4B에서 26B는 모델 전체 파라미터 규모를 뜻하고, A4B는 답을 만들 때 실제로 계산에 참여하는 부분이 약 4B 규모라는 뜻입니다.

이 방식은 `MoE(Mixture-of-Experts)` 구조라고 부릅니다. 모델 안에 여러 "전문가" 부분이 있고, 답을 만들 때마다 필요한 일부 전문가만 골라 계산하는 방식입니다. 공식 모델 카드 기준으로는 총 파라미터가 `25.2B`, 실제 계산에 참여하는 파라미터가 `3.8B`입니다.

그래서 26B A4B는 31B처럼 모든 주요 계산 블록을 함께 쓰는 모델보다 더 빠르게 느껴질 수 있습니다. 다만 모델 파일은 전체 26B 규모를 메모리에 올려야 하므로, 4B 모델처럼 가볍게 실행된다는 뜻은 아닙니다. 32GB 이상 장비에서 권장하는 이유가 이 점 때문입니다.

8GB 추가 메모:

- Windows와 macOS에서 LM Studio 공식 권장 메모리는 16GB 이상입니다.
- 따라서 8GB는 "무난한 기본 조합"이 아니라, 작은 모델(E2B)만 시도하는 최소 준비로 이해해 주세요.
- 8GB Apple Silicon Mac도 E2B/E4B 실행 중 속도가 크게 떨어지거나 시스템이 멈춘 것처럼 보일 수 있으므로, 안정적인 실습 장비로 보기 어렵습니다.
- 8GB 장비로 참석하는 경우에는 작은 컨텍스트, 브라우저 탭 최소화, 다른 앱 종료를 전제로 준비해 주세요.

## 로컬 모델 크기 참고

LM Studio Gemma 4 모델 페이지 기준 표시 크기:

- `google/gemma-4-e2b`: 4.20GB
- `google/gemma-4-e4b`: 5.90GB
- `google/gemma-4-26b-a4b`: 17.00GB
- `google/gemma-4-31b`: 19.00GB

Ollama Gemma 4 태그 페이지 기준으로는 기본 태그가 `gemma4:e2b` 7.2GB, `gemma4:e4b` 9.6GB, `gemma4:26b` 18GB, `gemma4:31b` 20GB로 표시됩니다.

이번 핸즈온 권장은 위 표시 크기보다 여유 있게 잡았습니다. 모델 크기 외에도 실행 도구, 긴 대화 컨텍스트, 브라우저, 운영체제가 메모리를 함께 사용합니다.

## 어떤 모델을 미리 받아두면 좋을까?

행사 전에 미리 받아 둘 모델은 아래처럼 고르면 됩니다.

중요:

- 이번 핸즈온은 채팅 / 질의응답 / 에이전트 흐름이 중심이므로 instruction-tuned (`-it`) 또는 chat-ready 모델을 받으세요.
- Hugging Face 공식 Gemma 4 컬렉션도 같은 크기에서 `google/gemma-4-26B-A4B`와 `google/gemma-4-26B-A4B-it`처럼 base 모델과 instruction-tuned 모델을 나눠 제공합니다.
- 이번 준비에서는 `google/gemma-4-E2B-it`, `google/gemma-4-E4B-it`, `google/gemma-4-26B-A4B-it`, `google/gemma-4-31B-it`처럼 `-it`가 붙은 쪽을 기준으로 보면 됩니다.
- `base`나 `pretrained`로 표시된 모델은 기본 준비용으로 권장하지 않습니다. 모델을 연구하거나 파인튜닝을 시작할 때는 의미가 있지만, 바로 대화하고 지시를 수행하는 실습에는 덜 적합합니다.
- Ollama는 `gemma4:e2b`, `gemma4:e4b`, `gemma4:26b`, `gemma4:31b` 같은 런타임 태그를 쓰므로, 별도로 `-it` 접미사를 고를 필요는 없습니다.

- LM Studio는 모델 목록 이름이 `google/gemma-4-e4b`처럼 더 짧게 보일 수 있으므로, 채팅/실습용 instruction-tuned 또는 chat-ready 항목인지 설명을 함께 확인해 주세요.

- 8GB: E2B만 미리 다운로드
- 8GB는 현장 안정성이 낮으므로, 가능하면 16GB 이상 장비로 변경
- 16GB: E4B 다운로드, 불안하면 E2B도 함께 준비
- 32GB: E4B 또는 26B A4B
- 36GB: 26B A4B 또는 31B

중요:

- 행사 당일에 모델 다운로드를 시작하지 마세요.
- 모델 파일이 커서 네트워크 상황이 좋지 않으면 실습 시간 대부분을 다운로드에 쓰게 될 수 있습니다.
- 최소한 설치, 모델 다운로드, 1회 실행 테스트까지 끝낸 상태로 참석해 주세요.

## 권장 준비 시나리오

### 가장 안전한 시나리오

- Windows/macOS/Linux 사용자: LM Studio 설치 + chat-ready / instruction-tuned `google/gemma-4-e2b` 또는 `google/gemma-4-e4b` 다운로드
- 추가로 CLI/API 실습도 원하면 Ollama까지 설치
- 공통 필수 조건: 행사 전 모델 다운로드와 실행 테스트까지 완료

### 메모리별 추천

- 8GB: 가능하면 다른 장비 권장. 꼭 가져오면 E2B만 시도, 속도 저하 감안
- 16GB: LM Studio + chat-ready / instruction-tuned `google/gemma-4-e4b`
- 32GB: LM Studio + chat-ready / instruction-tuned `google/gemma-4-e4b` 또는 `google/gemma-4-26b-a4b`
- 36GB: LM Studio + chat-ready / instruction-tuned `google/gemma-4-26b-a4b` 또는 `google/gemma-4-31b`
- ChromeOS: Ollama + `gemma4:e2b`

### 가장 비추천하는 조합

- 8GB Chromebook + 큰 모델
- Intel Mac + LM Studio
- 행사 당일 현장에서 처음 모델 다운로드

### Gemini CLI Gemma 기능은 별도 선택 경로

Gemini CLI의 Gemma 기능은 현재 두 가지로 나눠서 봐야 합니다.

- `v0.41.0-preview.0` 이상 preview: `experimental.gemma`를 켜면 Gemma 4 모델 선택을 실험적으로 확인할 수 있습니다.

- `gemini gemma` / `gemini gemma setup`: LiteRT-LM으로 Gemma 3 기반 `gemma3-1b-gpu-custom`을 내려받아 로컬 라우팅 판단에 씁니다.

즉, preview에서 Gemma 4 모델을 선택할 수 있어도 `gemini gemma setup`이 Gemma 4 모델을 로컬로 내려받는다는 뜻은 아닙니다. 또한 `/model` 목록에 Gemma 4가 보여도 실제 호출 권한은 계정, API 키, 조직의 Preview Release Channel 설정에 따라 막힐 수 있습니다.

따라서 행사 기본 준비는 여전히 아래 중 하나를 먼저 끝내세요.

- LM Studio에서 Gemma 4 모델 다운로드 및 채팅 1회 성공
- Ollama에서 `gemma4:e2b` 또는 `gemma4:e4b` 실행 성공

Gemini CLI를 이미 쓰고 있거나 CLI 실험 기능까지 확인하고 싶은 경우에만 Gemini CLI Gemma 4 Preview 및 Gemma 라우팅 사전 준비 가이드를 추가로 보면 됩니다.

## Ollama 태그 관련 주의

- 2026-04-27 기준 Ollama Gemma 4 태그 페이지에는 29개 태그가 표시됩니다.
- `gemma4:latest`는 현재 9.6GB, 128K 컨텍스트의 E4B 계열로 표시됩니다.
- `latest`는 향후 바뀔 수 있으므로, 사전 준비용으로는 `gemma4:e2b`, `gemma4:e4b`, `gemma4:26b`, `gemma4:31b`처럼 명시적 태그를 사용하세요.
- `*-cloud`, `*-mlx-*`, `*-mxfp8`, `*-nvfp4`, `*-q8_0` 같은 변형 태그는 목적을 알고 선택할 때만 사용하세요.
- 도구별 배포 포맷이 달라 다운로드 크기와 표시 용량은 서로 다를 수 있습니다.

문서 02

docs/02-windows-guide.md

# Windows 빠른 준비 가이드

## 이 문서는 누구를 위한 문서인가요?

- Windows 노트북으로 이번 핸즈온에 참가하는 분
- Apple Silicon Mac이 아니라 일반적인 `Windows x64` 또는 `Windows ARM` 장비를 쓰는 분
- 어떤 도구를 설치해야 할지 빠르게 결정하고 싶은 분

## 먼저 결론

- Windows에서는 LM Studio부터 준비하는 것이 가장 무난합니다.
- CLI/API도 같이 써 보고 싶으면 Ollama를 추가로 설치하면 됩니다.
- `llama.cpp`는 터미널 사용에 익숙한 경우에만 추가로 보면 됩니다.
- `Hermes Agent`는 Windows 네이티브가 아니라 WSL2 안에서만 공식 지원이므로, 처음 준비할 때는 뒤로 미뤄도 됩니다.

즉 가장 현실적인 준비 순서는 아래입니다.

1. 가장 쉬운 준비: `LM Studio`
2. CLI까지 써 보고 싶은 경우: `LM Studio + Ollama`
3. Hermes까지 해 보고 싶은 경우: LM Studio 또는 llama.cpp + WSL2 안의 Hermes

## 어떤 도구를 설치하면 되나요?

### 방법 A: 가장 쉬운 준비

설치:

- `LM Studio`

이런 경우에 적합합니다:

- GUI가 편한 분
- 로컬 모델을 한 번 실행해 보는 것이 우선인 분
- 핸즈온 현장에서 가장 안정적인 방법을 원하는 분

### 방법 B: LM Studio + CLI/API

설치:

- `LM Studio`
- `Ollama`

이런 경우에 적합합니다:

- GUI와 터미널 둘 다 써 보고 싶은 분
- 나중에 로컬 API를 붙여 보고 싶은 분

- 발표 도중 CLI 예시도 따라가 보고 싶은 분

### 방법 C: 고급 에이전트 실습

설치:

- `LM Studio` 또는 `llama.cpp`
- `WSL2`
- `Hermes Agent`는 WSL2 안에 설치

이런 경우에 적합합니다:

- `Hermes + Gemma 4` AI office 데모를 직접 해 보고 싶은 분
- 로컬 OpenAI 호환 서버에 에이전트를 연결해 보고 싶은 분

## 추천 조합

| 목표 | 추천 조합 | 비고 |
|---|---|---|
| 가장 쉽게 준비 | `LM Studio` | 먼저 이것부터 준비 |
| GUI + CLI 둘 다 | `LM Studio + Ollama` | 가장 무난한 추가 준비 |
| 직접 서버 띄우기 | `llama.cpp` | 터미널에 익숙할 때만 |
| Hermes까지 실습 | LM Studio 또는 llama.cpp + WSL2 + Hermes | 시간이 충분할 때만 |

## 메모리 기준 추천

| 메모리 | 추천 모델 | 권장 도구 |
|---|---|---|
| 8GB | E2B만 시도 | LM Studio 우선 |
| 16GB | E4B | LM Studio 우선, 필요하면 Ollama 추가 |
| 32GB | E4B 또는 26B A4B | LM Studio, Ollama, llama.cpp 선택 가능 |
| 36GB+ | 26B A4B 또는 31B | Hermes나 llama.cpp도 고려 가능 |

중요:

- 8GB Windows 노트북은 가능한 경우 다른 장비를 권장합니다.
- 16GB가 가장 무난한 최소 기준에 가깝습니다.
- 발표자 데모용으로는 32GB 이상이 훨씬 여유롭습니다.

## Windows에서 가장 먼저 해야 할 일

### 1. 쉬운 준비부터 끝내기

먼저 아래 중 하나 이상을 반드시 끝내세요.

- `LM Studio` 설치 + Gemma 4 실행 성공
- `Ollama` 설치 + `ollama run gemma4:e2b` 성공

- `llama.cpp` 설치 + `llama-server` 실행 성공

### 2. 모델도 미리 받아 두기

행사 당일에 모델을 처음 다운로드하지 않는 것이 중요합니다.

권장:

- 8GB: E2B
- 16GB: E4B
- 32GB+: E4B 또는 26B A4B

### 3. Hermes는 나중에

Windows에서는 `Hermes`를 가장 먼저 설치하기보다:

1. 모델이 실제로 한 번 도는지 먼저 확인
2. 그다음 필요하면 WSL2 + Hermes로 확장

이 순서가 훨씬 안전합니다.

## WSL2는 꼭 필요한가요?

- `LM Studio`: 필요 없음
- `Ollama`: 필요 없음
- `llama.cpp`: 필요 없음
- `Hermes Agent`: 필요함

즉 Windows에서 WSL2는 Hermes를 쓸 때만 필수라고 이해하면 됩니다.

WSL2 설치는 PowerShell 관리자 권한으로 아래처럼 시작하면 됩니다.

```powershell
wsl --install -d Ubuntu
```

설치가 멈추면:

```powershell
wsl --install --web-download -d Ubuntu
```

설치 후 상태 확인:

```powershell
wsl --status
wsl --list --verbose
```

## Windows + Hermes 권장 아키텍처

Windows에서 `Hermes`까지 하고 싶다면 가장 안전한 조합은 아래입니다.

```text
Windows native:
  LM Studio or llama.cpp server

WSL2:
  Hermes Agent
```

즉:

1. Windows에서 모델 서버 실행
2. WSL2 안에서 Hermes 실행
3. Hermes가 `http://127.0.0.1:PORT/v1` 또는 host endpoint에 연결

이 방식이 좋은 이유:

- WSL2 안에서 대용량 모델을 다시 받지 않아도 됨
- Windows GPU/GUI 앱 활용이 쉬움
- Hermes는 공식 지원 방식인 WSL2 안에서 돌릴 수 있음

## 설치 우선순위

### 기본 준비

1. LM Studio 설치 가이드
2. 트러블슈팅 / 최종 체크 / 참고 링크

### CLI도 해 보고 싶은 경우

1. LM Studio 설치 가이드
2. Ollama 설치 가이드
3. 트러블슈팅 / 최종 체크 / 참고 링크

### Hermes까지 해 보고 싶은 경우

1. LM Studio 설치 가이드 또는 llama.cpp 설치 가이드
2. PowerShell 관리자 권한으로 WSL2 준비

```powershell
wsl --install -d Ubuntu
```

1. Ubuntu 실행

```powershell
wsl -d Ubuntu
```

1. Hermes Agent란 무엇인가
2. Hermes Agent 설치 가이드

## 절대 행사 당일에 처음 하면 안 되는 것

- WSL2 첫 설치
- 큰 모델 첫 다운로드
- `Hermes + WSL2 + local server`를 한 번도 테스트하지 않은 상태로 참석
- 8GB 노트북에서 큰 모델을 현장에서 바로 시도

## 행사 전 최종 체크리스트

아래 중 자신이 선택한 준비 방식에 맞는 항목만 확인하면 됩니다.

### LM Studio 준비

- 앱 설치 완료
- Gemma 4 다운로드 완료
- Chat 탭에서 1회 응답 생성 성공

### Ollama 준비

- `ollama --version`
- `ollama pull gemma4:e2b` 또는 `ollama pull gemma4:e4b`
- `ollama run gemma4:e2b` 또는 `ollama run gemma4:e4b`

### llama.cpp 준비

- `llama-cli --version`
- `llama-server --version`
- `llama-server` 또는 `llama-cli`로 1회 실행 성공
- 필요하면 `Invoke-RestMethod http://127.0.0.1:8080/v1/models` 확인

### Hermes 준비

- WSL2 설치 완료
- PowerShell에서 `wsl --status` 확인
- WSL2 터미널에서 `hermes version`
- `hermes doctor`
- 로컬 서버 주소 연결 1회 성공

## 권장 한 줄 요약

- 대부분의 경우: LM Studio만 준비해도 충분
- 조금 더 해 보고 싶으면: Ollama 추가
- AI office / agents까지 직접 하려면: WSL2 + Hermes까지 추가 준비

문서 03

docs/03-memory-based-model-selection.md

# 메모리 기준으로 Gemma 4 모델 고르는 방법

## 이 문서에서 다루는 내용

- 공식 모델 스펙과 로컬 배포 크기 기준 메모리 판단법
- 노트북/데스크톱 메모리로 어떤 모델을 고르면 되는지
- 준비용 간단 계산식
- 8GB, 16GB, 32GB, 36GB 환경별 예시

## 먼저 알아둘 점

- 아래 표는 공식 모델 스펙과 LM Studio / Ollama에 표시되는 로컬 배포 크기를 함께 보는 기준입니다.
- 표시 크기는 다운로드 파일 또는 로컬 배포 패키지 기준이라, 실제 실행 중 최대 메모리 사용량과 항상 같지는 않습니다.
- 운영체제, 실행 도구, 브라우저, 긴 대화를 빠르게 이어가기 위해 저장하는 내부 메모리(KV cache), 긴 컨텍스트, 멀티모달 입력으로 인한 추가 메모리는 별도로 필요합니다.
- 따라서 실제 개인 노트북에서는 표의 숫자보다 여유를 더 잡아야 합니다.
- Apple Silicon Mac처럼 unified memory를 쓰는 환경에서는 전체 메모리를 하나의 풀처럼 보면 됩니다.
- Windows/Linux의 전용 GPU 노트북은 시스템 메모리(RAM)뿐 아니라 GPU 메모리(VRAM) 도 같이 확인해 주세요.

## 로컬 배포 크기와 컨텍스트 참고

2026-04-27 기준 LM Studio 모델 페이지와 Ollama Gemma 4 태그 페이지에 표시되는 기본 항목입니다.

| 모델 | LM Studio 표시 크기 | Ollama 기본 태그 표시 크기 | 컨텍스트 |
|---|---|---|---|
| Gemma 4 E2B | 4.20GB | 7.2GB | 128K |
| Gemma 4 E4B | 5.90GB | 9.6GB | 128K |
| Gemma 4 26B A4B | 17.00GB | 18GB | 256K |
| Gemma 4 31B | 19.00GB | 20GB | 256K |

읽는 법:

- LM Studio와 Ollama는 같은 모델이라도 배포 포맷과 양자화 방식이 달라 표시 크기가 다를 수 있습니다.
- 일반 사용자가 LM Studio나 Ollama에서 받는 로컬 실행용 모델은 보통 원본 BF16 가중치 그대로가 아니라 4-bit, 5-bit, 8-bit 등으로 양자화된 배포본입니다.
- Ollama의 `gemma4:latest`는 현재 E4B 계열로 표시되지만, `latest`는 바뀔 수 있으므로 명시적 태그를 사용하세요.
- 실행 중 메모리는 모델 크기에 더해 컨텍스트 길이, KV cache, 이미지 입력, 실행 도구 설정에 따라 늘어납니다.

## 모델 이름의 B는 무엇인가요?

B는 `Billion`, 즉 10억 개를 뜻합니다. 26B는 대략 260억 개 파라미터를 가진 모델이고, 31B는 대략 310억 개 파라미터를 가진 모델입니다.

파라미터는 모델이 학습하면서 갖게 된 숫자 값입니다. 모델은 이 숫자들을 바탕으로 문장을 이어 쓰고, 질문에 답하고, 코드를 만듭니다.

헷갈리기 쉬운 점:

- B는 모델 내부 숫자 규모입니다.
- GB는 파일 크기나 메모리 용량 단위입니다.
- 26B라고 해서 다운로드 파일이 26GB라는 뜻은 아닙니다.
- 실제 표시 크기는 양자화 방식과 배포 포맷에 따라 달라집니다.

그래서 모델을 고를 때는 이름의 B만 보지 말고, 위 표의 로컬 배포 크기와 실제 노트북 메모리를 같이 봐야 합니다.

## 26B A4B에서 A4B는 무엇인가요?

A4B는 Active 4B 로 이해하면 됩니다. 모델 전체는 26B 규모이지만, 답을 만들 때 실제로 계산에 참여하는 부분은 약 4B 규모라는 의미입니다. 공식 모델 카드 기준 수치는 `3.8B`입니다.

이런 구조를 `MoE(Mixture-of-Experts)`라고 부릅니다. 모델 안에 여러 "전문가" 부분이 있고, 답을 만들 때마다 필요한 일부 전문가만 골라 계산합니다.

중요한 점은 A4B가 메모리 사용량이 4B 모델만큼 작다는 뜻이 아니라는 것입니다. 26B A4B는 계산할 때 일부 전문가만 쓰지만, 빠른 추론을 위해 전체 26B 규모의 모델 파일을 메모리에 올려야 합니다. 그래서 로컬 배포 크기도 E4B보다 훨씬 큽니다.

## 메모리 계획 포인트

- E2B, E4B는 이름에 보이는 크기보다 모델 파일이 더 클 수 있습니다. 작은 모델을 효율적으로 쓰기 위한 추가 구조가 포함되어 있기 때문입니다.
- 26B A4B는 계산할 때 일부만 쓰는 구조이지만, 전체 모델 파일을 메모리에 올려야 하므로 메모리 요구량이 4B 모델처럼 작아지지 않습니다.
- 긴 컨텍스트는 이전 대화 내용을 빠르게 이어가기 위한 내부 저장 공간 때문에 메모리를 더 사용합니다.
- fine-tuning은 inference보다 훨씬 더 많은 메모리가 필요합니다.

## 모델 추론에 필요한 메모리는 어떻게 계산하나요?

모델 추론에 필요한 메모리는 크게 아래 항목을 더해서 생각하면 됩니다.

```text
추론 메모리 ≈ 모델 가중치 + KV cache + 입력/출력 처리 메모리 + 실행 도구 오버헤드
```

이 중에서 가장 큰 부분은 보통 모델 가중치입니다. 가중치는 모델이 학습한 파라미터 숫자들을 말합니다.

가중치 메모리는 아래처럼 대략 계산합니다.

```text
가중치 메모리 ≈ 파라미터 수 x 파라미터 1개당 저장 크기
```

자주 보는 숫자 형식:

| 형식 | 파라미터 1개당 크기 | 26B 모델 가중치만 단순 계산 |
|---|---|---|
| FP32 | 4 bytes | 약 104GB |
| BF16 / FP16 | 2 bytes | 약 52GB |
| 8-bit | 1 byte | 약 26GB |
| 4-bit | 0.5 byte | 약 13GB |

즉 26B 모델을 BF16으로 저장하면 가중치만 약 52GB가 필요하지만, 4-bit 양자화 모델은 가중치 기준으로 약 13GB까지 줄어들 수 있습니다.

```text
26B BF16 ≈ 52GB
26B 4-bit ≈ 13GB
```

여기서 52GB가 13GB가 되는 이유는 2 bytes였던 숫자를 0.5 byte 수준으로 줄이기 때문입니다. 즉 파라미터 수를 줄이는 것이 아니라, 파라미터 하나를 저장하는 크기를 줄이는 것입니다.

다만 실제 추론 메모리는 13GB에서 끝나지 않습니다. 양자화 메타데이터, KV cache, 컨텍스트 길이, 이미지 입력, 실행 도구 오버헤드가 추가됩니다. 그래서 26B 계열 로컬 배포 크기가 17~18GB처럼 보일 수 있고, 실제 실행 중에는 더 많은 여유 메모리가 필요할 수 있습니다.

NVFP4처럼 scale 정보를 함께 쓰는 4-bit floating point 데이터 타입은 실제로 파라미터 1개당 정확히 4-bit만 쓰는 것은 아닙니다. NVIDIA 설명 기준 NVFP4는 4-bit 값에 scale overhead가 붙어 유효 bit 폭이 약 4.5 bits/value에 가깝습니다. 그래도 BF16/FP16보다 훨씬 작기 때문에 큰 모델을 더 작은 GPU 메모리에 올릴 수 있습니다.

다만 "31B NVFP4가 24GB GPU에서 된다"는 말은 짧은 context와 특정 런타임 조건이 붙은 고급 실험으로 봐야 합니다. 긴 256K context, 이미지/비디오 입력, 큰 batch까지 쓰면 KV cache와 activation 메모리가 커져서 24GB를 넘길 수 있습니다.

## 준비용 간단 계산식

아래 계산식은 공식 수치가 아니라 준비용 간단 추정식입니다.

### 1. 사용 가능 메모리 계산

```text
사용 가능 메모리 = 전체 메모리 - 시스템/앱 여유분 - 컨텍스트 여유분
```

권장 기본값:

- 시스템/앱 여유분: 4~6GB
- 컨텍스트 여유분: 1~3GB

여유 있게 잡은 권장값:

```text
사용 가능 메모리 ≈ 전체 메모리 - 6GB - 2GB
```

즉,

```text
사용 가능 메모리 ≈ 전체 메모리 - 8GB
```

이 값보다 모델의 대략적 로드 메모리가 작아야, 현장에서 비교적 안정적으로 쓸 가능성이 높습니다.

## 가장 쉬운 선택 공식

LM Studio를 기준으로 준비할 때는 아래처럼 생각하면 됩니다.

```text
권장 전체 메모리 ≈ 로컬 모델 표시 크기 + 8GB
```

예시:

- E2B: 4.2GB + 8GB ≈ 12.2GB
- E4B: 5.9GB + 8GB ≈ 13.9GB
- 26B A4B: 17GB + 8GB ≈ 25GB
- 31B: 19GB + 8GB ≈ 27GB

Ollama 기본 태그는 같은 모델이라도 표시 크기가 더 클 수 있습니다. 특히 16GB 장비에서 `gemma4:e4b`가 버겁게 느껴지면 `gemma4:e2b`로 낮추세요.

이 계산은 "행사에서 너무 빡빡하지 않게 쓰는 기준"입니다. 즉:

- E2B는 8GB에서는 거의 최소치이며 속도 저하 가능
- E4B는 16GB에서 현실적
- 26B A4B, 31B는 32GB 이상에서 현실적

## RAM만 보면 되나요?

항상 그렇지는 않습니다.

- Apple Silicon Mac: unified memory 구조라서, 보통은 이 문서의 "전체 메모리 기준 계산"을 그대로 적용해도 됩니다.
- Windows/Linux + 전용 GPU: GPU 가속을 쓸 때는 시스템 RAM과 별개로 VRAM이 부족할 수 있습니다.
- 예를 들어 시스템 메모리가 32GB여도 VRAM이 작으면 큰 모델은 GPU에 충분히 못 올라가서 속도가 느려지거나, 일부 오프로딩으로 체감이 달라질 수 있습니다.
- 전용 GPU 사양이 애매하면 더 작은 모델(E2B/E4B) 을 고르세요.

## 메모리별로 어떻게 고르면 되나요?

### 8GB

간단 계산:

```text
사용 가능 메모리 ≈ 8GB - 8GB = 0GB
```

실제로는 아주 빡빡합니다.

해석:

- E2B도 운영체제와 앱이 같이 뜬 상태에서는 여유가 거의 없습니다.
- 따라서 8GB는 E2B만 시도하고, 속도 저하를 감안해 주세요.
- Windows/macOS에서 LM Studio 공식 권장은 16GB 이상이라, 8GB는 "무난한 기본 사양"이 아닙니다.
- 특히 8GB Apple Silicon Mac이나 8GB 일반 노트북에서는 E2B/E4B 실행 시 전체 시스템이 매우 느려지거나 일시적으로 멈춘 것처럼 보일 수 있습니다.

권장:

- E2B만 준비
- 가능하면 16GB 이상 장비 사용
- 브라우저 탭 최소화
- 다른 앱 종료
- 가능하면 전원 연결

## 16GB

간단 계산:

```text
사용 가능 메모리 ≈ 16GB - 8GB = 8GB
```

해석:

- E4B는 LM Studio로 준비할 때 현실적인 선택
- E2B는 더 여유 있음
- 26B A4B, 31B는 사실상 어렵습니다

권장:

- 기본: E4B
- 실패 시: E2B

## 32GB

간단 계산:

```text
사용 가능 메모리 ≈ 32GB - 8GB = 24GB
```

해석:

- E4B는 매우 여유 있음
- 26B A4B는 현실적
- 31B도 가능 범위에 들어오지만, 긴 컨텍스트와 다른 앱까지 생각하면 여유가 줄어듭니다

권장:

- 안정성 우선: E4B
- 성능 우선: 26B A4B
- 실험적 상위 옵션: 31B

### 36GB

간단 계산:

```text
사용 가능 메모리 ≈ 36GB - 8GB = 28GB
```

해석:

- 26B A4B, 31B 모두 현실적
- 31B도 가능성이 높지만, 속도와 반응성은 26B A4B가 더 나은 경우가 있을 수 있습니다

권장:

- 균형형: 26B A4B
- 품질 우선: 31B

## GGUF / MLX / 도구별로 이 표를 어떻게 보면 되나요?

- Ollama / LM Studio GGUF: 보통 4비트~8비트 계열 로컬 모델을 많이 쓰지만, 도구별 표시 크기와 실제 메모리 사용량은 다를 수 있습니다.
- MLX: Apple Silicon에서 잘 맞는 선택지이지만, 실제 메모리 사용량은 모델 변환 방식과 정밀도에 따라 달라질 수 있습니다.
- 즉, 위 표는 준비용 기준점이고, 실제 파일 크기와 런타임 메모리는 도구별로 조금씩 다릅니다.

## 한 줄 정리

- 8GB: E2B만 시도, 속도 저하 감안
- 16GB: E4B
- 32GB: E4B 또는 26B A4B
- 36GB: 26B A4B 또는 31B

## 참고 링크

- Gemma 4 model overview
- Gemma 4 model card
- LM Studio Gemma 4 models
- Ollama Gemma 4 tags
- Google Korean blog: Gemma 4 소개

문서 04

docs/04-gguf-mlx-llamacpp-explainer.md

# GGUF, MLX, llama.cpp 개념 설명

## 이 문서는 누구를 위한 문서인가요?

- `GGUF`, `MLX`, `llama.cpp`가 무엇인지 헷갈리는 분
- LM Studio에서 왜 어떤 모델은 `GGUF`, 어떤 모델은 `MLX`로 보이는지 알고 싶은 분
- Apple Silicon Mac과 Windows/Linux에서 실행 방식이 어떻게 다른지 이해하고 싶은 분

## 먼저 아주 짧게 정리

- `GGUF`: 모델 파일 포맷
- `llama.cpp`: 로컬 LLM 실행 엔진 / 런타임
- `MLX`: Apple Silicon용 머신러닝 프레임워크

즉:

- `GGUF`는 파일
- `llama.cpp`는 그 파일을 실행하는 쪽
- `MLX`는 Apple Silicon에서 모델을 실행하는 다른 경로

## 가장 중요한 오해 하나

`GGUF`, `MLX`, `llama.cpp`는 같은 종류의 단어가 아닙니다.

- `GGUF`는 파일 포맷입니다.
- `llama.cpp`는 추론 엔진입니다.
- `MLX`는 Apple의 프레임워크이자 그 위에서 돌아가는 실행 생태계입니다.

예를 들면:

- 같은 Gemma 4 모델도 `GGUF` 버전이 있을 수 있고
- Apple Silicon에서는 `MLX` 버전이 따로 있을 수 있습니다.

즉, 같은 모델이 여러 실행 포맷/런타임으로 제공될 수 있다고 이해하면 됩니다.

## instruction-tuned와 base 모델은 무엇이 다른가요?

모델을 받을 때 `-it`, `instruct`, `instruction-tuned`, `chat-ready`, `base`, `pretrained` 같은 표현을 보게 됩니다. 이것은 `GGUF`, `MLX`, `Q4`, `NVFP4`처럼 파일 포맷이나 숫자 저장 방식을 말하는 것이 아니라, 모델이 어떤 방식으로 훈련되어 어떤 행동을 하도록 준비되었는지를 말합니다.

Gemma 4 공식 Hugging Face 컬렉션도 같은 크기에서 두 계열을 나눠 제공합니다.

| 계열 | 예시 | 성격 |
|---|---|---|
| base / pre-trained | `google/gemma-4-26B-A4B` | 원본에 가까운 사전학습 모델 |
| instruction-tuned | `google/gemma-4-26B-A4B-it` | 채팅과 지시 수행에 맞게 추가 학습된 모델 |

`base` 또는 `pretrained` 모델은 대량의 텍스트, 코드, 이미지 같은 데이터를 보고 다음 토큰을 예측하도록 학습한 모델입니다. 이 단계에서 모델은 언어와 지식, 코드 패턴, 시각 정보를 폭넓게 배웁니다. 하지만 "질문에 답하기", "요청을 단계적으로 수행하기", "대화 형식을 지키기" 같은 행동을 안정적으로 하도록 별도 조정된 상태는 아닙니다.

그래서 base 모델에 바로 질문을 던지면 답을 하기도 하지만, 질문을 이어 쓰는 식으로 반응하거나, 출력 형식이 흔들리거나, 대화형 앱에서 기대하는 역할 구분을 잘 따르지 못할 수 있습니다. base 모델은 주로 모델 연구, 자체 instruction tuning, 도메인 fine-tuning, 평가 실험을 시작할 때 쓰는 출발점에 가깝습니다.

`instruction-tuned` 모델은 base 모델을 바탕으로 사람이 쓴 지시문과 기대 답변 형식에 더 맞도록 추가 훈련한 모델입니다. Google의 Gemma 실행 문서도 처음 시작할 때는 instruction-tuned(IT) 모델을 추천합니다. IT 모델은 별도 추가 학습 없이도 자연어 요청에 답하기 좋고, 채팅 UI나 로컬 에이전트 흐름에 바로 연결하기 쉽습니다.

차이를 실습 관점에서 보면 아래와 같습니다.

| 구분 | base / pre-trained | instruction-tuned / chat-ready |
|---|---|---|
| 주된 목적 | 모델 연구, 파인튜닝 출발점 | 바로 채팅, 질의응답, 지시 수행 |
| 프롬프트 반응 | 텍스트를 이어 쓰는 성격이 강함 | 요청에 대한 답변을 만들도록 조정됨 |
| 대화 형식 | 직접 템플릿을 맞추거나 추가 튜닝이 필요할 수 있음 | 도구가 제공하는 chat template과 잘 맞음 |
| 이번 핸즈온 적합도 | 기본 준비용으로는 비추천 | 기본 준비용으로 추천 |

중요한 점은 instruction-tuned 여부와 양자화 여부는 서로 다른 축이라는 것입니다.

예를 들어 아래 표현들은 서로 다른 질문에 답합니다.

- `google/gemma-4-26B-A4B-it`: 26B A4B의 instruction-tuned 모델인가?
- `Q4`, `Q5`, `Q8`, `NVFP4`: 숫자를 어떤 크기와 정밀도로 저장했는가?
- `GGUF`, `MLX`, `Safetensors`: 어떤 파일 포맷이나 실행 생태계로 배포되는가?

즉, 같은 instruction-tuned 모델도 GGUF로 변환될 수 있고, 4-bit로 양자화될 수 있습니다. 반대로 base 모델도 GGUF나 양자화 파일로 배포될 수 있습니다. 이번 핸즈온에서는 먼저 instruction-tuned / chat-ready 계열인지 확인하고, 그다음에 내 장비에 맞는 크기와 양자화 방식을 고르면 됩니다.

## GGUF란?

GGUF 공식 문서 기준:

- GGUF는 GGML 및 GGML 기반 실행기에서 추론용 모델을 저장하는 파일 포맷입니다.
- 빠른 로딩, `mmap` 친화성, 단일 파일 배포, 메타데이터 포함을 목표로 설계되었습니다.
- 보통 모델은 처음에 PyTorch 같은 프레임워크에서 학습되고, 이후 로컬 추론용으로 GGUF로 변환되어 배포됩니다.

쉽게 말하면:

- Hugging Face 원본 체크포인트를
- 로컬 PC에서 다루기 쉬운 형태로 바꾼
- 로컬 실행용 파일 포맷

이라고 보면 됩니다.

실습에서 중요한 점:

- Windows/Linux에서 로컬 LLM을 돌릴 때 가장 흔히 보는 포맷 중 하나가 `GGUF`입니다.

- LM Studio, Ollama, llama.cpp 같은 도구와 함께 자주 등장합니다.
- Q4, Q5, Q8 같은 양자화 표기가 붙은 GGUF 모델을 많이 보게 됩니다.

## FP32, BF16, FP16, 4-bit는 무엇인가요?

LLM 모델은 아주 많은 숫자로 이루어져 있습니다. 이 숫자 하나하나를 어떤 형식으로 저장하고 계산할지에 따라 모델 크기와 실행 속도가 달라집니다.

자주 보는 표기는 아래처럼 이해하면 됩니다.

| 표기 | 의미 | 파라미터 1개당 대략 크기 | 느낌 |
|---|---|---|---|
| FP32 | 32-bit floating point | 4 bytes | 정확하지만 너무 큼 |
| BF16 | bfloat16 | 2 bytes | LLM 원본 가중치에서 자주 쓰임 |
| FP16 | 16-bit floating point | 2 bytes | GPU 추론에서 흔함 |
| INT8 / Q8 | 8-bit 양자화 | 1 byte 안팎 | 원본보다 작고 비교적 안정적 |
| INT4 / Q4 | 4-bit 양자화 | 0.5 byte 안팎 | 훨씬 작지만 품질/지원 차이 있음 |

Gemma 4를 FP32가 기본이라고 생각하면 메모리를 너무 크게 잡게 됩니다. 공식 Transformers 설정은 BF16 계열로 보는 것이 맞고, LM Studio/Ollama에서 받는 로컬 모델은 보통 여기서 한 번 더 줄인 양자화 모델입니다.

예를 들어 31B 모델을 단순 계산하면:

```text
FP32: 31B x 4 bytes ≈ 124GB
BF16: 31B x 2 bytes ≈ 62GB
4-bit: 31B x 0.5 bytes ≈ 15.5GB + 오버헤드
```

즉, 31B라는 이름만 보고 31GB라고 생각하면 안 됩니다. 모델 이름의 B는 파라미터 수이고, 실제 파일 크기는 숫자 형식과 양자화 방식에 따라 달라집니다.

## BF16은 왜 많이 쓰나요?

BF16은 `bfloat16`의 줄임말입니다. FP32보다 메모리는 절반만 쓰면서, 숫자를 표현할 수 있는 범위는 FP32와 비슷하게 유지하는 16-bit 형식입니다.

FP16과 BF16은 둘 다 16-bit지만 성격이 다릅니다.

- FP16: 소수점 정밀도는 조금 더 좋지만, 표현 가능한 숫자 범위가 좁습니다.
- BF16: 소수점 정밀도는 낮지만, FP32와 비슷한 넓은 숫자 범위를 가집니다.

LLM에서는 계산 중 숫자가 매우 커지거나 작아질 수 있습니다. 그래서 BF16은 FP32보다 훨씬 가볍고, FP16보다 안정적인 경우가 많습니다.

## 모델 추론에 필요한 메모리는 어떻게 계산하나요?

모델 추론에 필요한 메모리는 한 숫자로 딱 떨어지지 않습니다. 대략 아래 항목을 더해서 생각하면 됩니다.

```text
추론 메모리 ≈ 모델 가중치 + KV cache + 입력/출력 처리 메모리 + 실행 도구 오버헤드
```

이 중에서 가장 큰 부분은 보통 모델 가중치입니다. 가중치 메모리는 아래처럼 계산합니다.

```text
가중치 메모리 ≈ 파라미터 수 x 파라미터 1개당 저장 크기
```

예를 들어 26B 모델은 파라미터가 약 260억 개입니다.

```text
BF16: 26B x 2 bytes ≈ 52GB
4-bit: 26B x 0.5 bytes ≈ 13GB
```

BF16 기준 약 52GB였던 가중치가 4-bit에서는 약 13GB로 줄어듭니다. 이 차이는 파라미터 수가 줄어서가 아니라, 파라미터 하나를 저장하는 크기가 줄어서 생깁니다.

```text
2 bytes -> 0.5 byte
가중치 메모리 약 1/4
```

하지만 실제 추론 메모리는 가중치 계산보다 더 듭니다.

- 양자화 메타데이터
- 일부 비양자화 레이어
- KV cache
- 긴 컨텍스트
- 이미지 입력
- 실행 도구와 GPU/CPU 오버헤드

이 붙기 때문입니다. 그래서 26B 4-bit 모델이 이론상 13GB 근처로 보이더라도, 실제 LM Studio/Ollama 표시 크기는 17~18GB처럼 보일 수 있습니다.

## NVIDIA GPU마다 지원 타입이 다른 이유

NVIDIA GPU마다 FP16, BF16, INT8, FP8, FP4 같은 지원이 다른 이유는 GPU 세대마다 Tensor Core가 처리할 수 있는 숫자 형식이 다르기 때문입니다.

중요한 구분:

```text
모델을 4-bit로 저장할 수 있음
≠
GPU가 4-bit를 전용 회로로 바로 빠르게 계산할 수 있음
```

많은 경우 4-bit 가중치를 읽은 뒤 실제 행렬 곱은 FP16/BF16으로 풀어서 계산합니다. 그래서 같은 모델이라도 GPU 세대, CUDA, 드라이버, 실행 도구에 따라 속도와 지원 여부가 달라질 수 있습니다.

실습에서는 아래 정도만 기억하면 충분합니다.

- 최신 GPU일수록 더 다양한 저정밀 타입을 빠르게 처리할 가능성이 큽니다.
- 모델 파일의 Q4, Q5, Q8 표기는 저장 방식에 가깝습니다.

- 실제 계산이 어떤 타입으로 도는지는 실행 엔진과 하드웨어 지원에 따라 달라집니다.
- 타입 지원이 애매하면 작은 모델이나 더 보수적인 양자화 모델을 고르는 편이 안전합니다.

## NVFP4는 무엇인가요?

`NVFP4`는 NVIDIA가 Blackwell 세대 GPU를 중심으로 밀고 있는 4-bit floating point 데이터 타입입니다. NVIDIA 기술 블로그도 NVFP4를 low-precision inference를 위한 data type으로 설명하고, Hugging Face의 `Gemma-4-31B-IT-NVFP4` 모델 카드도 weights와 activations를 NVFP4 data type으로 양자화했다고 설명합니다.

다만 NVFP4를 "그냥 4-bit 숫자 하나"로만 이해하면 부족합니다. NVFP4는 4-bit 값 + scale 정보 + 하드웨어 가속 방식이 함께 묶인 micro-scaled FP4 데이터 타입에 가깝습니다.

즉, NVFP4는 아래 세 가지 성격을 같이 가집니다.

- 데이터 타입: 숫자를 4-bit floating point로 표현하는 방식
- 양자화 포맷: 원래 BF16/FP16 값을 NVFP4 값과 scale로 줄여 저장하는 방식
- 실행 경로: Blackwell Tensor Core가 이 형식을 빠르게 처리하도록 설계된 하드웨어 지원 방식

그래서 문서나 모델 카드에서 NVFP4를 볼 때는:

```text
NVFP4 = 4-bit floating point data type + block scaling
+ NVIDIA 하드웨어/런타임 지원
```

으로 이해하는 것이 가장 안전합니다.

일반적인 4-bit 양자화와 비슷하게 모델을 작게 만들지만, 단순히 숫자를 4-bit로 자르는 방식은 아닙니다.

NVFP4의 핵심은 작은 블록마다 scale을 따로 두는 것입니다.

- 4-bit 값 자체는 `1 sign bit + 2 exponent bits + 1 mantissa bit` 구조입니다.
- 값 16개마다 FP8(E4M3) scale을 공유합니다.
- tensor 전체에는 추가 FP32 scale을 한 번 더 적용합니다.

즉:

```text
원래 값 ≈ 4-bit 값 x 16개 단위 FP8 scale x tensor 단위 FP32
scale
```

이렇게 하면 4-bit처럼 작게 저장하면서도, 각 블록의 값 범위에 더 잘 맞출 수 있습니다. NVIDIA 기술 블로그 기준으로 NVFP4는 FP16 대비 약 3.5배, FP8 대비 약 1.8배 메모리 절감 효과를 목표로 합니다.

## Gemma 4 31B NVFP4는 왜 주목받나요?

NVIDIA는 `Gemma-4-31B-IT-NVFP4` 모델을 Hugging Face에 공개했습니다. 공식 모델 카드 기준으로 이 모델은 Google의 Gemma 4 31B instruction-tuned 모델을 NVIDIA Model Optimizer로 NVFP4 양자화한 버전입니다.

주요 포인트:

- 31B dense 모델을 NVFP4로 줄인 버전
- weights와 activations가 NVFP4 data type으로 양자화된 버전

- 256K context window 명시
- text, image, video 입력 지원
- vLLM 런타임 기준
- NVIDIA Blackwell 호환성을 명시
- calibration에는 `cnn_dailymail` 데이터셋 사용
- 평가표에서 BF16 원본 대비 여러 벤치마크 손실이 0.5%p 미만으로 표시됨

Hugging Face 파일 정보에 `BF16`, `F8_E4M3`, `U8` 같은 tensor type이 함께 보일 수 있습니다. 이것은 NVFP4가 4-bit 값만 단독으로 있는 구조가 아니라, FP8 scale이나 packing된 저장 표현을 함께 쓰는 구조이기 때문입니다. 모델 이름에 NVFP4가 붙어도 내부 파일에는 scale, metadata, 일부 보조 tensor가 같이 들어갈 수 있습니다.

다만 이 모델을 행사 기본 준비로 보기는 어렵습니다.

- 공식 모델 카드의 예시 serve 명령은 `--tensor-parallel-size 8`입니다.
- 모델 카드의 test hardware는 H100으로 표시되어 있습니다.
- Blackwell에서 가장 잘 맞도록 설계된 경로입니다.
- 24GB GPU에서 짧은 context로 가능하다는 이야기는 조건부로 봐야 합니다.
- 256K full context까지 한 장의 24GB GPU에서 안정적으로 된다는 뜻은 아닙니다.

따라서 실습 문서에서는 이렇게 이해하면 됩니다.

- 개념적으로: NVFP4는 4-bit 양자화의 최신 고성능 경로 중 하나입니다.
- 실습 준비로: LM Studio/Ollama의 E2B/E4B 또는 26B A4B 준비가 우선입니다.
- 고급 실험으로: Blackwell/Hopper급 NVIDIA GPU, vLLM, Linux 환경이 있으면 별도 실험 항목으로 볼 수 있습니다.

## llama.cpp란?

llama.cpp 공식 README 기준:

- llama.cpp는 C/C++ 기반 LLM inference 엔진입니다.
- 최소한의 설정으로 다양한 하드웨어에서 로컬/클라우드 추론을 할 수 있도록 만드는 것이 목표입니다.
- Apple Silicon, x86, NVIDIA GPU, AMD GPU 등 여러 백엔드를 지원합니다.
- OpenAI 호환 API 서버(`llama-server`)도 제공할 수 있습니다.

이름 때문에 오해하기 쉬운데:

- `llama.cpp`는 Llama 모델만 실행하는 도구가 아닙니다.
- 실제로 Gemma, Qwen, Mistral 등 `llama.cpp` 호환 모델도 많이 실행합니다.

쉽게 말하면:

- GGUF 파일을 실제로 돌려주는 대표적인 실행 엔진

입니다.

정리하면:

- LM Studio는 문서상 GGUF 모델을 `llama.cpp`로 실행

라고 이해하면 큰 흐름을 잡는 데 도움이 됩니다.

## MLX란?

Apple MLX 공식 README 기준:

- MLX는 Apple Silicon용 머신러닝 프레임워크입니다.
- CPU와 GPU를 아우르는 unified memory 구조를 활용합니다.
- Python, C++, C, Swift API를 제공합니다.

LLM 관점에서 더 직접적인 것은 `mlx-lm` 입니다.

Apple `mlx-lm` 공식 README 기준:

- `mlx-lm`은 Apple Silicon에서 LLM을 실행하고 fine-tuning 하는 패키지입니다.
- Hugging Face 모델을 불러오고
- 양자화하고
- MLX 형식으로 변환하고
- LoRA/QLoRA fine-tuning도 할 수 있습니다.

즉, 실습에서는:

- `MLX`는 Apple Silicon 전용 경로
- MLX 모델은 Apple Silicon에서 잘 맞는 실행 방식

으로 이해하면 됩니다.

## GGUF와 MLX는 어떻게 다른가요?

| 항목 | GGUF | MLX |
|---|---|---|
| 성격 | 로컬 추론용 모델 파일 포맷 | Apple Silicon용 프레임워크/실행 생태계 |
| 대표 실행 경로 | `llama.cpp`, LM Studio, Ollama | MLX, `mlx-lm`, LM Studio(Apple Silicon) |
| 운영체제 | macOS, Windows, Linux 전반 | 사실상 Apple Silicon 중심 |
| 실습 기준 | 가장 흔한 범용 로컬 포맷 | Mac Apple Silicon에서만 고려할 선택지 |

핵심:

- `GGUF`는 더 범용적입니다.
- `MLX`는 Apple Silicon에 특화된 선택지입니다.

## LM Studio에서는 왜 둘 다 보이나요?

LM Studio 공식 문서 기준:

- LM Studio는 macOS, Windows, Linux에서 `llama.cpp` 기반 GGUF 모델 실행을 지원합니다.
- Apple Silicon Mac에서는 Apple의 MLX 실행 경로도 지원합니다.

그래서 LM Studio에서:

- Windows/Linux에서는 주로 `GGUF`

- Apple Silicon Mac에서는 `GGUF`와 `MLX` 둘 다

보일 수 있습니다.

## 무엇을 선택하면 되나요?

이번 핸즈온에서는:

- Windows/Linux: `GGUF` 중심으로 생각하면 됩니다.
- Apple Silicon Mac: GGUF 기본, MLX는 옵션

즉:

- 개념이 헷갈리면 그냥 GGUF를 기본 선택지로 보면 됩니다.
- Apple Silicon 사용자만 필요할 때 `MLX`를 추가로 이해하면 충분합니다.

## 왜 GGUF가 행사 기본 설명에 더 자주 나오나요?

이유는 단순합니다.

- 실습 환경이 macOS만 있는 것이 아니고
- Windows/Linux/ChromeOS까지 같이 봐야 하며
- LM Studio, Ollama, llama.cpp 같은 로컬 도구에서 범용적으로 이해하기 쉬운 축이 `GGUF`이기 때문입니다.

그래서 이번 핸즈온에서는:

- `GGUF`를 기본 개념
- `MLX`를 Apple Silicon 추가 옵션

으로 이해하면 됩니다.

## llama.cpp는 행사에서 왜 알아두면 좋나요?

꼭 직접 설치할 필요는 없지만, 아래를 이해하는 데 도움이 됩니다.

- 왜 GGUF 모델이 여러 로컬 도구에서 공통으로 보이는지
- 왜 `Q4`, `Q5`, `Q8` 같은 양자화 모델이 자주 나오는지
- 왜 OpenAI 호환 로컬 서버를 만들 수 있는지

즉, llama.cpp는 로컬 LLM 생태계에서 기초 엔진 역할을 하는 경우가 많습니다.

## 한 줄 정리

- `GGUF`: 범용 로컬 모델 파일
- `llama.cpp`: 그 파일을 돌려주는 대표 엔진
- `MLX`: Apple Silicon에서 쓰는 별도 최적화 경로

## 이번 핸즈온 기준 추천 해석

- 개념이 어렵다면 GGUF 기본으로 이해해도 충분합니다.
- Apple Silicon Mac 사용자만 MLX는 선택 사항으로 추가 이해하면 됩니다.

- `llama.cpp`는 직접 만지는 도구가 아니더라도, LM Studio/GGUF를 이해하는 배경지식으로 알면 좋습니다.

## 같이 보면 좋은 문서

- LM Studio 설치 가이드
- Apple Silicon + MLX 안내
- 메모리 기준으로 Gemma 4 모델 고르는 방법

## 참고 링크

- GGUF spec (ggml)
- llama.cpp README
- Apple MLX README
- Apple MLX LM README
- LM Studio Docs
- Google AI for Developers: Run Gemma
- Google Gemma 4 Hugging Face collection
- google/gemma-4-26B-A4B-it
- NVIDIA Gemma-4-31B-IT-NVFP4
- NVIDIA Technical Blog: Introducing NVFP4

문서 05

docs/05-lm-studio-setup.md

# LM Studio 설치 가이드

## 먼저 알아둘 점

- 이번 세션은 개발자, 비개발자 모두 참여 가능한 세션이므로, 하나만 설치한다면 LM Studio를 기본 권장합니다.
- LM Studio는 macOS, Windows, Linux를 지원합니다.
- macOS는 Apple Silicon만 지원합니다.
- macOS 앱은 공식 문서 기준 13.4 이상이 필요하며, MLX 모델은 macOS 14.0 이상이 필요합니다.
- Windows는 x64와 ARM을 지원합니다.
- Linux는 Ubuntu 20.04+ AppImage가 공식 기준입니다.
- 모델 다운로드 후에는 완전 오프라인 사용이 가능합니다.
- Apple Silicon Mac에서는 GGUF를 기본으로 사용해도 되고, 필요하면 MLX 모델도 선택할 수 있습니다.
- 이번 핸즈온은 채팅/실습/에이전트 흐름이 중심이므로 instruction-tuned(chat-ready) 모델을 선택하세요.
- `base`나 `pretrained`로 표시된 모델은 기본 준비용으로 받지 않는 편이 안전합니다.
- LM Studio에서는 모델 이름이 `...-it` 로 그대로 보일 수도 있고, `google/gemma-4-e4b`처럼 더 짧게 보일 수도 있으므로 설명이나 모델 카드에서 chat-ready / instruct 계열인지 함께 확인해 주세요.

## Windows

이런 환경에 맞습니다:

- Windows x64 또는 ARM
- RAM 16GB 이상 권장

설치 순서:

1. 공식 다운로드 페이지로 이동합니다. LM Studio 다운로드
2. Windows용 설치 파일을 받아 실행합니다.
3. 앱을 한 번 실행합니다.
4. 필요 시 런타임을 설치하거나 업데이트합니다. 단축키: `Ctrl + Shift + R`
5. `Discover` 탭에서 `gemma 4`를 검색합니다.
6. 아래 중 하나를 다운로드합니다.

- `google/gemma-4-e2b` 계열의 chat-ready / instruction-tuned 항목
- `google/gemma-4-e4b` 계열의 chat-ready / instruction-tuned 항목

1. `Chat` 탭으로 이동해 모델을 메모리에 로드합니다.
2. 테스트 프롬프트를 입력합니다.

예시 프롬프트:

```text
안녕하세요. Build with AI Seoul 2026 핸즈온 테스트입니다. 한 줄로 자기소개해
주세요.
```

선택 사항:

- 터미널에서 `lms --help`를 실행해 CLI까지 확인할 수 있습니다.
- 단, `lms`는 LM Studio를 최소 1회 실행한 뒤 사용하는 것이 안전합니다.
- Windows에서는 PowerShell 기준으로 아래처럼 확인하면 됩니다.

```powershell
lms --help
```

## macOS

이런 환경에 맞습니다:

- Apple Silicon (M1/M2/M3/M4)
- macOS 13.4 이상
- MLX 모델까지 활용하려면 macOS 14.0 이상

설치 순서:

1. 공식 다운로드 페이지로 이동합니다. LM Studio 다운로드
2. macOS용 앱을 설치합니다.
3. 앱을 실행합니다.
4. 필요 시 런타임을 설치하거나 업데이트합니다. 단축키: `Cmd + Shift + R`
5. `Discover` 탭에서 `gemma 4`를 검색합니다.
6. 아래 중 하나를 다운로드합니다.

- `google/gemma-4-e2b` 계열의 chat-ready / instruction-tuned 항목
- `google/gemma-4-e4b` 계열의 chat-ready / instruction-tuned 항목

1. `Chat` 탭에서 모델을 로드하고 테스트합니다.

권장 메모:

- LM Studio 공식 문서는 macOS에서 16GB+ RAM 권장입니다.
- LM Studio 앱 자체는 macOS 13.4 이상을 지원하고, MLX 모델은 macOS 14.0 이상이 필요합니다.
- 행사 준비 기준으로는 최신 macOS를 권장합니다.
- 8GB Mac도 가능할 수 있지만, 작은 모델과 작은 컨텍스트만 권장합니다.
- Apple Silicon에서도 GGUF 기반 fine-tuned 모델을 사용하는 것은 대체로 가능하지만, 양자화 방식, 채팅 템플릿, 멀티모달 지원 여부에 따라 모델별 확인이 필요합니다.
- MLX는 선택 사항이며, 행사 준비 기준으로는 GGUF만 사용해도 충분합니다.

- `GGUF`, `MLX`, `llama.cpp` 개념이 헷갈리면 GGUF, MLX, llama.cpp 개념 설명 문서를 먼저 확인해 주세요.
- Apple Silicon 사용자는 필요하면 Apple Silicon + MLX 안내도 함께 확인해 주세요.
- LM Studio를 `opencode` 같은 코딩 에이전트 도구와 연결해 보고 싶다면 OpenCode 설치 및 LM Studio 연동 가이드도 함께 확인해 주세요.
- Windows 사용자는 LM Studio CLI 예시를 볼 때 `bash` 대신 PowerShell로 따라오시면 됩니다.

## Linux (Ubuntu)

이런 환경에 맞습니다:

- Ubuntu 20.04 이상
- x64 또는 ARM64

설치 순서:

1. 공식 다운로드 페이지로 이동합니다. LM Studio 다운로드
2. Linux용 AppImage를 다운로드합니다.
3. 터미널에서 실행 권한을 부여합니다.

```bash
chmod +x LM-Studio-*.AppImage
```

1. AppImage를 실행합니다.

```bash
./LM-Studio-*.AppImage
```

1. 앱이 실행되면 `Discover` 탭에서 `gemma 4`를 검색합니다.
2. `google/gemma-4-e2b` 또는 `google/gemma-4-e4b` 계열의 chat-ready / instruction-tuned 항목을 다운로드합니다.
3. `Chat` 탭에서 모델을 로드하고 테스트합니다.

참고:

- 공식 문서는 Linux 배포 형식을 AppImage로 안내합니다.
- 위 AppImage 실행 절차는 그 배포 형식에 맞춘 일반적인 설치 흐름입니다.
- Ubuntu 22.04보다 최신 버전은 공식 문서상 충분히 테스트되지 않았다고 안내되어 있습니다.

## ChromeOS

ChromeOS에서는 LM Studio를 권장하지 않습니다.

이유:

- LM Studio Linux 공식 지원 기준은 Ubuntu 20.04+ AppImage입니다.
- ChromeOS Linux 개발 환경은 Debian 기반입니다.

- ChromeOS Linux 환경에서는 로컬 LLM 실행에 필요한 전용 GPU/VRAM 기반 compute 가속을 기대하기 어렵습니다.

즉, ChromeOS에서 LM Studio는 공식 권장 경로 밖이며, 행사 당일 설치 이슈가 생길 가능성이 있습니다.

ChromeOS를 쓰는 경우:

1. Ollama로 준비
2. 가능하면 Windows 또는 macOS 노트북 추가 준비

문서 06

docs/06-ollama-setup.md

# Ollama 설치 가이드

## 이 문서는 언제 보면 되나요?

- LM Studio 외에 CLI/API 방식도 함께 준비하고 싶을 때
- ChromeOS를 쓸 때
- Intel Mac처럼 LM Studio 데스크톱 앱이 맞지 않는 환경일 때

## Windows

이런 환경에 맞습니다:

- Windows 10 22H2 이상
- NVIDIA 또는 AMD GPU가 있으면 더 좋음

설치 순서:

1. 공식 다운로드 페이지로 이동합니다. Ollama Windows 다운로드
2. 가장 쉬운 방법은 설치 프로그램(`OllamaSetup.exe`)을 사용하는 것입니다.
3. 설치 후 PowerShell 또는 명령 프롬프트를 열고 아래 명령으로 확인합니다.

```powershell
ollama --version
```

1. 메모리에 맞는 Gemma 4 모델을 미리 받습니다.

```powershell
ollama pull gemma4:e2b
```

또는

```powershell
ollama pull gemma4:e4b
```

1. 실행 테스트:

```powershell
ollama run gemma4:e2b
```

권장 메모:

- Ollama Windows 설치는 공식 문서 기준 관리자 권한이 필수는 아닙니다.
- 모델은 기본적으로 사용자 홈 디렉터리 아래에 저장됩니다.

- 설치 후 Ollama는 백그라운드에서 실행되고, 로컬 API는 기본적으로 `http://localhost:11434`에서 동작합니다.
- Windows에서는 명령 프롬프트(CMD) 도 가능하지만, 이 가이드는 PowerShell 기준입니다.
- Ollama는 `gemma4:e2b`, `gemma4:e4b` 같은 런타임 태그를 사용하므로, 따로 `-it` 접미사를 고르지 않아도 됩니다. 핸즈온에서는 이 태그들을 chat-ready 실행 경로로 이해하면 됩니다.
- 세부 태그를 직접 고를 때는 `it`, `instruct`, `chat-ready` 계열을 우선하고, `base` 또는 `pretrained`로 보이는 항목은 기본 준비용으로 피하세요.

## Gemma 4 태그 선택

2026-04-27 기준 Ollama Gemma 4 태그 페이지에는 29개 태그가 표시됩니다.

사전 준비에서는 목적이 명확한 로컬 기본 태그를 사용하세요.

- 8GB: `gemma4:e2b`
- 16GB: `gemma4:e4b`, 버거우면 `gemma4:e2b`
- 32GB 이상: `gemma4:26b`
- 36GB 이상: `gemma4:26b` 또는 `gemma4:31b`

주의:

- `gemma4:latest`는 현재 E4B 계열로 표시되지만, 나중에 바뀔 수 있습니다.
- `*-cloud` 태그는 로컬에 모델을 받아 두는 경로가 아니므로, 사전 준비 기본값으로 쓰지 마세요.
- `*-mlx-*`, `*-mxfp8`, `*-nvfp4`, `*-q8_0` 같은 변형 태그는 정밀도와 실행 목적을 알고 선택할 때만 사용하세요.

## macOS

이런 환경에 맞습니다:

- macOS 14 Sonoma 이상
- Apple Silicon 권장
- Intel Mac도 가능하지만 CPU 전용

설치 순서:

1. 공식 다운로드 페이지로 이동합니다. Ollama macOS 다운로드
2. `ollama.dmg`를 열고 Ollama 앱을 `Applications` 폴더로 옮깁니다.
3. 앱을 처음 실행합니다.
4. 터미널에서 설치 확인:

```bash
ollama --version
```

1. 메모리에 맞는 모델을 미리 받습니다.

```bash
ollama pull gemma4:e2b
```

또는

```bash
ollama pull gemma4:e4b
```

1. 실행 테스트:

```bash
ollama run gemma4:e2b
```

권장 메모:

- 첫 실행 시 CLI 경로 연결을 요청하면 허용하세요.
- Apple Silicon에서는 CPU/GPU를 함께 활용하기 쉽고, 행사 진행에도 유리합니다.

## Linux (Ubuntu 등)

이런 환경에 맞습니다:

- 일반 Linux 노트북 또는 데스크톱
- NVIDIA/AMD GPU가 있으면 더 유리
- ARM64 장치도 가능하지만 행사 안정성 기준으로는 x64가 더 무난

설치 순서:

1. 터미널을 엽니다.
2. Ollama를 설치합니다.

```bash
curl -fsSL https://ollama.com/install.sh | sh
```

1. 설치 확인:

```bash
ollama -v
```

1. 모델 다운로드:

```bash
ollama pull gemma4:e2b
```

또는

```bash
ollama pull gemma4:e4b
```

1 실행 테스트:

```bash
ollama run gemma4:e2b
```

추가 메모:

- AMD GPU가 있으면 ROCm 패키지를 별도로 설치할 수 있습니다.
- ARM64 장치에서는 ARM64용 패키지를 써야 합니다.
- 서버처럼 계속 띄워둘 경우 `ollama serve`와 systemd 서비스 설정도 가능합니다.

## ChromeOS

ChromeOS는 Ollama만 권장합니다. 다만 이 경로는 안정성이 낮을 수 있으며, Windows/macOS보다 느릴 수 있습니다.

중요 제한:

- Chromebook에서 Linux 개발 환경이 켜져 있어야 합니다.
- 회사/학교 지급 기기에서는 Linux 사용이 막혀 있을 수 있습니다.
- ChromeOS Linux 환경에서는 Ollama/Gemma 4 같은 로컬 LLM 실행에서 전용 GPU/VRAM 기반 compute 가속을 기대하기 어렵습니다.
- 따라서 ChromeOS에서는 E2B 위주로 준비하는 것을 권장합니다.

설치 순서:

1 ChromeOS에서 Linux 개발 환경을 켭니다. 경로: 설정 > About ChromeOS > Developers > Linux development environment > Set up

2 Linux 터미널이 열리면 패키지를 업데이트합니다.

```bash
sudo apt-get update && sudo apt-get dist-upgrade
```

1 Ollama를 설치합니다.

```bash
curl -fsSL https://ollama.com/install.sh | sh
```

1 설치 확인:

```bash
ollama -v
```

1 모델 다운로드:

```bash
ollama pull gemma4:e2b
```

1 실행 테스트:

```bash
ollama run gemma4:e2b
```

ChromeOS 권장 메모:

- 8GB Chromebook: 가능하면 다른 기기를 권장, 꼭 써야 한다면 E2B만 시도
- 16GB 이상 Chromebook: E2B 권장, E4B는 속도가 매우 느릴 수 있음

문서 07

docs/07-llamacpp-setup.md

# llama.cpp 설치 가이드

## 이 문서는 누구를 위한 문서인가요?

- `GGUF + llama.cpp`로 Gemma 4를 로컬에서 돌려 보고 싶은 분
- `llama-server`를 띄워 `Hermes`, `OpenCode`, `Pi` 같은 에이전트와 연결하고 싶은 분
- `LM Studio` 대신 CLI 기반 local server 방식을 선호하는 분

## 먼저 결론

- `llama.cpp`는 이번 세션의 고급 / 선택형 경로입니다.
- 가장 쉬운 설치 경로는 pre-built package 설치입니다.
- 공식 문서 기준으로:
- macOS / Linux: `brew install llama.cpp`
- Windows: `winget install llama.cpp`
- 이번 핸즈온에서는 소스 빌드보다 pre-built 설치를 우선 권장합니다.

## 왜 llama.cpp를 쓰나요?

- `GGUF` 모델을 직접 로컬에서 돌릴 수 있습니다.
- `llama-server`로 OpenAI 호환 API 서버를 띄울 수 있습니다.
- `Hermes` 같은 에이전트가 로컬 서버 주소에 붙기 쉽습니다.

## 권장 설치 방법

### macOS / Linux

```bash
brew install llama.cpp
```

설치 확인:

```bash
llama-cli --version
llama-server --version
```

### Windows

PowerShell:

```powershell
winget install llama.cpp
```

설치 확인:

```powershell
llama-cli --version
llama-server --version
```

모델 파일 경로에 공백이 있다면 큰따옴표로 감싸 주세요.

## 다른 설치 방법

공식 문서에는 아래도 있습니다.

- pre-built binaries 다운로드
- Docker
- 소스 빌드

이번 핸즈온에서는 일단 패키지 매니저 설치를 우선 사용하세요.

## 소스 빌드는 언제 필요한가요?

- GPU backend를 직접 선택해 빌드해야 할 때
- 최신 기능이 패키지보다 먼저 필요할 때
- 배포 패키지 대신 특정 빌드 옵션을 직접 제어하고 싶을 때

공식 CPU build 기본 예시는 아래입니다.

```bash
git clone https://github.com/ggml-org/llama.cpp
cd llama.cpp
cmake -B build
cmake --build build --config Release
```

이번 사전 준비 가이드에서는 소스 빌드를 기본 설치 방법으로 권장하지 않습니다.

## 가장 간단한 실행 예시

### 로컬 GGUF 파일 사용

```bash
llama-cli -m /path/to/model.gguf
```

Windows PowerShell:

```powershell
llama-cli -m "C:\path\to\model.gguf"
```

### Hugging Face에서 바로 다운로드해서 실행

```bash
llama-cli -hf ggml-org/gemma-4-26b-a4b-it-GGUF:Q4_K_M
```

Windows PowerShell:

```powershell
llama-cli -hf ggml-org/gemma-4-26b-a4b-it-GGUF:Q4_K_M
```

### OpenAI 호환 API 서버 실행

```bash
llama-server -hf ggml-org/gemma-4-26b-a4b-it-GGUF:Q4_K_M
```

Windows PowerShell:

```powershell
llama-server -hf ggml-org/gemma-4-26b-a4b-it-GGUF:Q4_K_M
```

또는 이미 내려받은 GGUF 파일이 있다면:

```bash
llama-server -m /path/to/model.gguf --host 127.0.0.1 --port 8080
```

Windows PowerShell:

```powershell
llama-server -m "C:\path\to\model.gguf" --host 127.0.0.1 --port 8080
```

기본 Web UI는 보통 아래에서 열 수 있습니다.

```text
http://localhost:8080
```

chat completions endpoint는 아래입니다.

```text
http://localhost:8080/v1/chat/completions
```

## Gemma 4와 함께 쓸 때

이번 핸즈온에서는 `Gemma 4 GGUF`를 `llama.cpp`로 띄우고, 필요하면 그 위에 `Hermes` 같은 에이전트를 연결하는 구성이 가능합니다.

예시:

```bash
llama-server -hf ggml-org/gemma-4-26b-a4b-it-GGUF:Q4_K_M
```

또는:

```bash
llama-server -m /absolute/path/to/gemma-4-E4B-it-Q4_K_M.gguf --host 127.0.0.1 --port 8080
```

## 행사 전 최종 확인

아래가 되면 준비 완료입니다.

### llama.cpp 준비 완료 기준

macOS / Linux:

```bash
llama-cli --version
llama-server --version
```

Windows PowerShell:

```powershell
llama-cli --version
llama-server --version
```

그리고 아래 중 하나 이상:

1. `llama-cli`로 한 번 답변 생성 성공
2. `llama-server` 실행 성공
3. `curl http://127.0.0.1:8080/v1/models` 확인 성공

예시:

```bash
curl http://127.0.0.1:8080/v1/models
```

Windows PowerShell:

```powershell
Invoke-RestMethod http://127.0.0.1:8080/v1/models
```

## 주의할 점

- `llama.cpp`만 설치해도 모델 파일은 자동으로 준비되지 않습니다.
- 대용량 모델은 다운로드 시간이 길 수 있으므로 행사 전에 미리 받아 두세요.

- GGUF 경로를 직접 관리해야 할 수 있습니다.
- Windows에서는 `llama.cpp` 자체는 설치 가능하지만, 그 위에 Hermes를 붙일 때는 Hermes가 WSL2 안에서 돌아가야 한다는 점을 별도로 이해해야 합니다.

## 이런 경우에 해 보세요

- CLI에 익숙한 경우
- LM Studio보다 더 직접적인 local server 구성을 원하는 경우
- Hermes + Gemma 4를 직접 묶어 보고 싶은 경우

## 공식 참고 링크

- llama.cpp README
- llama.cpp Install Guide
- llama.cpp Build Guide

문서 08

docs/08-apple-silicon-mlx.md

# Apple Silicon + MLX 안내

## 이 문서는 누구를 위한 문서인가요?

- Apple Silicon Mac(M1/M2/M3/M4)을 쓰는 분
- LM Studio에서 GGUF 말고 MLX 방식도 이해하고 싶은 분
- MLX 기반 파생 모델이나 fine-tuned 모델이 있는지 궁금한 분

## MLX란?

- MLX는 Apple Silicon용 머신러닝 프레임워크입니다.
- Apple의 `mlx-lm` 프로젝트는 Apple Silicon에서 LLM 실행뿐 아니라 fine-tuning(LoRA/QLoRA 포함)도 지원합니다.
- LM Studio 공식 문서도 Apple Silicon Mac에서 Apple의 MLX를 지원한다고 안내합니다.
- `GGUF`, `MLX`, `llama.cpp` 차이를 먼저 보고 싶다면 GGUF, MLX, llama.cpp 개념 설명 문서를 함께 확인해 주세요.

## 행사 준비에서 MLX는 어떻게 보면 되나요?

- Apple Silicon 사용자는 LM Studio에서 GGUF뿐 아니라 MLX 모델도 선택할 수 있습니다.
- Gemma 4도 LM Studio 모델 페이지 기준 GGUF와 MLX로 제공됩니다.
- 다만 이번 핸즈온 준비에서는 MLX 자체가 필수는 아닙니다.
- Apple Silicon에서도 GGUF 기반 fine-tuned 모델을 대체로 사용할 수 있지만, 양자화 방식, 채팅 템플릿, 멀티모달 지원 여부에 따라 모델별 호환성과 품질 확인이 필요합니다.
- 즉, macOS 사용자는 GGUF 기본 + 필요하면 MLX 추가 고려 정도로 이해하면 됩니다.
- 가장 중요한 것은 행사 전에 사용할 모델을 미리 다운로드하고 실행 확인을 끝내는 것입니다.

## MLX 기반 Gemma 4 모델 예시

Hugging Face에는 아래와 같은 MLX 형식 Gemma 4 모델이 올라와 있습니다.

- `mlx-community/gemma-4-e2b-it-8bit`
- `mlx-community/gemma-4-e4b-it-8bit`

이 모델들은 Hugging Face 카드 기준으로 각각 원본 Google Gemma 4 체크포인트를 MLX 형식으로 변환한 모델입니다.

## MLX 기반 fine-tuned 모델도 있나요?

있습니다.

- Apple의 `mlx-lm`은 Gemma 계열을 포함한 여러 모델 패밀리에 대해 LoRA/QLoRA fine-tuning을 지원합니다.
- Hugging Face에는 MLX 기반 파생 모델이나 fine-tuned 모델도 존재합니다.
- 예를 들어 `emanubiz/gemma4-E4B-opus-finetuned` 같은 모델은 `mlx-community/gemma-4-e4b-it-4bit`를 바탕으로 LoRA fine-tuning 한 사례입니다.

즉, Apple Silicon 환경에서는 MLX로 실행 가능한 Gemma 파생 모델이나 fine-tuned 모델을 활용할 수 있습니다.

## `mlx-community` 관련 주의

- `mlx-community`는 공식 Google 배포처가 아니라 Hugging Face 커뮤니티 업로드/변환 흐름을 사용하는 조직입니다.
- 따라서 `mlx-community` 계열 모델은 공식 Google 원본이나 Ollama, LM Studio의 기본 모델 목록과 비교해 업데이트나 패치 반영 시점이 늦을 수 있습니다.
- 커뮤니티 모델은 변환 방식, 채팅 템플릿, 툴 호출, 멀티모달 동작이 제각각일 수 있습니다.

권장:

- 행사 당일에는 공식 Gemma 4 기본 모델(E2B/E4B) 사용을 우선 권장합니다.
- MLX 파생 모델, 커뮤니티 변환 모델, 개인 fine-tuned 모델은 추가 실험용으로만 권장합니다.

## Apple Silicon을 쓰는 경우

- 하나만 설치한다면 LM Studio
- 메모리 8GB면 E2B만 시도, 속도 저하 감안
- 메모리 16GB면 E4B
- 행사 준비는 공식 기본 모델 우선
- GGUF로 준비해도 충분함
- GGUF fine-tuned 모델은 대체로 가능하지만, 모델별 호환성과 품질 확인 필요
- MLX는 선택 사항

8GB Apple Silicon Mac 추가 안내:

- 8GB 장비는 E2B에서도 속도가 많이 느리거나 시스템이 잠깐 멈춘 것처럼 보일 수 있습니다.
- E4B는 실행되더라도 행사 당일 안정성이 떨어질 수 있습니다.
- 가능하면 16GB 이상 메모리 장비를 권장합니다.
- 8GB 장비로 참석하는 경우에는 작은 모델, 짧은 컨텍스트, 최소한의 앱 실행을 전제로 준비해 주세요.

## 참고 문서

- LM Studio MLX Engine Blog
- Apple MLX GitHub
- Apple MLX LM GitHub
- MLX LoRA / QLoRA Guide
- Hugging Face MLX Community

문서 09

docs/09-gemma4-benchmarks-and-agent-expectations.md

# Gemma 4 벤치마크와 코딩 에이전트 기대치

## 왜 이 문서가 필요한가요?

Gemma 4는 로컬에서 실행 가능한 매우 강력한 오픈 모델이지만, 모델 크기에 따라 체감 성능 차이가 큽니다. 특히 E2B, E4B는 작은 노트북에서도 돌아가도록 설계된 모델이라, Codex, Claude Code 같은 상용 코딩 에이전트 수준의 성능을 그대로 기대하면 차이가 크게 느껴질 수 있습니다.

이 문서는 공식 Gemma 4 모델 카드 기준 벤치마크를 정리하고, 실습에서 어떤 기대치를 가져야 하는지 설명합니다.

## 공식 벤치마크 요약

아래 수치는 Google 공식 Gemma 4 모델 카드에 나온 수치입니다.

| 모델 | MMLU Pro | LiveCodeBench v6 | Codeforces ELO | Tau2 | MRCR v2 8 needle 128k |
|---|---|---|---|---|---|
| Gemma 4 E2B | 60.0 | 44.0 | 633 | 24.5 | 19.1 |
| Gemma 4 E4B | 69.4 | 52.0 | 940 | 42.2 | 25.4 |
| Gemma 4 26B A4B | 82.6 | 77.1 | 1718 | 68.2 | 44.1 |
| Gemma 4 31B | 85.2 | 80.0 | 2150 | 76.9 | 66.4 |

읽는 법:

- `LiveCodeBench v6`, `Codeforces ELO`는 코딩 성능을 볼 때 참고하기 좋습니다.
- `Tau2`는 에이전트형 작업을 볼 때 참고하기 좋은 지표입니다.
- `MRCR v2 8 needle 128k`는 긴 문맥 처리 능력을 볼 때 참고할 수 있습니다.

## 모델별 해석

### E2B

- 가장 작은 경량 모델입니다.
- 로컬 데모, 간단한 질의응답, 가벼운 보조 작업에는 적합합니다.
- 하지만 코딩과 에이전트 성능은 상위 모델과 차이가 큽니다.
- `LiveCodeBench 44.0`, `Tau2 24.5` 수준이라, 복잡한 코드 수정, 다단계 계획, 도구 사용이 많은 작업에서는 한계가 빨리 드러날 수 있습니다.

권장 사용:

- 8GB 환경
- 최소 사양 테스트
- "로컬 LLM이 돌아간다"는 확인용

### E4B

- 실습 기본 모델로 가장 현실적인 선택지입니다.

- E2B보다 확실히 낫지만, 여전히 소형 모델입니다.
- `LiveCodeBench 52.0`, `Tau2 42.2`로 E2B보다 개선되지만, 26B/31B와는 차이가 큽니다.

권장 사용:

- 16GB 전후 노트북
- 일반적인 질의응답, 문서 정리, 간단한 코드 보조
- 행사 표준 기본 모델

주의:

- Codex, Claude Code 같은 강한 코딩 에이전트를 써봤다면 생각보다 답이 약하거나, 툴 사용이 덜 안정적으로 느껴질 수 있습니다.

### 26B A4B

- 성능과 속도의 균형이 좋은 상위 옵션입니다.
- A4B는 답을 만들 때 실제로 계산에 참여하는 부분이 약 4B 규모라는 뜻입니다. 공식 모델 카드 기준으로는 총 파라미터가 `25.2B`, 실제 계산에 참여하는 파라미터가 `3.8B`입니다.
- 이 모델은 여러 "전문가" 부분 중 필요한 일부만 골라 쓰는 `MoE` 방식이라서 계산량은 줄어듭니다. 다만 모델 파일은 전체 26B 규모를 메모리에 올려야 합니다.
- 공식 모델 카드도 이 모델을 31B 대비 빠른 로컬 추론에 유리한 선택지로 설명합니다.
- `LiveCodeBench 77.1`, `Tau2 68.2`로 E4B보다 큰 폭으로 올라갑니다.

권장 사용:

- 32GB 이상
- 코딩, 계획, 에이전트형 흐름을 좀 더 안정적으로 쓰고 싶은 경우
- 31B보다 속도를 더 중시할 때

### 31B

- Gemma 4 계열 중 가장 높은 일반 성능을 보여주는 상위 모델입니다.
- `LiveCodeBench 80.0`, `Codeforces 2150`, `Tau2 76.9`로 공식 벤치마크에서 가장 강합니다.
- 하드웨어가 허용하면 가장 좋은 선택입니다.

권장 사용:

- 36GB 이상
- 긴 문맥, 코딩, 복잡한 작업, 더 높은 응답 품질이 중요할 때

## 코딩 에이전트 사용 시 기대치

공식 Gemma 4 모델 카드는 Gemma 4가 함수/도구 호출(function calling) 을 지원하고, reasoning, coding, agentic workflow에 적합하다고 설명합니다. 하지만 모델 크기별 벤치마크 차이를 보면, 특히 E2B, E4B는 상위 모델과 성능 차이가 큽니다.

- E2B, E4B는 `LiveCodeBench`, `Codeforces`, `Tau2`, `MRCR` 같은 코딩·에이전트 프록시 지표에서 26B/31B보다 낮습니다.

- 따라서 작은 모델일수록 복잡한 코드 수정, 긴 지시 처리, 다단계 계획에서 한계가 더 빨리 드러날 가능성이 높습니다.
- Gemma 4는 로컬 실습과 개인 환경에서 매우 유용합니다.
- 하지만 E2B, E4B에 대해 Codex, Claude Code 수준의 코딩 에이전트 경험을 그대로 기대하면 안 됩니다.
- 툴 사용 흐름의 안정성은 공식 벤치마크가 직접 측정한 항목이 아니라, 실행 도구, 프롬프트 구성, 로컬 OpenAI 호환 서버에 따라 달라질 수 있습니다.
- 작은 모델에서는 코드 수정 정확도, 긴 작업 지시 처리, 다단계 계획, 툴 사용 일관성이 약하게 느껴질 수 있습니다.

이번 핸즈온에서는 "상용 코딩 에이전트 대체"보다 로컬에서 실행 가능한 Gemma 4 체험과 기본 워크플로우 이해에 초점을 맞추면 됩니다.

## 모델별 권장 기대치

- 8GB: E2B는 "최소 실행 보장"에 가깝습니다.
- 16GB: E4B는 가장 현실적인 기본 모델입니다.
- 32GB 이상: 코딩/에이전트 체감을 더 원하면 26B A4B를 권장합니다.
- 36GB 이상: 가능하면 31B가 가장 좋습니다.
- 모델별 구조 차이를 더 자세히 보려면 Gemma 4 아키텍처 상세 정리 문서를 확인해 주세요.

## OpenCode 같은 코딩 에이전트에 연결할 때

- opencode 자체는 로컬 모델과 LM Studio를 공식적으로 지원하므로, 개발자 에이전트 시연 용도로는 조건부로 적절합니다.
- 다만 OpenCode 공식 문서도 코드 생성과 도구 호출(tool calling)을 모두 잘하는 모델은 많지 않다고 설명합니다.
- 따라서 Gemma 4를 opencode에 붙일 때는:
- E2B: 비추천
- E4B: 가벼운 보조 수준
- 26B A4B, 31B: 더 현실적인 선택
- 자세한 설치와 연동 방법은 OpenCode 설치 및 LM Studio 연동 가이드 문서를 확인해 주세요.

## 공식 참고 링크

- Google Gemma 4 official model card (31B)
- Google Gemma 4 official model card (26B A4B)
- Google Gemma 4 official model card (E2B)
- Ollama Gemma 4 library page

문서 10

docs/10-opencode-lmstudio-developer-agent.md

# OpenCode 설치 및 LM Studio 연동 가이드

## 이 문서는 누구를 위한 문서인가요?

- Gemma 4를 개발자 에이전트로 활용해 보고 싶은 분
- PM이 만든 PRD를 바탕으로 코드 구현 흐름을 실습해 보고 싶은 분
- LM Studio에 내려받은 로컬 Gemma 4 모델을 opencode와 연결하고 싶은 분

## 먼저 결론

- `opencode + LM Studio + Gemma 4` 조합은 조건부로 적절합니다.
- 이유는 opencode가 공식적으로 로컬 모델 + LM Studio의 OpenAI 호환 로컬 API 주소 구성을 지원하고, 에이전트별 프롬프트 / 모델 / 권한 / 단계 수를 설정할 수 있기 때문입니다.
- 다만 이 조합은 모두가 따라야 하는 기본 준비가 아니라, 개발자 에이전트를 시연하거나 직접 실습해 보고 싶을 때의 추가 준비입니다.

왜 이렇게 보나요?

- OpenCode 공식 문서는 로컬 모델과 LM Studio 구성을 지원합니다.
- 하지만 OpenCode 공식 문서도 코드 생성 + 툴 콜링을 모두 잘하는 모델은 많지 않다고 설명합니다.
- 실제로는 `Gemma 4 E2B/E4B` 같은 소형 모델에서 코딩 에이전트 체감이 약하거나, 툴 사용 흐름이 흔들릴 수 있습니다.
- 커뮤니티 이슈에서도 Gemma 4와 로컬 OpenAI 호환 서버 조합에서 도구 호출(tool calling) 실패나 루프 현상이 보고되어 있어, 100% 안정성을 전제로 한 데모로 잡는 것은 권장하지 않습니다.

권장:

- 발표자 데모: 26B A4B 또는 31B 권장
- 32GB 이상: 추가 실습으로 고려 가능
- 16GB: E4B로 가벼운 코딩 보조 정도만 권장
- 8GB: opencode 기반 개발자 에이전트 실습은 비권장

## 어떤 방식이 가장 안정적인가요?

PRD를 한 번에 길게 넣고 곧바로 전체 구현을 시키기보다, 아래처럼 2단계로 나누면 더 안정적입니다.

1 PM/Plan 단계

- PRD를 읽고 요구사항, 범위, 제약, 구현 순서를 짧은 체크리스트로 정리

1 Developer 단계

- 정리된 체크리스트를 바탕으로 코드 탐색, 수정, 테스트 수행

이 방식이 좋은 이유:

- 작은 모델일수록 긴 문서와 다단계 작업을 한 번에 처리할 때 흔들리기 쉽습니다.

- 먼저 범위를 좁히면 도구 호출(tool calling)과 코드 수정 정확도가 상대적으로 나아질 수 있습니다.
- 발표 흐름도 더 명확합니다.

즉, 이번 핸즈온에서는

- PM 역할: PRD 정리와 구현 체크리스트 작성
- Developer 역할: 체크리스트 기반 구현

처럼 나누는 구성이 더 적절합니다.

## 모델 선택 권장

opencode 같은 코딩 에이전트 흐름에서는 일반 채팅보다 모델 요구치가 더 높습니다.

| 환경 | 권장 모델 | 안내 |
|---|---|---|
| 8GB | 비권장 | opencode 기반 개발자 에이전트 실습은 권장하지 않습니다. |
| 16GB | E4B | 코드 읽기, 간단한 수정, 초안 생성 정도만 권장합니다. |
| 32GB | 26B A4B 우선 | 가장 현실적인 로컬 개발자 에이전트 실습 구간입니다. |
| 36GB+ | 31B 또는 26B A4B | 품질 우선이면 31B, 반응성 우선이면 26B A4B를 고려하세요. |

짧게 정리하면:

- E2B: 개발자 에이전트용으로는 비추천
- E4B: 가벼운 보조 수준
- 26B A4B: 가장 현실적인 선택
- 31B: 가능하면 가장 좋은 선택

## OpenCode 설치

이 가이드는 CLI/TUI 기준으로 설명합니다. OpenCode Desktop 앱도 있지만, 개발자 에이전트 실습은 보통 터미널 기반 흐름이 더 단순합니다.

### macOS / Linux

가장 간단한 방법:

```bash
brew install anomalyco/tap/opencode
```

또는 설치 스크립트:

```bash
curl -fsSL https://opencode.ai/install | bash
```

설치 확인:

```bash
opencode --version
```

### Windows

OpenCode 공식 문서는 Windows에서는 WSL 사용을 권장합니다.

즉, 이 가이드의 OpenCode 설치와 실행 예시는 PowerShell/CMD가 아니라 WSL 터미널 기준입니다.

권장 순서:

1. PowerShell 관리자 권한으로 WSL 설치

```powershell
wsl --install -d Ubuntu
```

설치가 멈추면:

```powershell
wsl --install --web-download -d Ubuntu
```

1. PowerShell에서 Ubuntu 실행

```powershell
wsl -d Ubuntu
```

1. WSL 터미널에서 OpenCode 설치

```bash
curl -fsSL https://opencode.ai/install | bash
```

설치 확인:

```bash
opencode --version
```

WSL 사용 시 프로젝트가 Windows 파일에 있다면 예시는 아래와 같습니다.

```bash
cd /mnt/c/Users/YourName/project
opencode
```

## LM Studio와 연결하기

### 1. LM Studio에서 Gemma 4 모델 준비

먼저 LM Studio에서 아래 중 하나를 다운로드하고 테스트합니다.

- `google/gemma-4-e4b` 계열의 instruction-tuned / chat-ready 모델
- `google/gemma-4-26b-a4b` 계열의 instruction-tuned / chat-ready 모델
- `google/gemma-4-31b` 계열의 instruction-tuned / chat-ready 모델

가능하면 행사 전 아래까지 확인해 두세요.

- 모델 다운로드 완료
- Chat 탭에서 1회 답변 생성 성공
- Hugging Face나 `llama.cpp` 예시에서는 보통 `...-it` 접미사가 보이지만, LM Studio에서는 모델 ID가 더 짧게 정리되어 보일 수 있습니다.

## 2. LM Studio 로컬 서버 시작

방법 1. GUI

- LM Studio `Developer` 탭으로 이동
- `Start server`를 켭니다

방법 2. CLI

macOS / Linux / WSL:

```bash
lms server start
```

Windows에서 WSL을 쓰지 않고 LM Studio CLI만 확인할 때는 PowerShell에서 아래처럼 실행할 수 있습니다.

```powershell
lms server start
```

기본 엔드포인트는 아래입니다.

```text
http://127.0.0.1:1234/v1
```

## 3. 모델 목록 확인

아래 명령으로 LM Studio 서버가 보는 모델 목록을 확인합니다.

macOS / Linux / WSL:

```bash
curl http://127.0.0.1:1234/v1/models
```

Windows PowerShell:

```powershell
Invoke-RestMethod http://127.0.0.1:1234/v1/models
```

여기서 보이는 실제 모델 identifier 를 확인해 두는 것이 안전합니다. 문서 예시에서는 `google/gemma-4-e4b` 같은 ID를 사용하지만, 실제 환경에서는 내려받은 모델 항목에 따라 표시 문자열이 다를 수 있습니다. 중요한 것은 이름 자체보다 instruction-tuned / chat-ready 모델을 선택했는지 입니다.

## `opencode.json` 예시

프로젝트 루트에 `opencode.json` 파일을 두는 방식을 권장합니다.

```json
{
  "$schema": "https://opencode.ai/config.json",
  "model": "lmstudio/google/gemma-4-e4b",
  "provider": {
    "lmstudio": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "LM Studio (local)",
      "options": {
        "baseURL": "http://127.0.0.1:1234/v1"
      },
      "models": {
        "google/gemma-4-e4b": {
          "name": "Gemma 4 E4B (local)"
        },
        "google/gemma-4-26b-a4b": {
          "name": "Gemma 4 26B A4B (local)"
        },
        "google/gemma-4-31b": {
          "name": "Gemma 4 31B (local)"
        }
      }
    }
  },
  "permission": {
    "bash": {
      "*": "ask",
      "git *": "allow",
      "npm *": "allow",
      "pnpm *": "allow",
      "bun *": "allow"
    },
    "edit": "ask"
  }
}
```

중요:

- `model` 값 형식은 `provider/model-id` 입니다.
- 위 예시에서 provider는 `lmstudio`, model-id는 `google/gemma-4-e4b` 입니다.
- 실제 모델 ID가 다르면 `curl http://127.0.0.1:1234/v1/models` 또는 PowerShell의 `Invoke-RestMethod http://127.0.0.1:1234/v1/models` 결과에 맞춰 수정하세요.
- Hugging Face나 `llama.cpp` 예시에서는 `gemma-4-26b-a4b-it` 같은 ID가 더 직접적으로 보일 수 있습니다.

## 개발자 에이전트 페르소나 예시

OpenCode는 에이전트별로 프롬프트와 권한을 설정할 수 있습니다. 즉, `PM`, `Developer`, `Reviewer` 같은 역할을 분리하는 구성이 가능합니다.

행사에서는 다음처럼 단순하게 구성합니다.

- `plan` 또는 PM 역할: PRD 해석, 범위 정리

- `developer` 역할: 코드 구현

예시 `opencode.json` 확장:

```json
{
  "$schema": "https://opencode.ai/config.json",
  "model": "lmstudio/google/gemma-4-e4b",
  "provider": {
    "lmstudio": {
      "npm": "@ai-sdk/openai-compatible",
      "name": "LM Studio (local)",
      "options": {
        "baseURL": "http://127.0.0.1:1234/v1"
      },
      "models": {
        "google/gemma-4-e4b": {
          "name": "Gemma 4 E4B (local)"
        },
        "google/gemma-4-26b-a4b": {
          "name": "Gemma 4 26B A4B (local)"
        }
      }
    }
  },
  "agent": {
    "developer": {
      "mode": "primary",
      "model": "lmstudio/google/gemma-4-26b-a4b",
      "prompt": "{file:./prompts/developer-agent.txt}",
      "steps": 6,
      "permission": {
        "bash": {
          "*": "ask",
          "git *": "allow",
          "npm *": "allow",
          "pnpm *": "allow",
          "bun *": "allow"
        },
        "edit": "ask"
      }
    }
  }
}
```

권장 모델 치환:

- 16GB 전후: `developer.model`도 `lmstudio/google/gemma-4-e4b`로 두세요.
- 32GB 이상: `lmstudio/google/gemma-4-26b-a4b` 권장
- 36GB 이상: 필요하면 `lmstudio/google/gemma-4-31b` 고려

예시 `prompts/developer-agent.txt`:

```text
You are the Developer employee in this AI Office.

Your input is a PRD or a short implementation checklist created by the PM.

Your job:
1. Summarize the task briefly.
2. Inspect the codebase before changing anything.
3. Propose a small implementation plan.
4. Make the smallest reasonable code change.
5. Report assumptions, risks, and test results clearly.

Rules:
- Do not make unrelated changes.
- Prefer small diffs.
- If the PRD is too broad, ask to narrow the scope.
- If tool use fails or becomes unstable, fall back to a text plan instead of hallucinating
success.
```

## 실제 사용 순서 예시

1. 프로젝트 폴더에서 `opencode` 실행
2. 필요하면 `/models` 로 `lmstudio/google/gemma-4-e4b` 또는 `lmstudio/google/gemma-4-26b-a4b` 선택
3. `plan` 또는 PM 역할로 PRD를 짧은 구현 체크리스트로 정리
4. `developer` 에이전트로 전환
5. 체크리스트를 넣고 구현 요청

예시 흐름:

```text
이 PRD를 읽고 이번 세션에서 구현할 최소 범위를 5개 이하 체크리스트로 정리해 줘.
```

그 다음:

```text
이 체크리스트를 기준으로 현재 코드베이스를 확인하고, 가장 작은 단위부터 구현해 줘.
```

## OpenCode 실습 주의사항

- `opencode`는 이번 행사에서 필수 준비 항목이 아닙니다.
- E2B, E4B는 일반 채팅보다 코딩 에이전트 체감이 약할 수 있습니다.
- 특히 작은 로컬 모델은 도구 호출(tool calling), 긴 PRD 처리, 다단계 수정 흐름에서 흔들릴 수 있습니다.
- 짧은 PRD, 좁은 구현 범위, 작은 단계, `ask` 기반 권한 설정을 권장합니다.
- 가능하면 26B A4B 또는 31B 모델에서 실습하는 것이 좋습니다.

## 참고 링크

- OpenCode Intro

- OpenCode Providers
- OpenCode Models
- OpenCode Agents
- OpenCode Permissions
- OpenCode Windows (WSL)
- OpenCode GitHub
- LM Studio local server
- LM Studio OpenAI 호환 endpoint 문서
- LM Studio List Models
- OpenCode issue: Gemma 4 e4b tool calling fails via Ollama
- OpenCode issue: Gemma 4 26B / 31B interaction issues

더 가벼운 대안은 Pi 경량 코딩 에이전트 대안과 이번 핸즈온 선택 이유 문서를 확인해 주세요.

문서 11

docs/11-pi-and-tool-selection-notes.md

# Pi 경량 코딩 에이전트 대안과 이번 핸즈온 선택 이유

## 이 문서는 누구를 위한 문서인가요?

- `opencode`보다 더 가벼운 터미널 코딩 에이전트를 찾는 분
- Gemma 4 local LLM을 붙일 수 있는 다른 코딩 에이전트 방식을 알고 싶은 분
- 왜 이번 핸즈온에서 `gemini-cli`, `Antigravity`를 code agent로 사용하지 않았는지 궁금한 분

## 먼저 결론

- `pi`는 좋은 경량 대안입니다.
- 이유는 `pi`가 스스로를 minimal terminal coding harness 로 설명하고, `~/.pi/agent/models.json` 을 통해 LM Studio / Ollama / vLLM 같은 로컬 OpenAI 호환 서버를 붙일 수 있다고 공식 문서에 적고 있기 때문입니다.
- 이번 핸즈온은 Gemma 4를 각자 노트북에 미리 내려받아 두고, 네트워크가 없거나 매우 불안정해도 로컬에서 실행 가능한 흐름을 중요하게 봅니다.
- 반대로 `gemini-cli`와 `Antigravity`는 이번 핸즈온 기준 Gemma 4 local LLM + LM Studio 기반의 오프라인 code agent 경로가 명확하지 않습니다.

짧게 정리하면:

- `opencode`: 기능이 더 많고, 역할 분리와 에이전트 구성이 쉬움
- `pi`: 더 가볍고 단순한 경량 대안
- `gemini-cli`: 공식 문서 기준 Gemini 중심 도구
- `Antigravity`: 공식 공개 자료 기준 hosted model 중심 도구에 가까우며, 로컬 Gemma 4 + LM Studio 기반 오프라인 경로가 명확하지 않음

## `pi`는 왜 대안으로 적절한가요?

Pi 공식 README 기준:

- `pi`는 minimal terminal coding harness 입니다.
- 기본 도구는 `read`, `write`, `edit`, `bash` 로 비교적 단순합니다.
- `sub agents`와 `plan mode` 같은 기능은 기본 내장보다 확장/패키지 방식에 가깝습니다.
- `~/.pi/agent/models.json` 으로 custom providers & models 를 추가할 수 있고, 문서에서 Ollama, LM Studio, vLLM 같은 로컬 모델 서버를 명시적으로 언급합니다.

즉, 이번 핸즈온에서는:

- `opencode`보다 설정 면적이 작고
- 로컬 OpenAI 호환 서버 연결이 가능하며
- "개발자 에이전트 최소 버전"을 보여주기 좋습니다.

## `opencode`와 `pi`는 어떻게 나눠 쓰나요?

| 도구 | 추천 상황 | 특징 |
|---|---|---|
| `opencode` | 역할 분리, 에이전트 구성, 데모 확장성이 중요할 때 | 기능이 더 많고, 에이전트/권한/프롬프트 구성이 풍부함 |
| `pi` | 더 가볍고 단순한 CLI 코딩 에이전트를 원할 때 | 최소 도구 구성, 빠른 시작, 커스텀 모델 연결 가능 |

권장 사용:

- 발표자 데모 메인 경로: `opencode`
- 가벼운 대안 소개: `pi`

## `pi` 설치 가이드

공식 Quick Start 기준:

macOS / Linux:

```bash
npm install -g @mariozechner/pi-coding-agent
```

Windows PowerShell:

```powershell
npm install -g @mariozechner/pi-coding-agent
```

설치 확인:

macOS / Linux:

```bash
pi --version
```

Windows PowerShell:

```powershell
pi --version
```

Windows 메모:

- `pi` 자체는 PowerShell에서도 설치와 실행이 가능합니다.
- 다만 이번 핸즈온의 code agent 메인 경로는 `OpenCode`이고, OpenCode는 Windows에서 WSL 권장입니다.
- Windows에서는 pi = PowerShell 가능, OpenCode = WSL 권장으로 구분하면 됩니다.

Ollama를 함께 쓰는 경우에는 `ollama launch pi --config`로 Pi의 Ollama provider 설정을 자동 생성할 수도 있습니다. 이 문서에서는 LM Studio 연결을 직접 확인하기 위해 `models.json` 수동 설정을 기준으로 설명합니다.

## LM Studio와 `pi` 연결하기

## 1. LM Studio 서버 준비

먼저 LM Studio에서 Gemma 4 모델을 다운로드하고, `Developer` 탭에서 서버를 시작합니다.

핸즈온 기준으로는 instruction-tuned / chat-ready 모델을 선택합니다. Hugging Face나 `llama.cpp` 예시에서는 보통 `...-it` 접미사가 보이지만, LM Studio에서는 이름이 더 짧게 보일 수 있습니다.

기본 엔드포인트:

```text
http://127.0.0.1:1234/v1
```

모델 확인:

macOS / Linux:

```bash
curl http://127.0.0.1:1234/v1/models
```

Windows PowerShell:

```powershell
Invoke-RestMethod http://127.0.0.1:1234/v1/models
```

## 2. `~/.pi/agent/models.json` 설정

Pi 공식 문서 기준으로, local model provider는 `models.json` 으로 추가합니다.

Pi 모델 문서 기준으로 `openai-completions` 가 most compatible 한 기본 선택입니다.

예시:

```json
{
  "providers": {
    "lmstudio": {
      "baseUrl": "http://127.0.0.1:1234/v1",
      "api": "openai-completions",
      "authHeader": false,
      "apiKey": "lm-studio",
      "models": [
        { "id": "google/gemma-4-e4b", "name": "Gemma 4 E4B (LM Studio)" },
        { "id": "google/gemma-4-26b-a4b", "name": "Gemma 4 26B A4B (LM Studio)" },
        { "id": "google/gemma-4-31b", "name": "Gemma 4 31B (LM Studio)" }
      ]
    }
  }
}
```

중요:

- 여기서 `id` 는 LM Studio 서버가 실제로 노출하는 모델 ID와 맞아야 합니다.
- 먼저 `curl http://127.0.0.1:1234/v1/models` 또는 PowerShell의 `Invoke-RestMethod http://127.0.0.1:1234/v1/models` 로 확인한 뒤 맞추세요.
- 예시처럼 `authHeader: false` 로 두면 LM Studio에 불필요한 Bearer 인증 헤더를 보내지 않게 됩니다.

- 만약 로컬 서버가 OpenAI 호환 필드를 일부만 지원한다면, Pi 모델 문서의 `compat` 설정으로 추가 조정할 수 있습니다.

### 3. 실행

실행 후 `/model` 로 모델을 고르거나, provider/model 형식으로 선택합니다.

예시:

macOS / Linux:

```bash
pi --model lmstudio/google/gemma-4-e4b
```

Windows PowerShell:

```powershell
pi --model lmstudio/google/gemma-4-e4b
```

또는 interactive 모드에서:

```text
/model
```

## `pi`에서 Gemma 4를 쓸 때 추천

- 8GB: 비권장
- 16GB: E4B로 가벼운 코드 보조만 권장
- 32GB: 26B A4B 권장
- 36GB+: 31B 또는 26B A4B

`pi`도 결국 코딩 에이전트이므로, 작은 모델의 한계는 `opencode`와 비슷하게 존재합니다.

## 왜 `gemini-cli`를 기본 code agent로 쓰지 않나요?

`gemini-cli`는 좋은 터미널 AI 에이전트이지만, 이번 핸즈온의 기본 code agent로는 쓰지 않습니다.

이번 핸즈온의 기준은 아래입니다.

- 각자 노트북에서 Gemma 4를 미리 다운로드
- LM Studio, Ollama, llama.cpp 같은 로컬 실행 경로 사용
- 현장 인터넷이 불안정해도 기본 실습 진행 가능

`gemini-cli`는 공식 문서상 Gemini, Google 계정, Gemini API key, Vertex AI 중심의 사용 흐름이 먼저 나옵니다. 로컬 Gemma 4를 LM Studio 또는 Ollama의 OpenAI 호환 로컬 API 주소로 붙이는 방식은 이번 사전 준비의 기본 방식으로 잡기 어렵습니다.

따라서 이번 핸즈온에서는:

- 기본 고급 code agent: `OpenCode`
- 더 가벼운 대안: `pi`

- Gemini 중심 워크플로: 별도 실험 항목

로 구분합니다.

## 왜 `Antigravity`를 기본 code agent로 쓰지 않나요?

`Antigravity`도 이번 핸즈온의 기본 code agent로는 쓰지 않습니다.

이유는 단순합니다.

- 공개 공식 자료는 personal Gmail account와 preview 사용 흐름을 전제로 설명합니다.
- 설치, 에이전트 사용, rules/workflows, 보안 설정은 확인할 수 있지만, LM Studio나 localhost의 OpenAI 호환 로컬 API 주소에 로컬 Gemma 4를 연결하는 안정적인 기본 방식은 확인하기 어렵습니다.
- 따라서 이번 핸즈온의 핵심 조건인 로컬 Gemma 4 + 사전 다운로드 + 오프라인 가능 실습과는 맞지 않습니다.

Antigravity는 hosted model 중심 agent IDE를 써 보고 싶은 별도 실험 주제로 보면 됩니다. 이번 사전 준비에서는 로컬 실행이 쉬운 `OpenCode`, 그리고 더 가벼운 `pi`를 우선합니다.

## 참고 링크

- Pi coding agent README
- Pi custom models docs
- Ollama Pi integration
- OpenCode 설치 및 LM Studio 연동 가이드
- Gemini CLI README
- Gemini CLI `/model` docs
- Gemini CLI issue: local/offline models
- Gemini CLI discussion: OpenAI-compatible local model support
- Google Antigravity Codelab

문서 12

docs/12-hermes-agent-overview.md

# Hermes Agent란 무엇인가

## 이 문서는 누구를 위한 문서인가요?

- `Hermes Agent`가 정확히 어떤 도구인지 먼저 이해하고 싶은 분
- `Codex`, `Claude Code`, `OpenCode` 같은 코딩 에이전트와 무엇이 다른지 감을 잡고 싶은 분
- 왜 이번 핸즈온에서 `Hermes`를 필수 준비 항목으로 두지 않는지 알고 싶은 분

## 먼저 한 줄 정의

`Hermes Agent`는 Nous Research가 만든 자율형 AI agent runtime 입니다.

단순 채팅 앱이나 IDE 안에 묶인 코딩 보조 도구라기보다:

- 도구를 호출하고
- 기억을 쌓고
- skills를 재사용하고
- 여러 플랫폼에서 동작하고
- 필요하면 subagent로 작업을 나누는

장기 실행형 agent framework 에 가깝습니다.

## 공식 문서가 설명하는 Hermes의 핵심

Hermes 공식 문서 기준으로 핵심 포인트는 아래입니다.

- self-improving agent
- built-in learning loop
- persistent memory
- skills system
- context files
- scheduled tasks
- subagent delegation
- messaging gateway
- provider-agnostic model routing

즉, 모델 하나에 얇게 붙는 채팅 UI가 아니라, 기억 + 도구 + 자동화 + 실행 환경까지 포함한 agent 시스템으로 이해하면 됩니다.

## Hermes는 무엇을 할 수 있나요?

Hermes Features Overview 기준으로 주요 기능은 아래와 같습니다.

### 1. Tools / Toolsets

- 터미널 실행

- 파일 읽기/쓰기
- 웹 검색/추출
- 브라우저 자동화
- 이미지/음성 관련 도구

즉, 모델이 답만 하는 것이 아니라 실제로 작업을 수행할 수 있습니다.

### 2. Skills System

- 필요한 지식을 on-demand로 로드
- 토큰 낭비를 줄이기 위한 progressive disclosure
- 재사용 가능한 플레이북 축적

핸즈온 관점에서는 이 부분이 특히 중요합니다.

### 3. Persistent Memory

- `MEMORY.md`
- `USER.md`

같은 형태로 세션 간 기억을 이어 갑니다.

### 4. Context Files

- `.hermes.md`
- `AGENTS.md`
- `CLAUDE.md`
- `SOUL.md`

같은 파일을 읽어 프로젝트 규칙과 정체성을 반영합니다.

### 5. Delegation / Automation

- `delegate_task`로 child agent 생성
- `cron` 기반 자동화
- 여러 플랫폼으로 결과 전달

즉 Hermes는 “한 번 답하는 도구”보다 “계속 일하는 agent”에 더 가깝습니다.

## 다른 도구와 무엇이 다른가요?

짧게 비교하면:

- `LM Studio`: 로컬 모델 실행/관리 GUI
- `Ollama`: 로컬 모델 실행 CLI/API
- `llama.cpp`: GGUF 실행 엔진 및 로컬 서버
- `OpenCode`: 코딩 중심 에이전트
- `Hermes`: 코딩도 할 수 있지만, 더 넓게는 에이전트 운영체제에 가까움

즉:

- `LM Studio`, `Ollama`, `llama.cpp`는 주로 추론 서버
- `Hermes`는 그 위에 붙는 agent layer

라고 보면 됩니다.

## 이번 핸즈온에서 Hermes를 어떻게 보나요?

이번 핸즈온에서는 `Hermes`를 필수 준비 항목으로 보지 않습니다.

이유:

- Windows에서는 WSL2가 필요합니다.
- 설치와 설정할 것이 많습니다.
- 처음 준비할 때 모두가 설치하기에는 연결 과정이 복잡합니다.

반대로 강점도 분명합니다.

- role-based agent workflow를 구성하기 좋음
- skills 실습 주제로 적합함
- 로컬 OpenAI 호환 서버와 연결 가능
- `Gemma 4 + llama.cpp` 또는 `Gemma 4 + LM Studio` 위에 agent layer를 얹을 수 있음

## 이번 핸즈온에서 Hermes를 어떤 그림으로 이해하면 되나요?

가장 쉬운 그림은 이렇습니다.

```text
Gemma 4
  -> local inference server
     (LM Studio / Ollama / llama.cpp)
        -> Hermes Agent
           -> tools, skills, memory, delegation
```

즉 `Gemma 4`는 두뇌이고, `Hermes`는 실행기이자 오케스트레이터입니다.

## 왜 필수 준비 항목이 아닌가요?

- 처음 준비한다면 `LM Studio`만으로도 충분히 세션을 따라올 수 있습니다.
- `Hermes`는 설치뿐 아니라 모델 endpoint 연결까지 이해해야 합니다.
- Windows에서는 WSL2까지 준비해야 합니다.
- 핸즈온 60분 안에 모두에게 같은 수준의 안정성을 기대하기 어렵습니다.

그래서 문서 흐름상으로도:

1. 모델과 실행 환경 이해
2. 기본 설치 먼저 완료
3. 그 다음 `Hermes`로 확장

순서가 자연스럽습니다.

## 이런 경우에 특히 잘 맞습니다

- 에이전트 워크플로에 관심 있는 경우
- `skills`를 직접 만들어 보고 싶은 경우
- role-based workflow를 구성해 보고 싶은 경우
- 로컬 모델 위에 agent system을 얹어 보고 싶은 경우

## 다음에 무엇을 보면 되나요?

- 실제 설치는 Hermes Agent 설치 가이드
- 로컬 서버는 llama.cpp 설치 가이드
- 코드 에이전트 기대치는 Gemma 4 벤치마크와 코딩 에이전트 기대치

## 공식 참고 링크

- Hermes Docs Home
- Features Overview
- Quickstart

문서 13

docs/13-hermes-agent-setup.md

# Hermes Agent 설치 가이드

Hermes가 어떤 도구인지 먼저 이해하고 싶다면 Hermes Agent란 무엇인가 문서를 먼저 확인해 주세요.

## 이 문서는 누구를 위한 문서인가요?

- Hermes Agent를 이번 핸즈온에서 추가로 써 보고 싶은 분
- Gemma 4 + llama.cpp 또는 다른 OpenAI 호환 로컬 서버에 Hermes를 연결해 보고 싶은 분
- CEO / CPO / PM / QA / Developer 같은 역할 기반 AI office 데모를 직접 구성해 보고 싶은 분

## 먼저 결론

- Hermes Agent는 이번 세션에서 필수 준비 항목이 아닙니다.
- macOS, Linux, WSL2에서는 설치가 비교적 단순합니다.
- Windows 네이티브는 공식 미지원이며, Windows 사용자는 WSL2 안에서 설치해야 합니다.
- 발표자 데모나 고급 실습에는 좋지만, 처음 준비할 때 기본으로 두기에는 설치와 연결 과정이 복잡합니다.

## 사전 안내

- 행사 전에 반드시 설치를 끝내고 오세요.
- 행사 전에 최소 1회 hermes 실행 테스트를 해 두세요.
- Hermes만 설치된 상태로는 로컬 모델 추론이 되지 않습니다.
- 로컬 모델을 쓰려면 Ollama, LM Studio, llama.cpp, vLLM 같은 별도 inference server가 필요합니다.

## 공식 지원 범위

### macOS / Linux / WSL2

공식 one-line installer를 사용할 수 있습니다.

```bash
curl -fsSL
https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh | bash
```

설치가 끝나면 셸을 다시 읽습니다.

```bash
source ~/.zshrc
```

또는 bash 사용자라면:

```bash
source ~/.bashrc
```

### Windows

Windows 네이티브는 공식 미지원입니다. 먼저 `WSL2`를 설치한 뒤, WSL 터미널 안에서 위 설치 명령을 실행해야 합니다.

Windows 사용자는 Hermes = WSL2 전용 선택 경로로 이해하면 됩니다.

PowerShell 관리자 권한에서 먼저 할 일:

```powershell
wsl --install -d Ubuntu
```

설치가 멈추거나 Store 경로가 불안정하면:

```powershell
wsl --install --web-download -d Ubuntu
```

설치 후 상태 확인:

```powershell
wsl --status
wsl --list --verbose
```

Ubuntu 실행:

```powershell
wsl -d Ubuntu
```

## 설치 전에 확인할 것

- `git --version`
- 관리자 권한 또는 설치 권한
- 안정적인 인터넷
- 여유 디스크 공간
- macOS / Linux / WSL2 셸 사용 가능 여부

`Hermes` 설치 스크립트는 공식 문서 기준으로 아래를 자동 처리합니다.

- uv
- Python 3.11
- Node.js
- `ripgrep`
- `ffmpeg`
- `hermes` 명령 연결
- setup wizard 실행

자동 설치가 실패하거나 uv만 미리 준비하고 싶다면 uv 설치 가이드를 먼저 확인하세요.

# 설치 순서

## 1. Windows 사용자는 먼저 WSL2 준비

PowerShell 관리자 권한:

```powershell
wsl --install -d Ubuntu
```

설치 확인:

```powershell
wsl --status
wsl --list --verbose
```

Ubuntu 셸 열기:

```powershell
wsl -d Ubuntu
```

이후 명령은 WSL 터미널 안에서 실행합니다.

macOS / Linux 사용자는 이 단계를 건너뛰고 바로 다음 단계로 가면 됩니다.

## 2. 설치 실행

```bash
curl -fsSL
https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/install.sh | bash
```

## 3. 셸 재로드

```bash
source ~/.zshrc
```

또는:

```bash
source ~/.bashrc
```

## 4. 설정 마법사 또는 모델 설정

처음 설치 후 setup wizard가 열리면 기본 모델 공급자를 설정할 수 있습니다.

나중에 다시 바꾸려면:

```bash
hermes model
```

### 5. 설치 확인

```bash
hermes version
hermes doctor
```

## 로컬 모델과 연결하는 법

이번 핸즈온에서는 `Hermes`가 직접 모델을 돌리는 것이 아니라, 별도 로컬 서버에 붙는 구조로 이해하면 됩니다.

중요:

- Hermes Agent는 다단계 tool-calling 작업을 위해 최소 64K 토큰 컨텍스트를 요구합니다.
- 로컬 모델 서버의 컨텍스트가 이보다 작으면 Hermes가 시작 단계에서 거부하거나, 작업 중 불안정하게 동작할 수 있습니다.
- 컨텍스트를 키우면 메모리 사용량도 함께 늘어나므로, 8GB/16GB 장비에서는 Hermes 경로를 무리하게 권장하지 않습니다.

도구별 확인 예시:

- `llama.cpp`: `llama-server ... --ctx-size 65536`
- `Ollama`: 필요 시 `OLLAMA_CONTEXT_LENGTH=64000 ollama serve`처럼 64K 이상으로 서버를 띄운 뒤 연결
- `LM Studio`: 서버를 켜기 전에 모델 설정에서 context length를 64K 이상으로 설정할 수 있는지 확인

따라서 Hermes와 연결할 로컬 서버는 아래 두 가지를 먼저 확인해야 합니다.

1. `/v1/models`에서 실제 model ID가 보이는지
2. 해당 서버/모델이 64K 이상 컨텍스트로 실행 중인지

예를 들어 `llama.cpp`의 `llama-server`를 `127.0.0.1:8080/v1`에 띄웠다면:

```bash
hermes model
```

입력 예시:

- Base URL: `http://127.0.0.1:8080/v1`
- API key: none
- Model name: `/v1/models`에서 보이는 모델 ID
- Context length: `64000` 또는 `65536` 이상

즉 구조는 아래와 같습니다.

```text
Hermes Agent -> OpenAI 호환 로컬 서버 -> Gemma 4
```

## Windows에서 권하는 방식

Windows에서 `Hermes`를 꼭 쓰고 싶다면 아래 방식으로 준비하세요.

1. Windows에서 `LM Studio` 또는 `llama.cpp` 서버 실행
2. WSL2 안에서 `Hermes` 실행
3. `Hermes`가 로컬 서버 주소에 연결

이 방식은 WSL 안에서 대용량 모델을 다시 내려받고 빌드하는 부담을 줄여 줍니다.

Windows PowerShell에서 WSL 진입 예시:

```powershell
wsl -d Ubuntu
```

그 다음 WSL 안에서:

```bash
hermes version
hermes doctor
```

## 행사 전 최종 확인

아래가 되면 준비 완료입니다.

### Hermes 준비 완료 기준

macOS / Linux / WSL2:

```bash
hermes version
hermes doctor
```

그 다음 `hermes` 또는 `hermes --tui`를 실행해 짧은 질문이 응답되는지 확인하세요.

Windows 사용자는 위 명령을 PowerShell이 아니라 WSL2 Ubuntu 셸 안에서 실행하면 됩니다.

로컬 모델 연결까지 확인하려면:

```bash
hermes model
```

에서 로컬 서버 주소를 등록한 뒤, 간단한 질문이 한 번 응답되면 충분합니다.

## 이런 경우에 해 보세요

- 발표자 데모
- 로컬 에이전트, skills, delegation, AI office 워크플로를 직접 구성해 보고 싶은 경우

## 이런 경우에는 나중에 해도 됩니다

- 당일 처음 WSL2를 설치해야 하는 경우
- 60분 안에 GUI 중심으로만 간단한 체험을 원하는 경우

- 에이전트보다 기본 로컬 추론만 빨리 체험하고 싶은 경우

## 공식 참고 링크

- Hermes Installation
- Hermes Quickstart
- Hermes CLI Commands Reference
- Hermes FAQ
- Hermes Docs Home

문서 14

docs/14-gemma4-architecture-deep-dive.md

# Gemma 4 아키텍처 상세 정리

## 이 문서는 누구를 위한 문서인가요?

- Gemma 4의 구조를 조금 더 깊게 이해하고 싶은 분
- E2B, E4B, 26B A4B, 31B가 왜 다른지 알고 싶은 분
- 설치 가이드보다 모델 자체의 설계 차이가 궁금한 분

## 먼저 알아둘 점

- 정확한 모델별 수치는 Google 공식 모델 카드 기준으로 정리했습니다.
- Hugging Face의 Gemma 4 blog 와 Maarten Grootendorst의 A Visual Guide to Gemma 4 는 이해를 돕는 설명 자료로 함께 참고했습니다.
- 따라서 아래 문서에서는:
- 모델 크기, 레이어 수, 컨텍스트, modality 같은 항목은 공식 모델 카드 기준
- 왜 이런 구조가 나왔는지에 대한 해설은 블로그/설명 자료를 참고한 해석

으로 이해하면 됩니다.

## 먼저 용어부터 보기

모델 구조 용어는 아래 흐름으로 이해하면 됩니다. 세부 구현 수식보다 "어떤 계산을 언제 아끼는가"에 초점을 두면 훨씬 읽기 쉽습니다.

- `Dense`: 모델의 주요 계산 블록을 대부분 함께 쓰는 일반적인 구조입니다. 구조가 단순하지만, 큰 모델일수록 계산과 메모리 부담이 커집니다.
- `MoE`: "여러 전문가 중 필요한 일부만 골라 쓰는 구조"입니다. 모델 안에 여러 전문가 블록(expert)이 있고, 답을 만들 때마다 그중 일부만 계산합니다.
- B: Billion의 줄임말입니다. 26B는 대략 260억 개 파라미터를 뜻합니다. B는 파일 크기 단위인 GB가 아닙니다.
- `active parameters`: 답을 만들 때 그 순간 실제 계산에 참여하는 파라미터입니다.
- `effective parameters`: 작은 모델의 체감 계산 규모를 설명하기 위한 표기입니다.
- RoPE: token 위치 정보를 attention 계산 안에 넣는 방식입니다. 긴 문맥에서 "이 token이 앞쪽에 있었는지, 가까운지, 먼지"를 다루는 데 중요합니다.
- PLE: E2B/E4B가 작은 모델 효율을 높이기 위해 쓰는 보조 장치입니다. 세부 구조보다 "작은 모델을 더 효율적으로 쓰기 위한 장치"로 이해하면 됩니다.

## 모델 이름의 B는 무엇인가요?

B는 `Billion`의 줄임말입니다. 한국어로는 10억입니다.

- E2B: 약 20억 개 규모
- E4B: 약 40억 개 규모
- 26B: 약 260억 개 규모
- 31B: 약 310억 개 규모

여기서 말하는 것은 모델 파일 크기가 아니라 파라미터 수입니다. 파라미터는 모델이 학습하면서 조정한 숫자 값입니다. 모델은 이 숫자들을 사용해서 "다음에 어떤 token이 올 가능성이 높은지"를 계산합니다.

쉽게 말하면:

- B는 모델 안에 들어 있는 학습된 숫자의 규모
- GB는 파일 크기나 메모리 용량
- 26B는 26GB 파일이라는 뜻이 아님

보통 파라미터가 많을수록 더 복잡한 패턴을 담을 수 있습니다. 그래서 큰 모델은 reasoning, coding, 긴 문맥 이해에서 더 좋은 결과를 낼 가능성이 큽니다.

하지만 그만큼:

- 모델 파일이 커지고
- 메모리를 더 많이 쓰고
- 한 token을 만들 때 계산도 더 무거워질 수 있습니다

그래서 로컬 실습에서는 "큰 모델이 무조건 좋다"보다 "내 노트북에서 안정적으로 돌아가는 모델"을 고르는 것이 더 중요합니다.

## Dense는 무엇인가요?

Transformer decoder block은 크게 보면 attention과 feed-forward network(FFN)로 이루어집니다. Dense 모델에서는 각 layer의 FFN이 하나의 큰 블록이고, 모든 token이 그 블록을 통과합니다.

쉽게 말하면:

- 모든 token이 같은 계산 경로를 지나갑니다.
- 구조가 단순하고 예측하기 쉽습니다.
- 모델이 커질수록 품질은 좋아질 수 있지만, 매번 계산해야 하는 양도 같이 커집니다.

Gemma 4에서는 31B가 대표적인 상위 dense 모델입니다. E2B, E4B도 dense 계열이지만, 작은 장비에서 더 잘 돌도록 PLE 같은 효율화 장치가 붙어 있습니다.

Dense 모델을 고를 때의 직관:

- 장점: 구조가 단순하고 안정적입니다.
- 단점: 큰 모델은 매 token마다 큰 FFN을 계속 계산해야 합니다.
- 실습 해석: 31B는 품질이 좋지만, 일반 노트북에서는 메모리와 속도 부담이 큽니다.

## MoE는 무엇인가요?

MoE는 Mixture of Experts의 줄임말입니다. Dense 모델처럼 하나의 큰 FFN만 두는 대신, 여러 개의 작은 FFN expert를 두고 token마다 일부 expert만 골라 계산합니다.

중요한 점은 MoE가 모델 전체를 아무렇게나 켰다 껐다 하는 구조가 아니라는 것입니다. 주로 FFN 부분을 여러 expert로 나누고, attention 등 나머지 구조는 계속 함께 동작합니다.

Gemma 4 26B A4B는 MoE 구조입니다.

- 전체 파라미터: 약 26B
- 실제 계산 참여 파라미터: 약 4B

- expert 수: 128개
- 한 번에 선택되는 expert: 8개
- 공통 expert: 항상 켜지는 shared expert 1개

그래서 26B A4B는 이름 그대로:

- 메모리에는 26B급 모델을 올려야 하지만
- 매 token 계산에서는 약 4B 규모만 활성화하는 구조

라고 보면 됩니다.

## MoE는 expert를 어떻게 고르나요?

MoE layer 안에는 `router`가 있습니다. router는 token embedding을 보고 어떤 expert를 쓸지 점수를 매깁니다.

개념적으로는 아래 순서입니다.

1. token embedding이 MoE layer에 들어옵니다.
2. router가 각 expert에 대한 점수 또는 확률을 계산합니다.
3. 점수가 높은 expert 일부를 고릅니다.
4. 선택된 expert들이 token을 처리합니다.
5. expert별 결과를 router 점수로 가중해서 합칩니다.
6. shared expert 결과도 함께 더해집니다.

이 선택은 token 단위로 일어납니다. 같은 문장 안에서도 token마다 선택되는 expert가 달라질 수 있고, layer마다 선택도 달라질 수 있습니다.

몇 가지 오해를 피해야 합니다.

- 사람이 "이번에는 3번 expert를 써줘"처럼 직접 고르는 구조가 아닙니다.
- expert가 반드시 "코딩 담당", "한국어 담당"처럼 사람이 읽기 좋은 역할로 나뉜다고 보장되지는 않습니다.
- 라우터는 학습 과정에서 좋은 결과가 나오도록 expert 선택 방식을 같이 배웁니다.
- MoE라고 해서 메모리까지 4B 모델처럼 줄어드는 것은 아닙니다. 선택되지 않은 expert도 가중치로는 메모리에 올라가야 합니다.

실습에서 중요한 해석은 단순합니다.

- 속도: 매번 전체 26B를 계산하지 않으므로 유리합니다.
- 품질: 전체 모델 용량은 크기 때문에 작은 dense 모델보다 더 좋은 답을 기대할 수 있습니다.
- 메모리: 전체 expert 가중치를 들고 있어야 하므로 26B급 부담이 남습니다.

## RoPE는 무엇인가요?

`RoPE`는 `Rotary Positional Embedding`의 줄임말입니다. Transformer는 token 순서를 그냥 문자열처럼 읽지 않습니다. attention 계산에서 "이 token이 몇 번째 위치에 있었는지"를 알 수 있어야 합니다.

RoPE는 위치 번호를 별도 값으로 붙이는 대신, query/key 벡터 일부를 위치에 따라 회전시키는 방식입니다. 그래서 attention이 두 token의 내용뿐 아니라 위치 관계도 같이 보게 됩니다.

쉽게 말하면:

- 가까운 token끼리는 가까운 위치 관계로 보이고
- 멀리 떨어진 token은 다른 회전 패턴을 갖고
- 모델은 이 패턴을 통해 상대적인 거리와 순서를 파악합니다

긴 context 모델에서 RoPE가 중요한 이유는, 위치 정보가 흔들리면 긴 문서 뒤쪽의 token을 제대로 다루기 어렵기 때문입니다. 128K, 256K context를 지원하려면 attention 구조뿐 아니라 position encoding도 긴 문맥에 맞게 설계되어야 합니다.

## Dual RoPE는 무엇인가요?

Hugging Face Gemma 4 블로그 기준으로 Gemma 4는 `Dual RoPE` 구성을 사용합니다.

- sliding-window layer: 표준 RoPE
- global full-context layer: pruned RoPE, 또는 low-frequency-pruned RoPE

이렇게 나누는 이유는 layer마다 하는 일이 다르기 때문입니다.

`sliding-window layer`는 가까운 주변 token을 자세히 봅니다. 여기서는 표준 RoPE처럼 위치 차이를 비교적 촘촘하게 다루는 방식이 잘 맞습니다.

`global layer`는 긴 문맥 전체를 넓게 봅니다. 긴 거리에서는 너무 빠르게 변하는 고주파 위치 성분이 오히려 불안정해질 수 있습니다. 그래서 global layer에서는 긴 context에 더 맞게 일부 위치 성분을 줄인 RoPE를 씁니다.

정리하면:

- 표준 RoPE는 가까운 문맥을 정확히 보는 데 유리합니다.
- pruned RoPE는 긴 문맥을 더 안정적으로 보는 데 유리합니다.
- Gemma 4는 local/global attention 구조에 맞춰 RoPE도 다르게 씁니다.

이 구조가 있어서 128K, 256K 같은 긴 context를 그냥 숫자만 늘린 것이 아니라, attention과 position encoding을 함께 조정한 설계로 볼 수 있습니다.

## 한눈에 보는 구조 차이

| 모델 | 구조 | 핵심 포인트 |
|---|---|---|
| E2B | Dense + PLE | 작은 온디바이스용, audio 지원, 128K context |
| E4B | Dense + PLE | 작은 온디바이스용, audio 지원, 128K context |
| 26B A4B | MoE | 총 26B지만 답을 만들 때는 약 4B 규모만 계산, 속도/성능 균형형 |
| 31B | Dense | 가장 단순하고 강한 상위 모델, 품질 우선 |

핵심:

- 작은 모델(E2B, E4B)은 PLE 로 효율을 높인 구조입니다.
- 26B A4B는 여러 전문가 중 일부만 골라 쓰는 MoE 구조입니다.
- 31B는 모든 주요 계산 블록을 함께 쓰는 상위 모델입니다.

## Gemma 4 전체가 공통으로 갖는 구조

공식 모델 카드와 Hugging Face 기술 블로그 기준으로, Gemma 4 전반에는 아래 같은 공통 특성이 있습니다.

## 1. Hybrid Attention

Gemma 4는 local sliding-window attention 과 global full attention 을 섞어 씁니다.

- local layer는 계산량과 메모리를 줄이는 역할
- global layer는 긴 문맥과 전체 구조를 다시 보는 역할
- 마지막 레이어는 global attention 이 되도록 설계되었습니다
- E2B/E4B의 sliding window는 512 token
- 26B A4B/31B의 sliding window는 1024 token

이 구조 덕분에:

- 긴 컨텍스트를 지원하면서도
- 무조건 모든 레이어가 full attention 인 모델보다
- 더 효율적으로 동작합니다.

조금 더 풀어 말하면, local layer는 가까운 token만 봅니다. 그래서 빠르고 가볍습니다. 하지만 local layer만 계속 쌓으면 앞쪽 정보가 여러 layer를 거쳐 간접적으로 전달되어야 하므로, 긴 문서 전체 구조를 놓칠 수 있습니다.

그래서 중간중간 global layer를 넣습니다. global layer는 전체 문맥을 다시 직접 보면서 local layer가 놓칠 수 있는 큰 흐름을 잡아 줍니다.

## 2. Long Context용 RoPE 설계

RoPE는 문장 안에서 각 토큰이 어느 위치에 있는지 모델이 이해하도록 돕는 방식입니다.

Hugging Face Gemma 4 블로그 기준으로 Gemma 4는:

- sliding layer에는 표준 RoPE
- global layer에는 긴 문맥에 맞춘 pruned RoPE / low-frequency-pruned RoPE

를 사용합니다.

Maarten Grootendorst 설명 자료 기준으로는 global attention layer에서 `p=0.25`인 p-RoPE를 사용합니다. 즉 모든 벡터 차원에 위치 회전을 넣는 대신 일부 차원에만 RoPE를 적용해, 긴 context에서 위치 정보가 과하게 흔들리지 않도록 하는 방식입니다.

실습에서는:

- “128K / 256K 문맥을 그냥 억지로 늘린 게 아니라”
- 긴 문맥을 버티기 위한 attention + positional encoding 조합이 들어갔다

정도로 이해하면 충분합니다.

## 3. Global attention 효율화

Global attention은 전체 context를 보기 때문에 비용이 큽니다. Gemma 4는 이 부분을 줄이기 위해 몇 가지 장치를 같이 씁니다.

- `GQA`: 여러 query head가 key/value head를 공유해 KV cache 부담을 줄입니다.
- `K=V`: 설명 자료 기준으로 global attention layer에서 key와 value를 같게 두어 cache 부담을 더 줄입니다.
- `p-RoPE`: global layer에서 긴 context를 더 안정적으로 다루기 위해 위치 회전 적용 범위를 줄입니다.

이 조합은 모두 같은 방향을 봅니다.

- 긴 context를 지원해야 함
- global attention 품질은 유지해야 함
- 하지만 로컬/온디바이스 실행을 위해 메모리와 계산량은 줄여야 함

## 4. Shared KV Cache

KV cache는 모델이 이미 읽은 문맥을 다시 계산하지 않도록 저장해 두는 내부 메모리입니다.

Hugging Face blog 기준으로 Gemma 4는 shared KV cache 를 사용합니다.

- 마지막 일부 레이어가 자기만의 key/value projection을 다시 만들지 않고
- 앞쪽 레이어의 KV 상태를 재사용합니다

의미:

- 긴 문맥에서 메모리와 계산량 절감
- 온디바이스/로컬 추론에 유리

## 5. Vision Encoder

Gemma 4는 전 모델이 image 입력을 지원합니다.

핵심 특징:

- 가변 aspect ratio
- 가변 resolution
- 고정된 soft token budget 으로 이미지 정보를 요약

Hugging Face blog 기준 대표 token budget:

- `70`
- `140`
- `280`
- `560`
- `1120`

즉, 같은 이미지 모델이라도:

- 더 적은 시각 토큰으로 빠르게 볼 수도 있고
- 더 많은 시각 토큰으로 자세히 볼 수도 있습니다.

## 6. Audio Encoder

audio 는 E2B / E4B 에만 있습니다.

Hugging Face blog 기준:

- USM-style conformer 기반 audio encoder
- Gemma-3n 계열과 유사한 audio 처리 경로

즉:

- 작은 모델은 text + image + audio
- 큰 모델은 text + image

로 이해하면 됩니다.

## 모델별 공식 구조 요약

Google 공식 Gemma 4 모델 카드 기준:

### Dense 모델

Dense 모델은 답을 만들 때 모델의 주요 계산 블록을 대부분 함께 쓰는 구조입니다.

| 항목 | E2B | E4B | 31B |
|---|---|---|---|
| 총 파라미터 | 2.3B effective (5.1B with embeddings) | 4.5B effective (8B with embeddings) | 30.7B |
| 레이어 수 | 35 | 42 | 60 |
| sliding window | 512 | 512 | 1024 |
| 컨텍스트 | 128K | 128K | 256K |
| vocab | 262K | 262K | 262K |
| modality | Text, Image, Audio | Text, Image, Audio | Text, Image |
| vision encoder | ~150M | ~150M | ~550M |
| audio encoder | ~300M | ~300M | 없음 |

### MoE 모델

| 항목 | 26B A4B |
|---|---|
| 총 파라미터 | 25.2B |
| 실제 계산 참여 파라미터 | 3.8B |
| 레이어 수 | 30 |
| sliding window | 1024 |
| 컨텍스트 | 256K |
| vocab | 262K |
| 전문가 블록 구성 | 128개 중 8개 선택 + 공통 전문가 블록 1개 |
| modality | Text, Image |
| vision encoder | ~550M |

## A4B는 무슨 뜻인가요?

26B A4B는 전체 규모는 26B급이지만, 답을 만들 때 실제로 계산에 참여하는 부분은 약 4B 규모인 모델이라는 뜻입니다.

- 26B: 모델 전체가 가진 총 파라미터 규모입니다. 공식 카드 기준 실제 수치는 25.2B입니다.
- A: active, 즉 실제 계산에 참여한다는 뜻입니다.

- 4B: 답을 만들 때 계산에 참여하는 파라미터 규모입니다. 공식 카드 기준 실제 수치는 3.8B입니다.

이 차이는 26B A4B가 MoE(Mixture-of-Experts) 구조이기 때문에 생깁니다. 모델 안에는 여러 전문가 블록이 있고, 답을 만들 때마다 그중 일부만 골라 계산합니다. 공식 카드 기준으로는 전체 128개 전문가 블록 중 8개와 1개 공통 전문가 블록을 사용합니다.

실습에서 가장 중요한 해석은 아래입니다.

- 속도: 답을 만들 때 약 4B 규모만 계산하므로 31B보다 빠르게 느껴질 수 있습니다.
- 품질: 전체 모델 규모는 26B급이라 E2B/E4B보다 더 좋은 코딩과 에이전트 체감을 기대할 수 있습니다.
- 메모리: 전체 26B 규모의 가중치를 메모리에 올려야 하므로, 4B 모델처럼 가볍게 실행되는 것은 아닙니다.

## E2B / E4B는 왜 “effective” 인가요?

공식 모델 카드와 Hugging Face blog 기준으로, 작은 모델에는 Per-Layer Embeddings (PLE) 가 들어갑니다.

이 구조 때문에:

- 총 embedding 테이블 크기는 꽤 크지만
- 실제 추론에서 답을 만들 때 무겁게 계산되는 파라미터는 더 적습니다

그래서 공식 카드도:

- `E2B = 2.3B effective (5.1B with embeddings)`
- `E4B = 4.5B effective (8B with embeddings)`

처럼 표기합니다.

직관적으로는:

- “모델 본체는 더 작게 유지하면서”
- “레이어별 추가 조건 신호를 embedding lookup 방식으로 넣어”
- 온디바이스 효율을 높인 구조

라고 이해하면 됩니다.

Hugging Face blog 해설 기준으로 PLE는:

- 각 token에 대해 레이어마다 작은 보조 벡터를 공급하고
- 각 레이어가 자기 시점에 필요한 token-specific 정보를 더 잘 받게 해줍니다

즉, 작은 모델에서 깊이를 무작정 늘리지 않고 효율을 높이는 장치라고 볼 수 있습니다.

## 26B A4B는 왜 빠를 수 있나요?

26B A4B는 위에서 설명한 MoE 구조를 사용합니다.

의미:

- 모델 전체는 26B급으로 크지만
- 매 토큰마다 전체를 다 활성화하지 않고
- 일부 전문가 블록만 선택적으로 계산합니다

그래서:

- 31B보다 더 가볍게 느껴질 수 있고
- 로컬 환경에서는 속도/품질 균형이 좋을 수 있습니다

다만 중요한 점:

- 메모리에는 전체 파라미터가 올라가야 하므로
- "4B 모델처럼 가볍게 메모리만 쓰는 것"은 아닙니다

즉:

- 속도 관점에서는 4B active의 장점
- 메모리 관점에서는 26B total의 부담

을 동시에 갖습니다.

## 31B는 왜 따로 의미가 있나요?

31B는 Gemma 4 계열의 상위 모델입니다.

장점:

- 구조가 비교적 단순하고 해석이 쉬움
- 최고 수준의 일반 성능
- coding / reasoning / long-context 품질이 가장 좋음

공식 카드와 Hugging Face 설명을 같이 보면:

- 31B는 "특별한 트릭"보다
- Gemma 4 공통 구조를 더 큰 규모로 밀어붙인 상위 모델

로 이해하면 됩니다.

이번 핸즈온에서는:

- 가능한 하드웨어가 있으면 가장 품질이 좋음
- 다만 일반 노트북에서는 부담이 큼

## Vision 쪽에서 Gemma 4가 중요한 이유

Gemma 4는 text-only 모델이 아니라 multimodal 모델입니다.

특히 image 쪽에서 중요한 점:

- 정사각형으로 무조건 찌그러뜨리지 않고
- 원본 aspect ratio를 더 잘 보존하려고 하고
- token budget을 선택해 속도/메모리/품질을 조절할 수 있습니다

그래서:

- OCR
- 문서 읽기

- UI 이해
- 화면 인식

같은 시나리오에서 활용도가 높습니다.

## 핸즈온 관점에서 이 구조 차이가 왜 중요할까요?

### E2B / E4B

- 로컬 노트북 친화적
- audio 지원
- 하지만 코딩 에이전트나 복잡한 계획에는 한계가 더 빨리 드러날 수 있음

### 26B A4B

- 로컬에서 코딩/에이전트 체감을 노리기 좋은 균형형
- 속도와 품질의 절충안으로 보기 좋음

### 31B

- 가장 좋은 품질
- 발표자 데모나 고사양 환경에 적합

## 한 줄 해석

- E2B/E4B: 작은 온디바이스 모델 + PLE
- 26B A4B: 빠른 상위 MoE
- 31B: 가장 강한 상위 모델

## 같이 보면 좋은 문서

- 하드웨어 / 운영체제 / 모델 선택 가이드
- 메모리 기준으로 Gemma 4 모델 고르는 방법
- Gemma 4 벤치마크와 코딩 에이전트 기대치

## 참고 링크

- Google Gemma 4 E2B model card
- Google Gemma 4 26B A4B model card
- Google Gemma 4 31B model card
- Gemma 4 documentation
- Hugging Face blog: Gemma 4
- A Visual Guide to Gemma 4

문서 15

docs/15-gemini-cli-gemma-routing-prep.md

# Gemini CLI Gemma 4 Preview 및 Gemma 라우팅 사전 준비 가이드

기준 확인일: 2026-05-01

## 이 문서는 누구를 위한 문서인가요?

- 이미 `Gemini CLI`를 쓰고 있거나, 행사 전에 추가로 설치해 보고 싶은 분
- Gemini CLI preview에서 `Gemma 4` 모델 선택을 확인해 보고 싶은 분
- Gemma 4 모델이 선택은 되지만 실제 응답에서 막힐 수 있다는 점을 미리 알고 싶은 분
- `gemini gemma setup`이 실제로 무엇을 내려받는지 헷갈리는 분
- `LM Studio`나 `Ollama` 기본 준비와 별개로, Gemini CLI의 Gemma 실험 기능을 미리 확인하고 싶은 분

## Gemini CLI란 무엇인가요?

Gemini CLI는 Google의 Gemini 모델을 터미널에서 직접 사용할 수 있게 해 주는 오픈소스 AI 에이전트입니다. 단순히 질문과 답변만 주고받는 채팅 앱이 아니라, 개발자가 터미널 안에서 자연어로 작업을 요청하고, 필요하면 파일 읽기/수정, 셸 명령 실행, 웹 검색/가져오기 같은 도구를 함께 사용할 수 있게 만든 CLI 도구입니다.

공식 문서 기준으로 Gemini CLI는 아래처럼 구성됩니다.

- 사용자는 터미널에서 `gemini` 명령으로 대화형 세션을 시작합니다.
- CLI 프런트엔드는 로컬의 core 프로세스와 통신합니다.
- core는 Gemini API와 모델 호출, 파일 시스템 도구, 셸 도구, 웹 도구 등을 관리합니다.
- `/model`, `/settings`, `/gemma` 같은 슬래시 명령으로 세션 중 설정과 상태를 바꿀 수 있습니다.

이번 행사 준비 관점에서 중요한 점:

- Gemini CLI는 `LM Studio`나 `Ollama`처럼 Gemma 4 모델 파일을 내려받아 로컬 채팅 앱으로 실행하는 기본 준비 방식이 아닙니다.
- Gemini CLI의 일반 대화와 도구 사용은 Gemini 계정/API 키/조직 설정의 영향을 받습니다.
- preview에서 Gemma 4 모델을 선택할 수 있어도, 실제 호출 권한은 별도로 필요합니다.
- `gemini gemma setup`은 Gemma 4 로컬 실행이 아니라, Gemma 3 기반 로컬 라우터를 준비하는 경로입니다.

따라서 Gemini CLI는 이번 세션에서 개발자용 추가 실험 항목으로 보는 것이 가장 안전합니다.

## 먼저 결론

Gemini CLI의 Gemma 관련 기능은 현재 두 갈래로 나눠서 이해해야 합니다.

| 구분 | 필요한 버전/명령 | 확인되는 모델 | 의미 |
|---|---|---|---|
| Gemma 4 모델 선택 | `@google/gemini-cli@preview` + `experimental.gemma` | `gemma-4-26b-a4b-it`, `gemma-4-31b-it` | Gemini CLI에서 Gemma 4 모델을 실험적으로 선택하는 경로 |

| 구분 | 필요한 버전/명령 | 확인되는 모델 | 의미 |
|---|---|---|---|
| 로컬 Gemma 라우터 | `gemini gemma` 또는 `gemini gemma setup` | `gemma3-1b-gpu-custom` | LiteRT-LM으로 로컬 라우팅 판단용 Gemma 3 모델을 준비하는 경로 |
| Gemma 4 로컬 실행 | `LM Studio`, `Ollama`, `llama.cpp` | Gemma 4 E2B/E4B/26B/31B | 이번 행사 기본 로컬 실행 경로 |

중요:

- `v0.41.0-preview.0`에는 Gemma 4 모델 지원이 실험 기능으로 들어왔습니다.
- 하지만 `gemini gemma setup`에서 내려받는 모델은 여전히 `gemma3-1b-gpu-custom`으로 보입니다.
- 따라서 Gemma 4 모델 선택 지원과 Gemma 3 로컬 라우터 설치는 같은 기능이 아닙니다.
- Gemini CLI preview에서 Gemma 4 모델을 선택할 수 있어도, 그것이 Gemma 4를 LiteRT-LM으로 로컬 다운로드해 실행한다는 뜻은 아닙니다.
- `/model` 목록에서 Gemma 4를 고를 수 있고 선택 상태로 보이더라도, 실제 메시지를 보내면 권한 오류로 응답이 막힐 수 있습니다.
- 행사 기본 사전 준비는 여전히 `LM Studio` 또는 `Ollama`입니다.

## 선택 가능과 응답 가능은 다릅니다

현재 Gemini CLI preview에서는 `/model` 목록에 `gemma-4-26b-a4b-it`, `gemma-4-31b-it` 같은 Gemma 4 모델이 보이고, 선택까지는 정상적으로 되는 경우가 있습니다.

하지만 이것만으로 실제 실행 권한이 확인된 것은 아닙니다. 모델을 선택한 뒤 첫 메시지를 보냈을 때 아래처럼 접근 권한 오류가 날 수 있습니다.

```text
It seems like you don't have access to gemma-4-26b-a4b-it.
Your admin might have disabled the access. Contact them to enable the Preview Release
Channel.
```

이 경우는 "모델 선택 UI 확인"까지만 성공한 상태입니다. Gemma 4 preview를 실제로 쓸 수 있다고 보려면, 모델을 선택한 뒤 짧은 메시지까지 응답에 성공해야 합니다.

행사 준비 관점에서는 이 상태를 실패로 오래 붙잡지 마세요. Gemini CLI preview 권한 문제는 현장에서 바로 해결하기 어려울 수 있으므로, Gemma 4 로컬 실행은 `LM Studio` 또는 `Ollama`로 먼저 준비하는 편이 안전합니다.

## 공식 문서 기준으로 알아둘 점

Gemini CLI 공식 문서의 `Model routing`, `gemini gemma`, `Manual Local Model Routing` 문서를 함께 보면 구조가 조금 더 분명해집니다.

### 1. Gemini CLI에는 기본 모델 라우팅이 있습니다

Gemini CLI는 선택한 모델이 쿼터, 서버 오류 같은 이유로 실패할 때 다른 모델로 전환할 수 있는 모델 라우팅 기능을 갖고 있습니다. 이 기능은 기본으로 켜져 있으며, 모델 상태는 `ModelAvailabilityService`가 확인합니다.

흐름은 보통 아래처럼 이해하면 됩니다.

1. 현재 선택한 모델 호출이 실패합니다.
2. 정책에 따라 fallback 모델로 바꿀지 물어볼 수 있습니다.

3 전환을 승인하거나, 정책상 조용히 전환해도 되는 경우에는 현재 턴 또는 남은 세션에서 fallback 모델을 사용합니다.

다만 prompt completion, classification 같은 내부 보조 호출은 현재 선택한 대화 모델을 바꾸지 않고 조용히 대체 호출 흐름을 탈 수 있습니다. 공식 문서에는 `gemini-2.5-flash-lite`에서 실패하면 `gemini-2.5-flash`, `gemini-2.5-pro`로 넘어갈 수 있다고 설명되어 있습니다.

### 2. Local Model Routing은 "Gemma 4 로컬 채팅"이 아닙니다

공식 `gemini gemma` 문서에서 말하는 Local Model Routing은 로컬 Gemma 모델이 요청을 분류하고 라우팅 결정을 돕는 기능입니다. 예를 들어 단순한 파일 읽기 요청은 Flash 쪽으로, 복잡한 설계 논의는 Pro 쪽으로 보내는 식입니다.

여기서 로컬로 받는 모델은 현재 공식 문서 기준 `Gemma 3 1B` 계열인 `gemma3-1b-gpu-custom`입니다. 즉, `gemini gemma setup`은 Gemma 4 모델 파일을 받아서 로컬 채팅을 하는 명령이 아니라, 라우팅 판단용 로컬 분류기를 준비하는 명령입니다.

### 3. 자동 설정은 `gemini gemma setup`이 담당합니다

공식 문서 기준 `gemini gemma setup`은 아래 작업을 한 번에 처리합니다.

- LiteRT-LM 런타임 다운로드
- `gemma3-1b-gpu-custom` 모델 다운로드
- Gemma 약관 동의 확인
- Gemini CLI 설정 반영
- 로컬 LiteRT 서버 시작

설정이 끝나면 평소처럼 Gemini CLI를 쓰면 되고, 라우팅은 요청마다 자동으로 일어납니다. 로컬 LiteRT 서버가 내려가면 CLI는 오류로 멈추기보다 클라우드 분류기로 조용히 되돌아갈 수 있습니다.

## 행사 준비에서 어떻게 봐야 하나요?

Gemini CLI Gemma 기능은 흥미롭지만, 모두가 설치할 필요는 없습니다.

해 보면 좋은 경우:

- 개발자이거나 터미널 사용에 익숙한 경우
- CLI 도구 설치와 설정 변경이 부담스럽지 않은 경우
- Gemini CLI를 이미 쓰고 있는 경우
- preview 버전의 불안정성을 감안할 수 있는 경우
- Gemini CLI에서 Gemma 4 모델 선택 UI/명령을 확인해 보고 싶은 경우

먼저 하지 않아도 되는 경우:

- 설치를 최소화하고 싶은 경우
- GUI 중심으로 실습하고 싶은 경우
- 8GB 장비에서 기본 Gemma 4 실행 준비도 아직 끝나지 않은 경우
- "Gemini CLI preview만 설치하면 Gemma 4가 완전히 로컬 실행된다"는 기대가 있는 경우

8GB 장비에서도 Gemini CLI 설치 자체는 가능할 수 있지만, 행사 준비 우선순위는 `LM Studio` 또는 `Ollama`에서 작은 Gemma 4 모델을 먼저 실행해 보는 것입니다.

## 사전 요구사항

- Node.js 20 이상
- 안정적인 인터넷 연결
- 디스크 여유 공간 최소 2GB 이상
- 터미널 사용 가능 환경
- Gemini CLI 로그인 또는 API 키 설정
- Gemma Terms of Use 확인 및 동의

Gemini CLI 자체는 메인 대화에 Gemini 계정 또는 API 인증을 사용합니다. 로컬 Gemma 라우터를 설정해도 Gemini CLI의 일반 대화가 모두 오프라인으로 바뀌는 것은 아닙니다.

## 설치 버전 선택

### Gemma 4 모델 선택을 확인하려면 preview 설치

Gemini CLI에서 Gemma 4 모델 선택을 확인하려면 preview 버전을 설치합니다.

```bash
npm install -g @google/gemini-cli@preview
```

설치 확인:

```bash
gemini --version
```

이 문서는 `v0.41.0-preview.0` 이상 preview를 기준으로 합니다.

### 안정성을 우선하면 latest/stable 사용

행사 기본 실습에는 Gemini CLI가 필수가 아닙니다. Gemini CLI를 꼭 써야 하는 것이 아니라면 `LM Studio` 또는 `Ollama` 준비를 먼저 끝내세요.

Gemini CLI를 안정 경로로만 쓰고 싶다면 일반 설치를 사용할 수 있습니다.

```bash
npm install -g @google/gemini-cli@latest
```

다만 Gemma 4 모델 선택 실험 기능은 preview 기준으로 보는 것이 안전합니다.

## Gemini CLI 사전 설정

Gemma 4 preview 모델 선택을 확인하려면 설치뿐 아니라 Gemini CLI 설정도 미리 확인해야 합니다.

설정 파일 위치:

- 사용자 설정: `~/.gemini/settings.json`
- 프로젝트 설정: `your-project/.gemini/settings.json`

중요:

- 프로젝트 설정은 사용자 설정을 덮어씁니다.
- 개인 설정은 가능하면 `~/.gemini/settings.json`에 두세요.

- 프로젝트의 `.gemini/settings.json`에 실험 기능을 커밋하면 다른 사람의 환경에도 영향을 줄 수 있으므로 권장하지 않습니다.

Gemini CLI 세션 안에서 설정을 여는 방법:

```bash
gemini
```

세션 안에서:

```text
/settings
```

`Gemma Models` 또는 Gemma 관련 experimental 항목을 켭니다.

직접 설정 파일을 수정해야 한다면 `~/.gemini/settings.json`에 아래 값을 병합합니다.

```json
{
  "experimental": {
    "gemma": true
  }
}
```

이미 `settings.json`이 있다면 파일 전체를 덮어쓰지 말고, 기존 설정 안에 `experimental.gemma`만 추가하세요. 설정 변경 후에는 Gemini CLI를 다시 시작하는 것이 안전합니다.

Gemma 4 모델을 기본 모델로 고정하고 싶다면 아래처럼 `model.name`을 추가할 수 있습니다.

```json
{
  "experimental": {
    "gemma": true
  },
  "model": {
    "name": "gemma-4-26b-a4b-it"
  }
}
```

다만 호출 권한이 없는 계정에서 기본 모델을 Gemma 4로 고정하면 Gemini CLI 시작 후 계속 오류를 만날 수 있습니다. 처음에는 `/model`에서 직접 선택하거나 `gemini --model gemma-4-26b-a4b-it`로 테스트한 뒤, 실제 호출이 성공할 때만 기본 모델로 저장하세요.

로컬 Gemma 라우터 관련 설정은 `gemini gemma` 또는 `gemini gemma setup`이 처리하는 것을 우선 권장합니다. 수동 설정이 필요할 때만 `experimental.gemmaModelRouter`를 직접 편집하세요.

## 모델 선택 우선순위

Gemini CLI에서 어떤 대화 모델을 쓸지는 공식 문서 기준 아래 순서로 결정됩니다.

1. 실행할 때 넘긴 `--model` 플래그
2. `GEMINI_MODEL` 환경 변수
3. `settings.json`의 `model.name`

4 `settings.json`에서 켠 로컬 Gemma 라우터

5 기본값 `auto`

실습에서 헷갈리기 쉬운 부분은 4번입니다. 로컬 Gemma 라우터를 켰다고 해서 대화 모델이 `gemma3-1b-gpu-custom`으로 바뀌는 것은 아닙니다. `gemma3-1b-gpu-custom`은 "이번 요청을 어떤 hosted Gemini 모델로 보낼지" 판단하는 데 쓰입니다.

따라서 Gemma 4 preview 모델을 명시적으로 테스트하려면 아래처럼 직접 모델을 지정하는 편이 가장 분명합니다.

```bash
gemini --model gemma-4-26b-a4b-it
```

기존 문서나 CLI 출력에서 `-m` 축약형을 쓰는 경우도 있지만, 처음 확인할 때는 공식 문서에서 설명하는 `--model` 형태가 더 읽기 쉽습니다.

## Gemma 4 모델 선택 확인

preview 설치와 `experimental.gemma` 설정이 끝났다면 Gemma 4 모델을 직접 지정해 볼 수 있습니다.

```bash
gemini --model gemma-4-26b-a4b-it
```

또는:

```bash
gemini --model gemma-4-31b-it
```

Gemini CLI 세션 안에서는 `/model` 명령으로 선택 가능한 모델 목록을 확인해 보세요.

```text
/model
```

주의:

- 계정, API 키, 지역, preview 접근 권한에 따라 모델 호출이 실패할 수 있습니다.
- `/model` 목록에 Gemma 4가 보여도 실제 호출 권한이 있다는 뜻은 아닙니다.
- 이 경로는 `gemini gemma setup`으로 Gemma 4 모델 파일을 로컬에 내려받는 방식이 아닙니다.
- 현재 확인되는 Gemini CLI 내장 Gemma 4 모델 ID는 `gemma-4-26b-a4b-it`, `gemma-4-31b-it`입니다.
- E2B/E4B까지 CLI 모델 선택에 표시되는지는 실제 `/model` 목록을 기준으로 확인하세요.

실제 호출 권한이 없으면 아래와 비슷한 메시지가 나올 수 있습니다.

```text
It seems like you don't have access to gemma-4-26b-a4b-it.
Your admin might have disabled the access. Contact them to enable the Preview Release
Channel.
```

이 경우 Gemini CLI Gemma 4 preview는 모델 목록 확인까지만 성공한 상태입니다. 행사 준비 완료 기준으로는 `LM Studio` 또는 `Ollama`에서 Gemma 4 로컬 실행을 먼저 완료하세요.

## 로컬 Gemma 라우터 설정

Gemini CLI의 로컬 라우터도 확인하고 싶다면 아래 명령을 실행합니다.

```bash
gemini gemma
```

버전에 따라 setup 하위 명령을 안내하면 아래처럼 실행합니다.

```bash
gemini gemma setup
```

실행 중에는 LiteRT-LM 런타임과 `gemma3-1b-gpu-custom` 모델 다운로드를 안내합니다. 실행 화면에는 모델 다운로드 크기가 약 1GB로 안내될 수 있습니다.

진행 중 확인할 것:

- Gemma Terms of Use 링크를 열어 확인
- 약관 동의 여부 선택
- LiteRT-LM 런타임 다운로드
- `gemma3-1b-gpu-custom` 모델 다운로드
- 설정 완료 메시지 확인

이 단계는 행사장 네트워크에서 처음 실행하지 않는 것이 좋습니다.

공식 문서 기준으로 함께 알아둘 명령은 아래와 같습니다.

| 명령 | 하는 일 |
|---|---|
| `gemini gemma setup` | LiteRT-LM, 모델, 설정, 서버 시작까지 한 번에 처리 |
| `gemini gemma status` | 설치 상태와 서버 실행 상태 확인 |
| `gemini gemma start` | LiteRT 서버 시작 |
| `gemini gemma stop` | LiteRT 서버 중지 |
| `gemini gemma logs` | 라우팅 요청 로그 확인 |
| `/gemma` | Gemini CLI 세션 안에서 Gemma 라우터 상태 확인 |

포트나 다운로드 방식을 조정해야 할 때는 setup 플래그를 쓸 수 있습니다.

```bash
gemini gemma setup --port 8080
gemini gemma setup --no-start
gemini gemma setup --force
gemini gemma setup --skip-model
```

처음 준비한다면 플래그 없이 `gemini gemma setup`만 실행하는 쪽이 가장 단순합니다.

## 상태 확인

자동 설정 후에는 먼저 상태를 확인합니다.

```bash
gemini gemma status
```

조금 더 확실히 보려면 터미널을 두 개 열어 확인할 수 있습니다.

```bash
gemini gemma logs
```

다른 터미널에서 Gemini CLI를 평소처럼 실행합니다.

```bash
gemini
```

세션 안에서:

```text
/gemma
```

여기서 Gemma 라우터가 설정되어 있는지 확인합니다.

추가로 짧은 질문을 한 번 보내 Gemini CLI 일반 응답도 확인하세요.

```text
간단히 자기소개를 한 문장으로 해줘.
```

## 완료 기준

### Gemma 4 preview 모델 선택 완료 기준

- `gemini --version`이 `v0.41.0-preview.0` 이상 preview로 표시됨
- `experimental.gemma` 설정을 켰음
- 사용자 설정 또는 프로젝트 설정 중 어디에서 설정이 적용되는지 확인함
- Gemini CLI 세션에서 `/model` 목록을 확인함
- `gemini --model gemma-4-26b-a4b-it` 또는 `gemini --model gemma-4-31b-it`로 짧은 질문에 응답 성공

모델 목록에는 보이지만 `You don't have access` 또는 `Preview Release Channel` 관련 오류가 나오면 Gemma 4 preview 호출 준비는 미완료로 보세요.

### 로컬 Gemma 라우터 완료 기준

- `gemini gemma setup`이 끝남
- `gemma3-1b-gpu-custom` 다운로드가 끝남

- `gemini gemma status`에서 설치와 서버 상태를 확인함
- Gemini CLI 세션에서 `/gemma` 상태 확인이 됨
- 짧은 일반 질문에 Gemini CLI가 응답함

## 수동 설정이 필요한 경우

자동 설정이 실패하거나, LiteRT-LM 런타임을 직접 띄워야 하는 경우에는 공식 Local Model Routing 문서의 수동 설정을 확인하세요.

수동 설정은 아래 구조를 직접 준비하는 방식입니다.

- LiteRT-LM 런타임을 운영체제에 맞게 다운로드
- `gemma3-1b-gpu-custom` 모델을 pull
- LiteRT-LM을 예를 들어 9379 포트로 실행
- `settings.json`에 로컬 classifier 주소와 모델명을 설정

수동 설정의 핵심 설정은 아래와 같습니다.

```json
{
  "experimental": {
    "gemmaModelRouter": {
      "enabled": true,
      "classifier": {
        "host": "http://localhost:9379",
        "model": "gemma3-1b-gpu-custom"
      }
    }
  }
}
```

자동 설정을 사용했다면 이 값을 직접 만들 필요가 없을 수 있습니다. 설정 변경 후에는 Gemini CLI를 다시 시작해야 반영됩니다.

문서와 실제 CLI 동작은 버전에 따라 빠르게 바뀔 수 있으므로, 우선 `gemini gemma setup` 명령을 기준으로 따라가세요.

## 자주 생기는 오해

### preview를 설치하면 Gemma 4를 쓸 수 있나요?

Gemini CLI의 모델 선택 경로에서는 가능해진 것으로 보입니다. 다만 `experimental.gemma`를 켜야 하고, 실제 호출은 계정/키/preview 접근 권한의 영향을 받을 수 있습니다.

### `gemini gemma setup`이 Gemma 3를 받는 게 이상한가요?

아닙니다. 현재 `gemini gemma setup` 준비 흐름에서 확인되는 로컬 모델은 `gemma3-1b-gpu-custom`입니다. 이 모델은 Gemma 4 채팅 모델이 아니라, 라우팅 판단을 위한 로컬 모델로 이해해야 합니다.

### Gemini CLI preview가 Gemma 4를 완전히 오프라인으로 실행하나요?

아닙니다. preview의 Gemma 4 모델 지원은 CLI 모델 선택/호출 경로의 실험 기능으로 봐야 합니다. Gemma 4를 로컬 파일로 내려받아 오프라인 실행하려면 이 저장소의 `LM Studio`, `Ollama`, `llama.cpp` 문서를 보세요.

### 기존 기본 준비 문서 대신 이것만 하면 되나요?

아닙니다. 이 문서는 선택형 추가 준비입니다. 행사 기본 실습 준비는 `LM Studio` 또는 `Ollama`에서 Gemma 4 모델을 미리 내려받고 1회 실행해 보는 것입니다.

### 설정 파일을 프로젝트에 커밋해야 하나요?

개인 설정이면 `~/.gemini/settings.json`에 두세요. 개인 환경 설정을 프로젝트 `.gemini/settings.json`으로 커밋하는 것은 권장하지 않습니다.

## 트러블슈팅

### Gemma 4 모델이 목록에 안 보일 때

- `gemini --version`이 preview인지 확인합니다.
- `v0.41.0-preview.0` 미만이면 preview로 업데이트합니다.
- `/settings`에서 Gemma experimental 항목을 켰는지 확인합니다.
- Gemini CLI를 완전히 종료한 뒤 다시 실행합니다.

```bash
npm install -g @google/gemini-cli@preview
```

### `gemini --model gemma-4-26b-a4b-it`가 실패할 때

- `/model` 목록에 보여도 실제 호출 권한은 별도로 필요합니다.
- `It seems like you don't have access to ...` 메시지가 나오면 현재 계정 또는 조직 설정에서 해당 preview 모델 호출이 막힌 상태로 봅니다.
- `Your admin might have disabled the access. Contact them to enable the Preview Release Channel.` 메시지가 나오면 조직/관리자 계정에서 Preview Release Channel 허용이 필요할 수 있습니다.
- 개인 계정이나 별도 API 키로 다시 로그인/설정해 볼 수 있지만, 행사장에서 이 문제를 해결하려고 시간을 쓰지 않는 것을 권장합니다.
- preview 기능이므로 일시적으로 동작이 바뀔 수 있음을 감안합니다.
- 행사 실습 준비가 목적이면 `LM Studio` 또는 `Ollama`의 Gemma 4 로컬 실행을 우선하세요.

### `gemini gemma` 명령이 없다고 나올 때

- `gemini --version`으로 버전을 확인합니다.
- `v0.40.0` 미만이면 Gemini CLI를 업데이트합니다.

```bash
npm install -g @google/gemini-cli@latest
```

### `/gemma`에서 준비되지 않았다고 나올 때

- `/settings`에서 Gemma experimental 항목을 켰는지 확인합니다.
- Gemini CLI를 완전히 종료한 뒤 다시 실행합니다.
- `gemini gemma`를 다시 실행합니다.

### `gemini gemma setup`에서 Gemma 3 모델을 받는다고 나올 때

현재 정상 동작으로 보세요. 이 명령은 Gemma 4 모델 선택 기능이 아니라, LiteRT-LM 기반 로컬 라우팅 설정 경로입니다.

### 다운로드가 오래 걸릴 때

- 모델과 런타임을 행사 전에 미리 받으세요.
- 회사/학교 네트워크에서 GitHub 또는 Google 다운로드가 막혀 있으면 개인 네트워크에서 준비하세요.
- 디스크 여유 공간을 확인하세요.

### macOS에서 LiteRT-LM 실행이 막힐 때

macOS 보안 설정 때문에 다운로드한 바이너리 실행이 막힐 수 있습니다.

- `System Settings -> Privacy & Security`에서 허용
- 또는 공식 문서의 quarantine 해제 안내 확인

### Windows에서 실행이 막힐 때

- PowerShell을 새로 열어 다시 실행합니다.
- 보안 프로그램이나 방화벽이 로컬 런타임 실행을 막는지 확인합니다.
- 관리자 권한이 필요한 환경이면 설치 권한을 먼저 확인합니다.

## 참고 링크

- Gemini CLI v0.41.0-preview.0 release
- Gemma 4 support PR #25604
- Gemini CLI v0.40.0 GitHub discussion #26216
- Gemini CLI README
- Gemini CLI settings reference
- Gemini CLI model selection reference
- Gemini CLI CLI reference
- Gemini CLI Model Routing
- Gemini CLI `gemini gemma` setup
- Gemini CLI Manual Local Model Routing
- Gemma Terms of Use
- Gemma Prohibited Use Policy

문서 16

docs/16-troubleshooting-and-final-check.md

# 트러블슈팅 / 최종 체크 / 참고 링크

## 문제 발생 시 바로 해볼 것

### Ollama

- `ollama --version`이 안 되면 앱 또는 설치 상태를 다시 확인
- 모델 로드 실패 시 더 작은 모델로 변경
- `gemma4:e4b` -> `gemma4:e2b`
- 메모리 부족 시 브라우저 탭과 다른 앱 종료
- 너무 느리면 큰 모델 대신 E2B/E4B 사용

### LM Studio

- 앱이 켜지지만 모델이 안 돌면 런타임 상태 확인
- `Cmd/Ctrl + Shift + R`
- 모델 로드 실패 시 더 작은 모델 선택
- 8GB 환경이면 `google/gemma-4-e2b`만 사용
- 8GB Mac/노트북에서 시스템이 멈춘 것처럼 보이면, 잠시 기다린 뒤 앱을 종료하고 브라우저 탭과 다른 앱을 먼저 줄인 다음 다시 시도
- Intel Mac이면 LM Studio 대신 Ollama 사용

### llama.cpp

- `llama-cli --version` 또는 `llama-server --version`이 안 되면 PATH 또는 설치 상태를 다시 확인
- 모델 경로 오류가 나면 `.gguf` 절대 경로를 다시 확인
- 모델이 너무 느리거나 메모리 부족이면 더 작은 모델이나 더 작은 quant 사용
- macOS / Linux 서버 확인은 `curl http://127.0.0.1:8080/v1/models`로 먼저 점검
- Windows PowerShell 서버 확인은 `Invoke-RestMethod http://127.0.0.1:8080/v1/models`가 더 안전함

### Hermes Agent

- `hermes version`이 안 되면 셸을 다시 열거나 `source ~/.zshrc` / `source ~/.bashrc` 실행
- `hermes doctor`에서 누락 의존성이 보이면 먼저 그 결과를 기준으로 수정
- Windows 사용자는 네이티브가 아니라 WSL2 안에서 실행해야 함
- Windows에서 `hermes`가 안 잡히면 먼저 PowerShell에서 `wsl --status`, `wsl -l -v`를 확인하고, WSL2 Ubuntu 셸 안에서 다시 점검
- 로컬 모델 연결 실패 시 `hermes model`에서 base URL과 model ID를 다시 확인

### Gemini CLI Gemma 4 preview / Gemma 라우팅

- Gemma 4 모델 선택을 확인하려면 `gemini --version`으로 `v0.41.0-preview.0` 이상 preview인지 확인

- `~/.gemini/settings.json` 또는 `/settings`에서 `experimental.gemma`가 켜져 있는지 확인
- 프로젝트의 `.gemini/settings.json`이 사용자 설정을 덮어쓸 수 있으므로, 현재 프로젝트 설정도 확인
- `/model` 목록에 Gemma 4가 보여도 실제 호출 권한이 있다는 뜻은 아님
- `gemini --model gemma-4-26b-a4b-it`가 `You don't have access` 또는 `Preview Release Channel` 관련 메시지로 실패하면 계정/API 키/조직 관리자 설정에서 preview 모델 호출이 막힌 상태로 봄
- `gemini gemma` 명령이 없으면 `gemini --version`으로 v0.40.0 이상인지 확인
- `/gemma`에서 준비되지 않았다고 나오면 `/settings`에서 Gemma experimental 항목을 켠 뒤 Gemini CLI를 재시작
- `gemini gemma setup`에서 `gemma3-1b-gpu-custom`을 받는 것은 현재 정상 동작으로 보고, Gemma 4 모델 선택 기능과 구분
- 다운로드가 오래 걸리면 행사장 네트워크에서 진행하지 말고 행사 전에 미리 완료
- macOS에서 LiteRT-LM 실행이 막히면 Privacy & Security에서 허용하거나 공식 문서의 quarantine 해제 안내 확인

### ChromeOS

- Linux 개발 환경이 꺼져 있지 않은지 확인
- 학교/회사 정책으로 Linux가 막혀 있지 않은지 확인
- 로컬 LLM용 전용 GPU/VRAM 가속을 기대하기 어려우므로 성능 기대치를 낮추고 작은 모델만 사용

## 행사 전 최종 확인

아래 중 하나 이상이 되면 준비 완료입니다.

### LM Studio 준비 완료 기준

- 앱 실행 성공
- `google/gemma-4-e2b` 또는 `google/gemma-4-e4b` 계열의 chat-ready / instruction-tuned 항목 다운로드 완료
- Chat 화면에서 한 번 답변 생성 성공
- Windows에서 CLI까지 확인하고 싶다면 PowerShell에서 아래처럼 확인할 수 있습니다.

```powershell
lms --help
```

## 8GB 장비 주의사항

8GB 메모리 노트북은 작은 모델(E2B)에서도 속도가 많이 느리거나 시스템이 일시적으로 멈춘 것처럼 보일 수 있습니다.

- 가능하면 16GB 이상 메모리 장비를 권장합니다.
- 8GB 장비로 참석하는 경우에는 E2B만 미리 다운로드하세요.
- 다른 앱을 종료한 상태에서 행사 전에 반드시 1회 실행 테스트를 완료해 주세요.

### Ollama 준비 완료 기준

macOS / Linux:

```bash
ollama --version
ollama run gemma4:e2b
```

Windows PowerShell:

```powershell
ollama --version
ollama run gemma4:e2b
```

### llama.cpp 준비 완료 기준

macOS / Linux:

```bash
llama-cli --version
llama-server --version
```

Windows PowerShell:

```powershell
llama-cli --version
llama-server --version
```

그리고 아래 중 하나 이상:

- `llama-cli`로 1회 응답 생성 성공
- `llama-server` 실행 성공
- macOS / Linux: `curl http://127.0.0.1:8080/v1/models`
- Windows PowerShell: `Invoke-RestMethod http://127.0.0.1:8080/v1/models`

### Hermes 준비 완료 기준

macOS / Linux / WSL2:

```bash
hermes version
hermes doctor
```

그 다음 `hermes` 또는 `hermes --tui`를 실행해 짧은 질문이 응답되는지 확인하세요.

로컬 모델을 붙여 볼 경우에는 `hermes model`에서 로컬 서버 주소 등록까지 확인하세요.

Windows 사용자는 위 명령을 PowerShell이 아니라 WSL2 Ubuntu 셸 안에서 실행해야 합니다.

### Gemini CLI Gemma 4 preview 준비 완료 기준

macOS / Linux / Windows:

```bash
gemini --version
gemini --model gemma-4-26b-a4b-it
```

Gemini CLI 세션 안에서:

```text
/model
```

아래가 확인되면 Gemma 4 preview 준비 완료입니다.

- Gemini CLI 버전이 `v0.41.0-preview.0` 이상 preview
- `experimental.gemma` 설정 활성화
- 사용자 설정과 프로젝트 설정 중 어디에서 설정이 적용되는지 확인
- `/model`에서 Gemma 4 모델 선택 가능 여부 확인
- `gemma-4-26b-a4b-it` 또는 `gemma-4-31b-it`로 짧은 일반 질문 응답 성공

목록에만 보이고 `You don't have access` 또는 `Preview Release Channel` 관련 오류가 나오면 Gemma 4 preview 호출 준비는 미완료입니다.

### Gemini CLI Gemma 라우팅 준비 완료 기준

macOS / Linux / Windows:

```bash
gemini --version
gemini gemma setup
gemini gemma status
```

Gemini CLI 세션 안에서:

```text
/gemma
```

아래가 확인되면 준비 완료입니다.

- Gemini CLI 버전이 `v0.40.0` 이상
- `experimental.gemma` 설정 활성화
- `gemma3-1b-gpu-custom` 다운로드 완료
- `gemini gemma status` 상태 확인 성공
- `/gemma` 상태 확인 성공
- 짧은 일반 질문에 Gemini CLI가 응답

## 공식 참고 링크

### Gemma 4

- Gemma 4 출시 블로그 - Google Developers Blog

### Ollama

- Ollama Quickstart
- Ollama macOS
- Ollama Windows
- Ollama Linux
- Ollama Gemma 4 Library
- Ollama Context Length

### LM Studio

- LM Studio Download
- LM Studio Docs
- LM Studio System Requirements
- LM Studio Get Started
- LM Studio Download an LLM
- LM Studio CLI
- LM Studio Offline Operation
- LM Studio Gemma 4 Models
- LM Studio MLX Engine Blog

### llama.cpp

- llama.cpp README
- llama.cpp Install Guide
- llama.cpp Build Guide

### Hermes Agent

- Hermes Installation
- Hermes Quickstart
- Hermes CLI Commands Reference
- Hermes FAQ
- Hermes Docs Home

### uv

- uv Installation
- uv First steps
- uv Tools Guide

### 코드 편집기

- Visual Studio Code Download
- VS Code setup overview
- VS Code command line
- Google Antigravity
- Google Antigravity Download
- Getting Started with Google Antigravity - Google Codelabs

## Gemini CLI

- Gemini CLI GitHub
- Gemini CLI v0.41.0-preview.0 release
- Gemma 4 support PR #25604
- Gemini CLI v0.40.0 discussion
- Gemini CLI settings reference
- Gemini CLI model selection reference
- Gemini CLI CLI reference
- Gemini CLI Model Routing
- Gemini CLI `gemini gemma` setup
- Gemini CLI Manual Local Model Routing
- Gemma Terms of Use

## Apple MLX / Community MLX

- Apple MLX GitHub
- Apple MLX LM GitHub
- MLX LoRA / QLoRA Guide
- Hugging Face MLX Community

## ChromeOS

- Set up Linux on your Chromebook

문서 17

docs/17-faq.md

# 자주 묻는 질문

## 준비 범위

### 하나만 설치한다면 무엇을 설치하면 되나요?

대부분의 Windows, macOS Apple Silicon, Linux 사용자는 LM Studio를 먼저 설치하면 됩니다.

예외:

- ChromeOS 사용자는 Ollama를 우선 권장합니다.
- Intel Mac 사용자는 LM Studio 데스크톱 앱 대신 Ollama를 사용하세요.
- CLI/API 사용에 익숙하고 자동화까지 해 보고 싶다면 Ollama도 함께 설치하면 좋습니다.

### Ollama와 LM Studio를 둘 다 설치해야 하나요?

둘 다 설치하면 좋지만 필수는 아닙니다.

- 처음 준비하는 경우: LM Studio 하나로 충분합니다.
- 개발자이거나 터미널 사용에 익숙한 경우: Ollama도 함께 설치하면 좋습니다.
- ChromeOS: Ollama 위주로 준비하세요.

행사 당일에는 설치보다 이미 다운로드한 모델을 바로 실행할 수 있는 상태가 더 중요합니다.

### Hermes Agent, OpenCode, llama.cpp도 꼭 준비해야 하나요?

아닙니다. 세 도구는 선택 사항입니다.

- 기본 실습: LM Studio 또는 Ollama만 준비해도 됩니다.
- 코딩 에이전트 흐름을 더 실험해 보고 싶은 경우: OpenCode, Hermes Agent를 추가로 보면 됩니다.
- 로컬 서버나 GGUF 실행을 직접 다뤄 보고 싶은 경우: llama.cpp를 보면 됩니다.

터미널, WSL2, 빌드 도구에 익숙하지 않다면 행사 전에는 LM Studio 준비에 집중하는 편이 안전합니다.

### Gemini CLI Gemma 기능도 필수인가요?

필수는 아닙니다.

Gemini CLI의 Gemma 기능은 기본 로컬 실행 준비와 별도 선택 경로입니다. preview에서 Gemma 4 모델 선택을 확인할 수 있지만, 계정/API 키/조직 설정에 따라 실제 호출 권한이 없을 수 있습니다.

또한 `gemini gemma setup`은 현재 Gemma 3 기반 로컬 라우팅 설정으로 이해해야 하며, Gemma 4 모델을 로컬로 내려받는 흐름과는 구분해 주세요.

## 모델 선택

### 내 노트북 메모리를 모르겠으면 어떻게 하나요?

운영체제 설정에서 메모리를 먼저 확인하세요.

- Windows: 설정 > 시스템 > 정보
- macOS: Apple 메뉴 > 이 Mac에 관하여
- ChromeOS: 설정 > About ChromeOS 또는 기기 사양 페이지

- Linux: 터미널에서 `free -h`

모르겠거나 애매하면 더 작은 모델을 선택하세요. 행사 준비에서는 큰 모델을 억지로 실행하는 것보다 작은 모델을 안정적으로 실행하는 것이 낫습니다.

### 8GB 노트북으로도 참여할 수 있나요?

가능은 하지만 권장 사양으로 보기는 어렵습니다.

- 가능하면 16GB 이상 노트북을 준비하세요.
- 8GB 장비라면 E2B만 미리 다운로드하세요.
- 브라우저 탭과 다른 앱을 최대한 닫고 테스트하세요.
- 응답이 매우 느리거나 시스템이 멈춘 것처럼 보일 수 있습니다.

8GB 장비로 참석하는 경우에는 행사 전에 반드시 1회 실행 테스트를 끝내야 합니다.

### 16GB 노트북이면 어떤 모델을 받으면 되나요?

E4B를 권장합니다.

- LM Studio: `google/gemma-4-e4b` 계열의 chat-ready / instruction-tuned 항목
- Ollama: `gemma4:e4b`

불안하면 E2B도 함께 받아 두면 좋습니다. 현장에서 문제가 생기면 작은 모델로 바로 바꿀 수 있습니다.

### instruction-tuned 모델을 받아야 하나요?

네. 이번 핸즈온 기본 준비는 instruction-tuned / chat-ready 모델을 기준으로 보세요.

Hugging Face의 Gemma 4 공식 컬렉션을 보면 같은 크기에서도 두 계열이 따로 있습니다.

| 계열 | 예시 | 이번 핸즈온 기준 |
|---|---|---|
| base / pre-trained | `google/gemma-4-26B-A4B` | 기본 준비용으로 비추천 |
| instruction-tuned | `google/gemma-4-26B-A4B-it` | 기본 준비용으로 추천 |

`base`나 `pretrained`로 표시된 모델은 원본에 가까운 사전학습 모델입니다. 대량의 텍스트, 코드, 이미지 같은 데이터를 보고 다음 토큰을 예측하도록 학습한 모델이라 언어와 패턴은 잘 배웠지만, 바로 대화하고 지시를 따르도록 조정된 모델은 아닙니다.

그래서 base 모델은 모델 연구, 자체 파인튜닝, instruction tuning 실험을 시작할 때 의미가 있습니다. 반면 이번 핸즈온처럼 채팅, 질의응답, 지시 수행, 로컬 에이전트 흐름을 바로 실습할 때는 답변 형식이나 지시 수행이 흔들릴 수 있습니다.

`instruction-tuned` 모델은 base 모델을 바탕으로 사람이 쓰는 요청과 기대 답변 형식에 맞게 추가 훈련된 모델입니다. Google의 Gemma 실행 문서도 처음 시작할 때는 IT 모델을 추천합니다. 별도 추가 학습 없이 자연어 요청에 응답하기 좋고, LM Studio나 Ollama 같은 채팅 도구에 연결하기 쉽습니다.

모델을 직접 고를 때는 아래 표현을 확인하세요.

- `-it`
- `instruct`
- `instruction-tuned`
- `chat-ready`

Gemma 4에서는 예를 들어 아래 이름을 기준으로 보면 됩니다.

- `google/gemma-4-E2B-it`
- `google/gemma-4-E4B-it`
- `google/gemma-4-26B-A4B-it`
- `google/gemma-4-31B-it`

LM Studio에서는 모델 이름이 짧게 보일 수 있으므로 모델 설명이나 모델 카드까지 확인하세요. Ollama의 `gemma4:e2b`, `gemma4:e4b`, `gemma4:26b`, `gemma4:31b` 같은 기본 런타임 태그는 별도로 `-it`를 붙이지 않아도 됩니다.

헷갈리기 쉬운 점이 하나 더 있습니다. `-it` 여부는 모델의 훈련 목적과 동작 방식이고, Q4, Q5, Q8, NVFP4는 모델 숫자를 얼마나 작게 저장했는지에 관한 표기입니다. 즉 instruction-tuned 모델도 4-bit로 양자화될 수 있고, base 모델도 4-bit로 배포될 수 있습니다.

### 32GB 이상이면 무조건 큰 모델을 받는 게 좋나요?

아닙니다.

- 안정성과 속도 우선: E4B
- 더 큰 모델을 실험하고 싶음: 26B A4B
- 36GB 이상이고 큰 모델을 시도하고 싶음: 26B A4B 또는 31B

핸즈온에서는 모델 크기보다 실행 안정성과 응답 속도가 더 중요할 수 있습니다.

### B는 GB와 같은 뜻인가요?

아닙니다.

B는 `Billion`, 즉 10억 개를 뜻합니다. 26B는 파일 크기가 26GB라는 뜻이 아니라, 모델 내부 파라미터가 대략 260억 개 규모라는 뜻입니다.

실제 다운로드 크기와 실행 중 메모리 사용량은 양자화 방식, 실행 도구, 컨텍스트 길이에 따라 달라집니다.

### 사용자가 받는 모델은 보통 양자화된 모델인가요?

네. LM Studio나 Ollama에서 일반 사용자가 받는 로컬 실행용 모델은 보통 원본 BF16 가중치 그대로가 아니라 4-bit, 5-bit, 8-bit 등으로 양자화된 배포본입니다.

그래서 모델 이름의 26B, 31B는 파라미터 규모를 뜻하고, 다운로드 크기는 그 파라미터를 어떤 숫자 형식으로 저장했는지에 따라 달라집니다. 다만 양자화되어도 KV cache, 긴 컨텍스트, 이미지 입력, 실행 도구 오버헤드는 별도로 메모리를 사용합니다.

### 26B A4B는 4B 모델처럼 가볍게 실행되나요?

아닙니다.

A4B는 답을 만들 때 실제 계산에 참여하는 부분이 약 4B 규모라는 뜻입니다. 하지만 전체 모델 파일은 26B 규모로 메모리에 올라가야 하므로, 4B 모델처럼 가볍게 실행된다는 뜻은 아닙니다.

그래서 26B A4B는 32GB 이상 장비에서 권장합니다.

### Gemma 4는 FP32가 기본인가요?

아닙니다.

Gemma 4를 FP32 기준으로 생각하면 필요한 메모리를 너무 크게 잡게 됩니다. 공식 Transformers 설정은 BF16 계열로 보는 것이 맞고, LM Studio/Ollama에서 받는 로컬 모델은 보통 4-bit, 5-bit, 8-bit처럼 더 작게 양자화된 버전입니다.

쉽게 계산하면:

```text
FP32: 파라미터 1개 = 4 bytes
BF16/FP16: 파라미터 1개 = 2 bytes
4-bit: 파라미터 1개 = 약 0.5 bytes
```

그래서 31B 모델도 FP32로 보면 가중치만 약 124GB지만, BF16이면 약 62GB, 4-bit 양자화면 그보다 훨씬 작아집니다.

## BF16은 무엇인가요?

BF16은 `bfloat16`의 줄임말입니다. 모델 안의 숫자를 16-bit로 저장하는 형식입니다.

FP32와 비교하면 메모리를 절반만 씁니다.

```text
FP32: 4 bytes
BF16: 2 bytes
```

FP16과 BF16은 둘 다 16-bit지만, BF16은 FP32와 비슷한 넓은 숫자 범위를 유지합니다. 그래서 LLM에서는 FP32보다 가볍고 FP16보다 안정적인 경우가 많아 원본 가중치 형식으로 자주 쓰입니다.

## 모델 추론에 필요한 메모리는 어떻게 계산하나요?

대략 아래 항목을 더해서 생각하면 됩니다.

```text
추론 메모리 ≈ 모델 가중치 + KV cache + 입력/출력 처리 메모리 + 실행 도구 오버헤드
```

가장 큰 부분은 보통 모델 가중치입니다. 가중치 메모리는 아래처럼 계산합니다.

```text
가중치 메모리 ≈ 파라미터 수 x 파라미터 1개당 저장 크기
```

예를 들어 26B 모델을 단순 계산하면:

```text
BF16: 26B x 2 bytes ≈ 52GB
4-bit: 26B x 0.5 bytes ≈ 13GB
```

이 차이는 파라미터 수가 줄어서가 아니라, 파라미터 하나를 저장하는 크기가 줄어서 생깁니다.

하지만 실제 실행에는 KV cache, 컨텍스트, 실행 도구 오버헤드가 추가됩니다. 그래서 4-bit 가중치 계산값보다 실제 필요한 메모리가 더 커질 수 있습니다.

## NVIDIA GPU마다 지원하는 타입이 다른 이유는 무엇인가요?

GPU 세대마다 Tensor Core가 처리할 수 있는 숫자 형식이 다르기 때문입니다.

예를 들어 어떤 GPU는 FP16은 잘 처리하지만 BF16은 약하거나, 최신 GPU는 FP8/FP4 같은 더 작은 타입을 더 잘 처리할 수 있습니다.

중요한 점은 아래입니다.

```text
모델 파일이 4-bit로 저장되어 있음
≠
GPU가 4-bit 전용 회로로 바로 계산함
```

실제로는 4-bit 가중치를 읽은 뒤 FP16/BF16으로 풀어서 계산하는 경우도 많습니다. 그래서 같은 모델이라도 GPU 세대, 드라이버, CUDA, 실행 도구에 따라 속도와 지원 여부가 달라질 수 있습니다.

## NVFP4는 무엇인가요?

`NVFP4`는 NVIDIA의 4-bit floating point 데이터 타입입니다. NVIDIA 기술 블로그는 NVFP4를 low-precision inference를 위한 data type으로 설명하고, `Gemma-4-31B-IT-NVFP4` 모델 카드도 weights와 activations가 NVFP4 data type으로 양자화되었다고 설명합니다.

다만 NVFP4는 4-bit 값만 쓰는 단순한 숫자 형식은 아닙니다. 작은 값 묶음마다 scale을 붙여 원래 숫자 범위를 더 잘 복원하는 micro-scaled FP4 데이터 타입으로 이해하면 됩니다.

NVIDIA 설명 기준으로는:

- 4-bit 값은 `1 sign bit + 2 exponent bits + 1 mantissa bit` 구조
- 값 16개마다 FP8 scale 사용
- tensor 전체에는 FP32 scale을 한 번 더 적용
- scale overhead까지 포함하면 약 4.5 bits/value에 가까움

그래서 일반적인 4-bit 양자화보다 정확도를 더 잘 보존하면서 메모리를 크게 줄이는 것을 목표로 합니다.

정리하면:

```text
NVFP4 = 4-bit floating point data type + block scaling
+ NVIDIA 하드웨어/런타임 지원
```

입니다.

## Gemma 4 31B NVFP4는 24GB GPU에서 무조건 되나요?

무조건 된다고 보면 안 됩니다.

NVIDIA의 `Gemma-4-31B-IT-NVFP4` 모델은 31B dense 모델을 NVFP4로 줄인 고급 실험 경로입니다. 짧은 context의 everyday inference에서는 24GB GPU에서 가능하다는 사례가 있지만, 공식 모델 카드 기준으로는 vLLM, NVIDIA GPU, Linux, tensor parallel 설정 같은 조건이 붙습니다.

특히 256K full context, 이미지/비디오 입력, 큰 batch를 쓰면 KV cache와 activation 메모리가 커져 24GB를 넘길 수 있습니다.

이번 핸즈온 기준으로는:

- 기본 준비: LM Studio/Ollama에서 E2B/E4B 우선
- 고사양 실험: 26B A4B 또는 31B
- NVFP4: NVIDIA GPU와 vLLM에 익숙한 경우의 별도 고급 실험

으로 보는 것이 안전합니다.

## LM Studio에서 모델 이름에 `-it`가 안 보이면 잘못 받은 건가요?

반드시 그렇지는 않습니다.

LM Studio에서는 모델 이름이 `google/gemma-4-e4b`처럼 짧게 보일 수 있습니다. 이 경우 모델 설명이나 모델 카드에서 chat-ready, instruction-tuned, instruct 계열인지 확인하세요.

이번 핸즈온은 채팅과 실습 흐름이 중심이므로 base 모델보다는 instruction-tuned/chat-ready 모델을 권장합니다.

### Ollama에서 `gemma4:latest`를 받아도 되나요?

사전 준비용으로는 명시적 태그를 권장합니다.

- 8GB: `gemma4:e2b`
- 16GB: `gemma4:e4b`
- 32GB 이상: `gemma4:26b`
- 36GB 이상: `gemma4:26b` 또는 `gemma4:31b`

`latest`는 나중에 가리키는 모델이 바뀔 수 있으므로, 행사 준비 문서에서는 명시적 태그를 기준으로 안내합니다.

## 설치와 실행

### 행사 당일에 모델을 다운로드해도 되나요?

권장하지 않습니다.

모델 파일은 수 GB에서 20GB 안팎까지 커질 수 있습니다. 현장 네트워크 상황에 따라 다운로드만 오래 걸릴 수 있고, 60분 핸즈온 시간이 모델 다운로드로 끝날 수 있습니다.

행사 전에 아래 세 가지를 끝내 주세요.

- 도구 설치 완료
- 모델 다운로드 완료
- 최소 1회 실행 테스트 완료

### 디스크 공간은 얼마나 필요하나요?

최소 20GB, 가능하면 40GB 이상 여유 공간을 권장합니다.

여러 모델을 함께 받거나 LM Studio와 Ollama를 둘 다 쓰면 모델 파일이 각각 저장될 수 있습니다. 설치 전에 불필요한 대용량 파일을 정리해 두면 좋습니다.

### 모델을 한 번 다운로드하면 오프라인에서도 쓸 수 있나요?

대체로 가능합니다.

- LM Studio는 모델 다운로드 후 완전 오프라인 사용이 가능합니다.
- Ollama도 모델을 미리 받은 뒤에는 로컬 실행이 가능합니다.

다만 설치, 모델 검색, 최초 다운로드, 일부 런타임 업데이트는 인터넷이 필요할 수 있으므로 행사 전에 완료해 주세요.

### 준비가 끝났는지 어떻게 확인하나요?

아래 중 하나 이상이 성공하면 기본 준비는 된 상태로 봅니다.

- LM Studio: 앱 실행, 모델 다운로드, Chat 화면에서 응답 1회 성공
- Ollama: `ollama --version`, `ollama run gemma4:e2b` 또는 `ollama run gemma4:e4b` 성공
- llama.cpp: `llama-cli --version`, `llama-server --version` 확인
- Hermes Agent: `hermes version`, `hermes doctor` 확인

- Gemini CLI Gemma 4 preview: `gemini --version`, `/model`에서 모델 선택 가능 여부 확인

더 자세한 기준은 트러블슈팅 / 최종 체크 / 참고 링크를 확인해 주세요.

## Windows에서는 PowerShell과 WSL2 중 무엇을 써야 하나요?

기본 설치 확인은 PowerShell 기준으로 보면 됩니다.

- Ollama: PowerShell
- LM Studio CLI: PowerShell
- Gemini CLI: PowerShell 또는 사용 중인 터미널
- OpenCode: WSL 경로 권장
- Hermes Agent: WSL2 Ubuntu 셸 기준

Windows에서 모든 명령을 한 터미널에서 처리하려고 하면 헷갈릴 수 있습니다. 문서의 해당 도구 섹션에서 PowerShell인지 WSL2인지 먼저 확인하세요.

## ChromeOS에서도 같은 방식으로 따라가면 되나요?

ChromeOS는 예외가 많습니다.

- Linux 개발 환경을 켜야 합니다.
- 학교/회사 정책으로 Linux 사용이 막혀 있을 수 있습니다.
- 로컬 LLM용 전용 GPU/VRAM 가속을 기대하기 어렵습니다.
- LM Studio보다는 Ollama와 E2B를 권장합니다.

가능하면 Windows 또는 macOS 노트북을 함께 준비하는 편이 안전합니다.

## 모델 실행 중 노트북이 멈춘 것처럼 보이면 어떻게 하나요?

먼저 몇 분 정도 기다려 보세요. 작은 장비에서는 모델을 메모리에 올리는 동안 응답이 늦을 수 있습니다.

계속 문제가 있으면 아래 순서로 처리하세요.

1. 브라우저 탭과 다른 앱을 닫습니다.
2. 더 작은 모델로 바꿉니다.
3. 컨텍스트 길이와 동시 실행 작업을 줄입니다.
4. 가능하면 전원 어댑터를 연결합니다.
5. 그래도 어렵다면 16GB 이상 장비를 준비합니다.

## 현장에서 문제가 생기면 어떤 문서를 먼저 보면 되나요?

먼저 트러블슈팅 / 최종 체크 / 참고 링크를 보세요.

가장 빠른 복구 방법은 보통 아래 순서입니다.

1. 큰 모델 대신 E2B 또는 E4B로 낮추기
2. LM Studio 또는 Ollama 중 이미 성공한 도구로 돌아가기
3. 브라우저 탭과 무거운 앱 닫기
4. 다운로드가 안 된 모델은 현장에서 새로 받지 않기

## 기대치

### 로컬 Gemma 4가 클라우드 최신 모델만큼 빠르고 똑똑한가요?

항상 그렇지는 않습니다.

로컬 모델의 장점은 내 장비에서 직접 실행하고, 네트워크 의존성을 줄이며, 로컬 에이전트 흐름을 실험할 수 있다는 점입니다. 반면 최신 대형 클라우드 모델보다 느리거나, 긴 복잡한 작업에서 품질 차이가 날 수 있습니다.

이번 핸즈온에서는 로컬 모델을 직접 준비하고 실행하는 감각을 익히는 것을 우선합니다.

### 코딩 에이전트에 연결하면 바로 실무 수준으로 쓸 수 있나요?

작은 로컬 모델은 간단한 코드 설명, 짧은 수정, 로컬 도구 호출 실험에는 유용할 수 있습니다. 하지만 큰 코드베이스를 오래 분석하거나 복잡한 리팩터링을 안정적으로 맡기는 수준을 기대하면 무리가 있습니다.

코딩 에이전트 실험은 Gemma 4 벤치마크와 코딩 에이전트 기대치를 먼저 보고 기대치를 맞추는 것을 권장합니다.

문서 18

docs/18-uv-setup.md

# uv 설치 가이드

## 이 문서는 누구를 위한 문서인가요?

- `Hermes Agent` 설치 중 uv 관련 오류가 난 분
- Python 기반 CLI 도구를 uv 또는 uvx로 실행해 보고 싶은 분
- 행사 전에 터미널 개발 환경을 미리 정리해 두고 싶은 분

## 먼저 결론

- uv는 Python 패키지와 실행 환경을 빠르게 관리하는 도구입니다.
- uv 자체가 Gemma 4 모델을 실행하는 도구는 아닙니다.
- 이번 핸즈온에서 `LM Studio` 또는 `Ollama`만 쓴다면 uv를 따로 설치하지 않아도 됩니다.
- `Hermes Agent` 공식 설치 스크립트는 uv를 자동으로 준비합니다.
- 다만 자동 설치가 실패하거나, uvx로 Python 기반 도구를 실행하고 싶다면 아래 절차로 미리 설치해 두면 됩니다.

공식 문서:

- uv Installation

## 설치 확인

이미 설치되어 있는지 먼저 확인합니다.

```bash
uv --version
uvx --version
```

둘 중 하나가 `command not found`로 나오면 아래 운영체제별 설치 방법을 따라가세요.

## macOS / Linux / WSL2

가장 단순한 공식 standalone installer 방식입니다.

```bash
curl -LsSf https://astral.sh/uv/install.sh | sh
```

`curl`이 없는 환경에서는 `wget`을 사용할 수 있습니다.

```bash
wget -qO- https://astral.sh/uv/install.sh | sh
```

설치 후 터미널을 새로 열거나 셸 설정을 다시 읽습니다.

```bash
source ~/.zshrc
```

`bash` 사용자라면:

```bash
source ~/.bashrc
```

다시 확인합니다.

```bash
uv --version
uvx --version
```

그래도 명령을 찾지 못하면 `~/.local/bin`이 `PATH`에 들어 있는지 확인합니다.

```bash
echo $PATH
```

임시로 현재 터미널에만 추가하려면:

```bash
export PATH="$HOME/.local/bin:$PATH"
```

## macOS Homebrew

Homebrew를 쓰고 있다면 아래 방식도 가능합니다.

```bash
brew install uv
```

확인:

```bash
uv --version
uvx --version
```

## Windows PowerShell

PowerShell에서 공식 installer를 실행합니다.

```powershell
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

설치 후 PowerShell을 새로 열고 확인합니다.

```powershell
uv --version
uvx --version
```

회사 또는 학교 장비처럼 보안 정책이 엄격한 환경에서는 인터넷에서 받은 스크립트 실행이 막힐 수 있습니다. 이런 경우에는 관리자 권한, 실행 정책, 보안 솔루션 정책을 먼저 확인해야 합니다.

## Windows 패키지 매니저

`winget`을 쓰는 경우:

```powershell
winget install --id=astral-sh.uv -e
```

`Scoop`을 쓰는 경우:

```powershell
scoop install main/uv
```

확인:

```powershell
uv --version
uvx --version
```

## pipx 또는 pip로 설치

Python 환경이 이미 준비되어 있다면 `pipx`로 격리 설치할 수 있습니다.

```bash
pipx install uv
```

`pip`로도 설치할 수는 있습니다.

```bash
pip install uv
```

다만 행사 준비용으로 처음 설치한다면 standalone installer, Homebrew, winget 같은 운영체제별 설치 경로가 더 단순합니다.

## 업그레이드

standalone installer로 설치했다면:

```bash
uv self update
```

Homebrew로 설치했다면:

```bash
brew upgrade uv
```

winget으로 설치했다면:

```powershell
winget upgrade --id=astral-sh.uv -e
```

pipx로 설치했다면:

```bash
pipx upgrade uv
```

## 자동완성 설정

필수는 아니지만 터미널에서 uv 명령을 자주 쓸 계획이면 자동완성을 켤 수 있습니다.

`zsh`:

```bash
echo 'eval "$(uv generate-shell-completion zsh)"' >> ~/.zshrc
echo 'eval "$(uvx --generate-shell-completion zsh)"' >> ~/.zshrc
source ~/.zshrc
```

`bash`:

```bash
echo 'eval "$(uv generate-shell-completion bash)"' >> ~/.bashrc
echo 'eval "$(uvx --generate-shell-completion bash)"' >> ~/.bashrc
source ~/.bashrc
```

PowerShell:

```powershell
if (!(Test-Path -Path $PROFILE)) {
  New-Item -ItemType File -Path $PROFILE -Force
}
Add-Content -Path $PROFILE -Value '(& uv generate-shell-completion powershell) | Out-String
| Invoke-Expression'
Add-Content -Path $PROFILE -Value '(& uvx --generate-shell-completion powershell) |
Out-String | Invoke-Expression'
```

설정 후 새 터미널을 열어 확인하세요.

## Hermes Agent와의 관계

`Hermes Agent` 설치 스크립트는 공식 문서 기준으로 uv, Python 3.11, Node.js, `ripgrep`, `ffmpeg` 등을 자동으로 준비합니다.

따라서 보통은:

1. Hermes Agent 설치 가이드를 먼저 따라갑니다.
2. 설치 중 uv 관련 오류가 나면 이 문서로 돌아와 uv를 수동 설치합니다.
3. 이후 다시 `Hermes Agent` 설치 명령을 실행합니다.

## 행사 전 확인

uv를 직접 설치했다면 행사 전에 아래까지만 확인하면 충분합니다.

```bash
uv --version
uvx --version
```

`Hermes Agent`까지 준비하는 경우에는 이어서 아래도 확인합니다.

```bash
hermes version
hermes doctor
```

## 자주 헷갈리는 점

### uv를 설치하면 Python도 자동으로 설치되나요?

uv는 Python 버전 관리 기능을 제공하지만, 설치 방식과 명령에 따라 실제 Python 다운로드 여부는 달라집니다. 단순히 uv만 설치했다고 해서 모든 프로젝트용 Python 환경이 자동으로 준비되는 것은 아닙니다.

### uvx는 무엇인가요?

uvx는 Python 기반 command-line tool을 별도 프로젝트에 설치하지 않고 바로 실행할 때 쓰는 명령입니다. Node.js 생태계의 npx와 비슷하게 이해하면 됩니다.

### uv가 Gemma 4 모델을 다운로드하나요?

아닙니다. Gemma 4 모델 다운로드는 `LM Studio`, `Ollama`, `llama.cpp` 같은 모델 실행 도구에서 처리합니다. uv는 Python 도구와 실행 환경을 관리하는 역할입니다.

문서 19

docs/19-code-editor-setup.md

# 코드 편집기 설치 가이드: Visual Studio Code / Google Antigravity

기준 확인일: 2026-05-02

## 이 문서는 누구를 위한 문서인가요?

- 핸즈온에서 `workshop/01_starter` 코드를 직접 수정해야 하는 분
- Python 파일과 Markdown 산출물을 편하게 열어 보고 싶은 분
- `Visual Studio Code` 또는 `Google Antigravity` 중 어떤 편집기를 준비할지 고르고 싶은 분

## 먼저 결론

- 이번 핸즈온에서는 `workshop/01_starter/demo/scenarios.py`를 직접 수정합니다.
- 코드 편집기는 Visual Studio Code 또는 Google Antigravity 중 하나만 준비해도 됩니다.
- 처음 준비한다면 Visual Studio Code를 권장합니다.
- AI agent 기반 코드 수정까지 실험해 보고 싶다면 Google Antigravity를 선택할 수 있습니다.
- Google Antigravity는 preview 성격의 도구이며, 공식 codelab 기준 개인 Gmail 계정과 Chrome 브라우저가 필요합니다.

선택 기준:

| 상황 | 권장 편집기 |
|---|---|
| 처음 코드를 수정해 보는 경우 | Visual Studio Code |
| 설치와 사용 흐름을 안정적으로 가져가고 싶은 경우 | Visual Studio Code |
| VS Code에 이미 익숙한 경우 | Visual Studio Code |
| agent가 계획, 코드 수정, 브라우저 검증까지 하는 흐름을 보고 싶은 경우 | Google Antigravity |
| 개인 Gmail 계정 사용이 어렵거나 회사/학교 계정만 있는 경우 | Visual Studio Code |

## 공통 준비

핸즈온 코드는 아래 위치에서 시작합니다.

text
```
workshop/01_starter
```

실습에서 주로 수정하는 파일은 아래입니다.

text
```
workshop/01_starter/demo/scenarios.py
```

터미널에서 저장소 루트에 있다면 아래처럼 열면 됩니다.

```bash
cd workshop/01_starter
```

또는 편집기에서 `workshop/01_starter` 폴더를 직접 열어도 됩니다.

## Visual Studio Code 설치

공식 문서:

- Visual Studio Code Download
- VS Code setup overview
- VS Code on Windows
- VS Code on macOS
- VS Code on Linux
- VS Code command line

### Windows

가장 쉬운 방법은 공식 설치 프로그램을 사용하는 것입니다.

1. VS Code Download에 접속합니다.
2. Windows용 installer를 다운로드합니다.
3. `VSCodeUserSetup-{version}.exe`를 실행합니다.
4. 설치가 끝나면 PowerShell 또는 명령 프롬프트를 새로 엽니다.
5. 아래 명령으로 확인합니다.

```powershell
code --version
```

설치 프로그램은 보통 `code` 명령을 `PATH`에 추가합니다. 설치 직후 `code` 명령이 안 되면 PowerShell을 완전히 닫았다가 다시 여세요.

### macOS

1. VS Code Download에 접속합니다.
2. macOS용 `.dmg` 파일을 다운로드합니다.
3. `.dmg`를 열고 `Visual Studio Code.app`을 `Applications` 폴더로 옮깁니다.
4. `Applications` 폴더에서 VS Code를 실행합니다.

터미널에서 `code .`로 열고 싶다면 VS Code 안에서 아래 작업을 한 번 해 주세요.

1. VS Code 실행
2. `Cmd + Shift + P`
3. `Shell Command: Install 'code' command in PATH` 실행

4 터미널을 새로 열기

확인:

```bash
code --version
```

Homebrew를 쓰는 경우에는 아래 방식도 가능합니다.

```bash
brew install --cask visual-studio-code
```

### Linux

Ubuntu/Debian 계열에서는 공식 `.deb` 패키지를 받아 설치하는 방식이 가장 단순합니다.

1 VS Code Download에 접속합니다.

2 Linux용 `.deb` 패키지를 다운로드합니다.

3 다운로드한 파일이 있는 폴더에서 아래처럼 설치합니다.

```bash
sudo apt install ./<file>.deb
```

설치 확인:

```bash
code --version
```

Linux 배포판마다 패키지 형식과 업데이트 방식이 다를 수 있으므로, Fedora/RHEL 계열이나 다른 배포판은 공식 Linux 설치 문서를 확인하세요.

## VS Code에서 핸즈온 코드 열기

저장소 루트에서:

```bash
code workshop/01_starter
```

Windows PowerShell에서도 같은 방식으로 열 수 있습니다.

```powershell
code .\workshop\01_starter
```

VS Code가 열리면 아래 파일을 확인합니다.

```text
demo/scenarios.py
```

권장 확장:

- `Python`
- `Markdown Preview Mermaid Support` 또는 Markdown preview 관련 확장
- `GitHub Copilot`은 선택 사항입니다.

확장 설치가 부담스럽다면 Python 확장만 설치해도 이번 핸즈온 진행에는 충분합니다.

## Google Antigravity 설치

공식 문서와 codelab:

- Google Antigravity
- Google Antigravity Download
- Getting Started with Google Antigravity - Google Codelabs
- Google Antigravity Docs

### 먼저 알아둘 점

- Google Antigravity는 agent-first 개발 환경입니다.
- VS Code 기반 편집 경험과 별도 Agent Manager 흐름을 함께 제공합니다.
- 공식 codelab 기준 현재 preview로 제공되며, 개인 Gmail 계정으로 시작할 수 있습니다.
- Chrome 브라우저가 필요합니다.
- 회사 또는 학교 Google Workspace 계정은 preview 정책에 따라 제한될 수 있으므로, 행사 전 개인 Gmail 계정으로 로그인이 되는지 확인하세요.

### 설치 순서

1. Google Antigravity Download에 접속합니다.
2. 본인 운영체제에 맞는 설치 파일을 다운로드합니다.
3. 설치 파일을 실행합니다.
4. 앱을 실행하고 초기 설정을 진행합니다.
5. 개인 Gmail 계정으로 로그인합니다.
6. 설정 중 `Command Line` 또는 `agy` command-line tool 설치 옵션이 보이면 켜 둡니다.
7. agent 설정은 처음에는 `Review-driven development`에 가까운 안전한 설정을 권장합니다.

운영체제별 설치 파일 이름과 세부 UI는 preview 기간에 바뀔 수 있습니다. 행사 전에는 반드시 공식 다운로드 페이지에서 최신 안내를 확인하세요.

### Antigravity에서 핸즈온 코드 열기

앱에서 직접 여는 방식:

1. Antigravity 실행

2. `Open Folder` 선택
3. 저장소의 `workshop/01_starter` 폴더 선택
4. `demo/scenarios.py` 열기

`agy` 명령을 설치했다면 아래 방식도 사용할 수 있습니다.

```bash
cd workshop/01_starter
agy .
```

`agy` 명령이 안 되면 앱에서 `Open Folder`를 사용하면 됩니다. CLI 옵션이 궁금하면 아래처럼 확인하세요.

```bash
agy --help
```

## 행사 전 확인

Visual Studio Code를 선택했다면:

```bash
code --version
code workshop/01_starter
```

Google Antigravity를 선택했다면:

- 앱 실행 성공
- 개인 Gmail 계정 로그인 성공
- `workshop/01_starter` 폴더 열기 성공
- `demo/scenarios.py` 파일 열기 성공
- 가능하면 `Review-driven development` 방식으로 agent 권한 설정

## 어떤 편집기를 준비해야 하나요?

처음 준비한다면 VS Code가 더 안정적입니다. 설치가 쉽고, 운영체제별 문서가 잘 정리되어 있으며, 핸즈온에서 필요한 직접 코드 수정 흐름에 충분합니다.

Antigravity는 agent 기반 개발 흐름을 보여 주기에 좋지만, preview 정책, 계정 로그인, 브라우저 extension, agent 권한 설정 같은 준비 요소가 더 많습니다. 이미 VS Code가 준비되어 있다면 Antigravity는 선택 실험 경로로 보면 됩니다.